In [1]:
```
!python --version
```

```
Python 3.12.7
```

In [2]:
```
# !pip install matplotlib
# !pip install seaborn
# !pip install scipy
# !pip install scikit-learn
```

In [3]:
```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations
from sklearn.linear_model import LinearRegression
from scipy import stats
import warnings

# Alias for stats to avoid confusion later
from scipy import stats as scipy_stats

# Suppress warnings to keep output clean
warnings.filterwarnings('ignore')

# ----------------------------
# Load the dataset
# ----------------------------
df = pd.read_csv('FE-GWP1_model_selection_1.csv')

print("Dataset loaded successfully!")
print(f"Shape of the dataset: {df.shape}")
print("\nHere are the first few rows:")
print(df.head())

# ----------------------------
# Define predictors and response
# ----------------------------
X = df[['X1', 'X2', 'X3', 'X4', 'X5']]  # predictor variables
y = df['Y']                              # dependent variable

# Number of observations
n = len(y)

# List of predictor names for convenience
predictors = ['X1', 'X2', 'X3', 'X4', 'X5']
```

```
Dataset loaded successfully!
Shape of the dataset: (100, 6)

Here are the first few rows:
          Y        X1        X2        X3        X4        X5
0  3.388410  0.017954 -0.800583 -0.352454  2.187210  1.014887
1  0.287191  0.083057 -0.597947 -0.357639 -1.630284  0.221841
2  3.989645 -0.923437 -1.386575  1.180202  0.632606 -1.576638
3 -2.959602 -0.313775  2.955133 -1.798692 -2.117621  0.159291
4  0.529773  0.388996  1.019611  0.472062  0.590497  0.877048
```

In [4]:
```
np.random.seed(77777)
```

In [5]:
```
arparams1 = np.r_[1, -np.array([0.6, 0.3])]
maparams1 = np.r_[1, np.array([0.0, 0.0])]
```

In [6]:
```
arparams1
```

Out[6]:
```
array([ 1. , -0.6, -0.3])
```

In [7]:
```
maparams1
```

Out[7]:
```
array([1., 0., 0.])
```

In [8]:
```
# ---------------------------
# Descriptive statistics
# ---------------------------
print("Some basic descriptive statistics for the dataset:")
print(df.describe())

# ---------------------------
# Correlation analysis
# ---------------------------
corr_matrix = df.corr()

print("\nCorrelation matrix for all variables:")
print(corr_matrix)

print("\nCorrelations of each predictor with Y (sorted):")
print(corr_matrix['Y'].sort_values(ascending=False))

# ---------------------------
# Visualize correlation matrix
# ---------------------------
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1, fmt='.3f')
plt.title('Correlation Matrix', fontsize=14, fontweight='bold')
plt.show()
```

Some basic descriptive statistics for the dataset:

	Y	X1	X2	X3	X4	X5
count	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000
mean	1.257388	0.026830	0.084613	-0.016037	0.122374	-0.201661
std	1.436655	0.481708	0.962145	0.976559	1.076935	1.073358
min	-2.959602	-1.275230	-2.041959	-2.228483	-2.697316	-3.526357
25%	0.349470	-0.316585	-0.676951	-0.572893	-0.431023	-0.960214
50%	1.278526	0.040237	0.040447	0.056879	0.279594	-0.185795
75%	2.152954	0.373899	0.748166	0.636408	0.747868	0.506592
max	5.176803	1.083062	2.955133	1.816825	3.167298	2.499820

Correlation matrix for all variables:

	Y	X1	X2	X3	X4	X5
Y	1.000000	-0.061400	-0.419326	0.411351	0.538308	-0.279807
X1	-0.061400	1.000000	0.103063	0.062876	-0.050210	0.061230
X2	-0.419326	0.103063	1.000000	-0.065582	-0.004993	-0.005223
X3	0.411351	0.062876	-0.065582	1.000000	-0.038886	-0.178590
X4	0.538308	-0.050210	-0.004993	-0.038886	1.000000	-0.125909
X5	-0.279807	0.061230	-0.005223	-0.178590	-0.125909	1.000000

Correlations of each predictor with Y (sorted):

```
Y     1.000000
X4    0.538308
X3    0.411351
X1   -0.061400
X5   -0.279807
X2   -0.419326
Name: Y, dtype: float64
```



In [9]:
```
def calculate_model_stats(X_subset, y, predictor_names):
    """
    Fit a linear regression model on the given predictors and return
    key statistics useful for model comparison.
    """

    n = len(y)                  # number of observations
    k = len(predictor_names)    # number of predictors in this model

    # Fit the linear regression model
    model = LinearRegression()
    model.fit(X_subset, y)

    # Get model predictions and residuals
    y_pred = model.predict(X_subset)
    residuals = y - y_pred

    # Compute total and residual sum of squares
    ss_total = np.sum((y - np.mean(y)) ** 2)
    ss_residual = np.sum(residuals ** 2)

    # R-squared
    r_squared = 1 - ss_residual / ss_total

    # Adjusted R-squared and mean squared error (MSE)
    # The check below is just for safety
    if n - k - 1 <= 0:
        adj_r_squared = np.nan
        mse = np.nan
    else:
        adj_r_squared = 1 - (1 - r_squared) * (n - 1) / (n - k - 1)
        mse = ss_residual / (n - k - 1)

    # Akaike and Bayesian information criteria
    aic = n * np.log(ss_residual / n) + 2 * (k + 1)
    bic = n * np.log(ss_residual / n) + (k + 1) * np.log(n)

    # Return all the important stats as a dictionary
    return {
        "predictors": predictor_names,
        "k": k,
        "r_squared": r_squared,
        "adj_r_squared": adj_r_squared,
        "aic": aic,
        "bic": bic,
        "mse": mse,
        "ss_residual": ss_residual,
        "coefficients": model.coef_,
        "intercept": model.intercept_,
        "y_pred": y_pred,
        "residuals": residuals
    }
```

In [10]:
```
print("\n" + "=" * 80)
print("EXHAUSTIVE SEARCH - CHECKING ALL POSSIBLE MODELS")
print("=" * 80)

# We'll store stats for every model we test
all_models = []
```

```
# Try all combinations of predictors, from 1 up to all of them
for i in range(1, len(predictors) + 1):
    for combo in combinations(predictors, i):
        cols = list(combo)
        X_subset = X[cols]

        # Get stats for this combination
        model_stats = calculate_model_stats(X_subset, y, cols)
        all_models.append(model_stats)

# Grab stats for the full model (needed for Mallows' Cp)
stats_full = [m for m in all_models if m["k"] == len(predictors)][0]
full_model_mse = stats_full["mse"]

# Calculate Mallows' Cp for all models
for m in all_models:
    k = m["k"]
    # Cp = SSE_model / MSE_full - (n - 2*(p+1))
    cp = m["ss_residual"] / full_model_mse - (n - 2 * (k + 1))
    m["mallows_cp"] = cp

# Build a summary table for easier viewing
summary_rows = []
for m in all_models:
    summary_rows.append({
        "Model": " + ".join(m["predictors"]),
        "k": m["k"],
        "R²": m["r_squared"],
        "Adj R²": m["adj_r_squared"],
        "AIC": m["aic"],
        "BIC": m["bic"],
        "Mallows Cp": m["mallows_cp"],
        "MSE": m["mse"]
    })

summary_df = pd.DataFrame(summary_rows)

print(f"\nTotal number of models evaluated: {len(all_models)}")

# Display top models by various criteria
print("\nTop 5 models by Adjusted R²:")
print(summary_df.nlargest(5, "Adj R²").to_string(index=False))

print("\nTop 5 models by AIC (lower is better):")
print(summary_df.nsmallest(5, "AIC").to_string(index=False))

print("\nTop 5 models by BIC (lower is better):")
print(summary_df.nsmallest(5, "BIC").to_string(index=False))

print("\nTop 5 models by Mallows' Cp (closer to k+1 is better):")
summary_df["Cp_distance"] = np.abs(summary_df["Mallows Cp"] - (summary_df["k"] +
print(
    summary_df
    .nsmallest(5, "Cp_distance")[["Model", "k", "R²", "Adj R²", "AIC", "BIC", "M
    .to_string(index=False)
)

# Save the summary to a CSV so we can check it later
summary_df.to_csv("all_models_summary.csv", index=False)
```

```
================================================================================
EXHAUSTIVE SEARCH - CHECKING ALL POSSIBLE MODELS
================================================================================

Total number of models evaluated: 31

Top 5 models by Adjusted R²:
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
MSE
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709 0.7
55469
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000 0.7
63321
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179 0.7
91247
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759 0.7
98778
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001 1.0
42729

Top 5 models by AIC (lower is better):
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
MSE
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709 0.7
55469
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000 0.7
63321
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179 0.7
91247
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759 0.7
98778
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001 1.0
42729

Top 5 models by BIC (lower is better):
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
MSE
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709 0.7
55469
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179 0.7
91247
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000 0.7
63321
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759 0.7
98778
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001 1.0
42729

Top 5 models by Mallows' Cp (closer to k+1 is better):
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001
```

In [11]:
```python
def forward_selection(X, y, predictors, criterion='adj_r_squared'):
    """
    Forward stepwise selection.
    Starts with no predictors and adds one at a time,
```

```
    choosing the variable that best improves the model.
    """
    remaining = list(predictors)
    selected = []
    history = []

    print(f"\nStarting Forward Selection using {criterion.replace('_', ' ').uppe
    print("-" * 80)

    while remaining:
        best_score = -np.inf if criterion == 'adj_r_squared' else np.inf
        best_var = None
        best_stats = None

        for var in remaining:
            trial_vars = selected + [var]
            X_subset = X[trial_vars]
            stats = calculate_model_stats(X_subset, y, trial_vars)

            # Compute Mallows' Cp for this subset
            k = stats['k']
            stats['mallows_cp'] = stats['ss_residual'] / full_model_mse - (n - 2

            score = stats[criterion]

            if (criterion == 'adj_r_squared' and score > best_score) or \
               (criterion in ['aic', 'bic'] and score < best_score):
                best_score = score
                best_var = var
                best_stats = stats

        # Stop if adding a variable doesn't improve Adjusted R²
        if selected and criterion == 'adj_r_squared' and best_score <= history[-
            print(f"\nStopping: Adding {best_var} does not improve Adjusted R²."
            break

        if best_var is None:
            break

        selected.append(best_var)
        remaining.remove(best_var)
        history.append(best_stats)

        print(f"\nStep {len(selected)}: Added {best_var}")
        print(f"  Current model: {' + '.join(selected)}")
        print(f"  R² = {best_stats['r_squared']:.6f}, Adj R² = {best_stats['adj_
        print(f"  AIC = {best_stats['aic']:.4f}, BIC = {best_stats['bic']:.4f}")
        print(f"  Mallows' Cp = {best_stats['mallows_cp']:.4f}")

    return selected, history


def backward_elimination(X, y, predictors, criterion='adj_r_squared'):
    """
    Backward stepwise elimination.
    Starts with all predictors and removes the least useful one at a time.
    """
    selected = list(predictors)
    history = []
```

```
    print(f"\nStarting Backward Elimination using {criterion.replace('_', ' ').u
    print("-" * 80)

    # Initial full model stats
    stats_full_model = calculate_model_stats(X[selected], y, selected)
    k = stats_full_model['k']
    stats_full_model['mallows_cp'] = stats_full_model['ss_residual'] / full_mode
    history.append(stats_full_model)

    print(f"\nInitial model: {' + '.join(selected)}")
    print(f"  R² = {stats_full_model['r_squared']:.6f}, Adj R² = {stats_full_mod
    print(f"  AIC = {stats_full_model['aic']:.4f}, BIC = {stats_full_model['bic'
    print(f"  Mallows' Cp = {stats_full_model['mallows_cp']:.4f}")

    while len(selected) > 1:
        worst_score = np.inf if criterion in ['aic', 'bic'] else -np.inf
        worst_var = None
        best_stats_after_removal = None

        for var in selected:
            trial_vars = [p for p in selected if p != var]
            stats = calculate_model_stats(X[trial_vars], y, trial_vars)
            k = stats['k']
            stats['mallows_cp'] = stats['ss_residual'] / full_model_mse - (n - 2
            score = stats[criterion]

            if (criterion == 'adj_r_squared' and score > worst_score) or \
               (criterion in ['aic', 'bic'] and score < worst_score):
                worst_score = score
                worst_var = var
                best_stats_after_removal = stats

        # Check if removal worsens the model
        if criterion == 'adj_r_squared' and best_stats_after_removal['adj_r_squa
            print(f"\nStopping: Removing {worst_var} decreases Adjusted R².")
            break
        if criterion == 'aic' and best_stats_after_removal['aic'] > history[-1][
            print(f"\nStopping: Removing {worst_var} increases AIC.")
            break
        if criterion == 'bic' and best_stats_after_removal['bic'] > history[-1][
            print(f"\nStopping: Removing {worst_var} increases BIC.")
            break

        selected.remove(worst_var)
        history.append(best_stats_after_removal)

        print(f"\nStep {len(history)}: Removed {worst_var}")
        print(f"  Current model: {' + '.join(selected)}")
        print(f"  R² = {best_stats_after_removal['r_squared']:.6f}, Adj R² = {be
        print(f"  AIC = {best_stats_after_removal['aic']:.4f}, BIC = {best_stats
        print(f"  Mallows' Cp = {best_stats_after_removal['mallows_cp']:.4f}")

    return selected, history
```

In [12]:

```
print("\n" + "="*80)
print("STEPWISE SELECTION EXECUTION")
print("="*80)

# ---------- Forward Selection ----------
print("\nRunning forward selection using different criteria...\n")
```

```
# Forward selection using Adjusted R²
selected_fwd_adj_r2, history_fwd_adj_r2 = forward_selection(
    X, y, predictors, 'adj_r_squared'
)

# Forward selection using AIC
selected_fwd_aic, history_fwd_aic = forward_selection(
    X, y, predictors, 'aic'
)

# Forward selection using BIC
selected_fwd_bic, history_fwd_bic = forward_selection(
    X, y, predictors, 'bic'
)

print("\n" + "="*80)
print("BACKWARD ELIMINATION EXECUTION")
print("="*80)

# ---------- Backward Elimination ----------
print("\nRunning backward elimination using different criteria...\n")

# Backward elimination using Adjusted R²
selected_back_adj_r2, history_back_adj_r2 = backward_elimination(
    X, y, predictors, 'adj_r_squared'
)

# Backward elimination using AIC
selected_back_aic, history_back_aic = backward_elimination(
    X, y, predictors, 'aic'
)

# Backward elimination using BIC
selected_back_bic, history_back_bic = backward_elimination(
    X, y, predictors, 'bic'
)
```

```
================================================================================
STEPWISE SELECTION EXECUTION
================================================================================

Running forward selection using different criteria...


Starting Forward Selection using ADJ R SQUARED
--------------------------------------------------------------------------------

Step 1: Added X4
  Current model: X4
  R² = 0.289776, Adj R² = 0.282529
  AIC = 41.2410, BIC = 46.4513
  Mallows' Cp = 94.1202

Step 2: Added X3
  Current model: X4 + X3
  R² = 0.476928, Adj R² = 0.466143
  AIC = 12.6548, BIC = 20.4703
  Mallows' Cp = 46.0213

Step 3: Added X2
  Current model: X4 + X3 + X2
  R² = 0.628256, Adj R² = 0.616639
  AIC = -19.4967, BIC = -9.0760
  Mallows' Cp = 7.5122

Step 4: Added X5
  Current model: X4 + X3 + X2 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Adding X1 does not improve Adjusted R².

Starting Forward Selection using AIC
--------------------------------------------------------------------------------

Step 1: Added X4
  Current model: X4
  R² = 0.289776, Adj R² = 0.282529
  AIC = 41.2410, BIC = 46.4513
  Mallows' Cp = 94.1202

Step 2: Added X3
  Current model: X4 + X3
  R² = 0.476928, Adj R² = 0.466143
  AIC = 12.6548, BIC = 20.4703
  Mallows' Cp = 46.0213

Step 3: Added X2
  Current model: X4 + X3 + X2
  R² = 0.628256, Adj R² = 0.616639
  AIC = -19.4967, BIC = -9.0760
  Mallows' Cp = 7.5122

Step 4: Added X5
  Current model: X4 + X3 + X2 + X5
  R² = 0.648763, Adj R² = 0.633974
```

```
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Step 5: Added X1
  Current model: X4 + X3 + X2 + X5 + X1
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Starting Forward Selection using BIC
--------------------------------------------------------------------------------

Step 1: Added X4
  Current model: X4
  R² = 0.289776, Adj R² = 0.282529
  AIC = 41.2410, BIC = 46.4513
  Mallows' Cp = 94.1202

Step 2: Added X3
  Current model: X4 + X3
  R² = 0.476928, Adj R² = 0.466143
  AIC = 12.6548, BIC = 20.4703
  Mallows' Cp = 46.0213

Step 3: Added X2
  Current model: X4 + X3 + X2
  R² = 0.628256, Adj R² = 0.616639
  AIC = -19.4967, BIC = -9.0760
  Mallows' Cp = 7.5122

Step 4: Added X5
  Current model: X4 + X3 + X2 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Step 5: Added X1
  Current model: X4 + X3 + X2 + X5 + X1
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

================================================================================
BACKWARD ELIMINATION EXECUTION
================================================================================

Running backward elimination using different criteria...


Starting Backward Elimination using ADJ R SQUARED
--------------------------------------------------------------------------------

Initial model: X1 + X2 + X3 + X4 + X5
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Step 2: Removed X1
  Current model: X2 + X3 + X4 + X5
  R² = 0.648763, Adj R² = 0.633974
```

```
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Removing X5 decreases Adjusted R².

Starting Backward Elimination using AIC
--------------------------------------------------------------------------------

Initial model: X1 + X2 + X3 + X4 + X5
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Step 2: Removed X1
  Current model: X2 + X3 + X4 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Removing X5 increases AIC.

Starting Backward Elimination using BIC
--------------------------------------------------------------------------------

Initial model: X1 + X2 + X3 + X4 + X5
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Step 2: Removed X1
  Current model: X2 + X3 + X4 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Removing X5 increases BIC.
```

In [13]:

```python
print("\n" + "="*80)
print("SUMMARY OF MODEL SELECTION RESULTS")
print("="*80)

print("\n1. FORWARD SELECTION RESULTS:")
print(f"   - Best model by Adj R²: {' + '.join(selected_fwd_adj_r2)}")
print(f"   - Best model by AIC: {' + '.join(selected_fwd_aic)}")
print(f"   - Best model by BIC: {' + '.join(selected_fwd_bic)}")

print("\n2. BACKWARD ELIMINATION RESULTS:")
print(f"   - Best model by Adj R²: {' + '.join(selected_back_adj_r2)}")
print(f"   - Best model by AIC: {' + '.join(selected_back_aic)}")
print(f"   - Best model by BIC: {' + '.join(selected_back_bic)}")

print("\n3. EXHAUSTIVE SEARCH RESULTS:")
best_adj_r2_model = summary_df.loc[summary_df['Adj R²'].idxmax()]
best_aic_model = summary_df.loc[summary_df['AIC'].idxmin()]
best_bic_model = summary_df.loc[summary_df['BIC'].idxmin()]
best_cp_model = summary_df.loc[summary_df['Cp_distance'].idxmin()]

print(f"   - Best model by Adj R²: {best_adj_r2_model['Model']}")
print(f"   - Best model by AIC: {best_aic_model['Model']}")
print(f"   - Best model by BIC: {best_bic_model['Model']}")
```

```
print(f"   - Best model by Mallows' Cp: {best_cp_model['Model']}")

# Create visualization of model selection paths
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Adj R² progression
ax1 = axes[0, 0]
forward_adj_r2_values = [h['adj_r_squared'] for h in history_fwd_adj_r2]
backward_adj_r2_values = [h['adj_r_squared'] for h in history_back_adj_r2]
ax1.plot(range(1, len(forward_adj_r2_values) + 1), forward_adj_r2_values,
         'o-', label='Forward Selection', linewidth=2, markersize=8)
ax1.plot(range(len(backward_adj_r2_values), 0, -1), backward_adj_r2_values,
         's-', label='Backward Elimination', linewidth=2, markersize=8)
ax1.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax1.set_ylabel('Adjusted R²', fontsize=11, fontweight='bold')
ax1.set_title('Adjusted R² by Model Size', fontsize=12, fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: AIC progression
ax2 = axes[0, 1]
forward_aic_values = [h['aic'] for h in history_fwd_aic]
backward_aic_values = [h['aic'] for h in history_back_aic]
ax2.plot(range(1, len(forward_aic_values) + 1), forward_aic_values,
         'o-', label='Forward Selection', linewidth=2, markersize=8)
ax2.plot(range(len(backward_aic_values), 0, -1), backward_aic_values,
         's-', label='Backward Elimination', linewidth=2, markersize=8)
ax2.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax2.set_ylabel('AIC', fontsize=11, fontweight='bold')
ax2.set_title('AIC by Model Size', fontsize=12, fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)

# Plot 3: BIC progression
ax3 = axes[1, 0]
forward_bic_values = [h['bic'] for h in history_fwd_bic]
backward_bic_values = [h['bic'] for h in history_back_bic]
ax3.plot(range(1, len(forward_bic_values) + 1), forward_bic_values,
         'o-', label='Forward Selection', linewidth=2, markersize=8)
ax3.plot(range(len(backward_bic_values), 0, -1), backward_bic_values,
         's-', label='Backward Elimination', linewidth=2, markersize=8)
ax3.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax3.set_ylabel('BIC', fontsize=11, fontweight='bold')
ax3.set_title('BIC by Model Size', fontsize=12, fontweight='bold')
ax3.legend()
ax3.grid(True, alpha=0.3)

# Plot 4: Comparison of criteria for all models
ax4 = axes[1, 1]
models_by_size = summary_df.groupby('k').agg({
    'Adj R²': 'max',
    'AIC': 'min',
    'BIC': 'min'
}).reset_index()

ax4_twin1 = ax4.twinx()
ax4_twin2 = ax4.twinx()
ax4_twin2.spines['right'].set_position(('outward', 60))

p1 = ax4.plot(models_by_size['k'], models_by_size['Adj R²'],
```

```
                'o-', color='blue', label='Max Adj R²', linewidth=2, markersize=8)
p2 = ax4_twin1.plot(models_by_size['k'], models_by_size['AIC'],
                    's-', color='red', label='Min AIC', linewidth=2, markersize=
p3 = ax4_twin2.plot(models_by_size['k'], models_by_size['BIC'],
                    '^-', color='green', label='Min BIC', linewidth=2, markersiz

ax4.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax4.set_ylabel('Adjusted R²', color='blue', fontsize=11, fontweight='bold')
ax4_twin1.set_ylabel('AIC', color='red', fontsize=11, fontweight='bold')
ax4_twin2.set_ylabel('BIC', color='green', fontsize=11, fontweight='bold')
ax4.set_title('Comparison of Selection Criteria', fontsize=12, fontweight='bold'

ax4.tick_params(axis='y', labelcolor='blue')
ax4_twin1.tick_params(axis='y', labelcolor='red')
ax4_twin2.tick_params(axis='y', labelcolor='green')

lines = p1 + p2 + p3
labels = [l.get_label() for l in lines]
ax4.legend(lines, labels, loc='best')
ax4.grid(True, alpha=0.3)

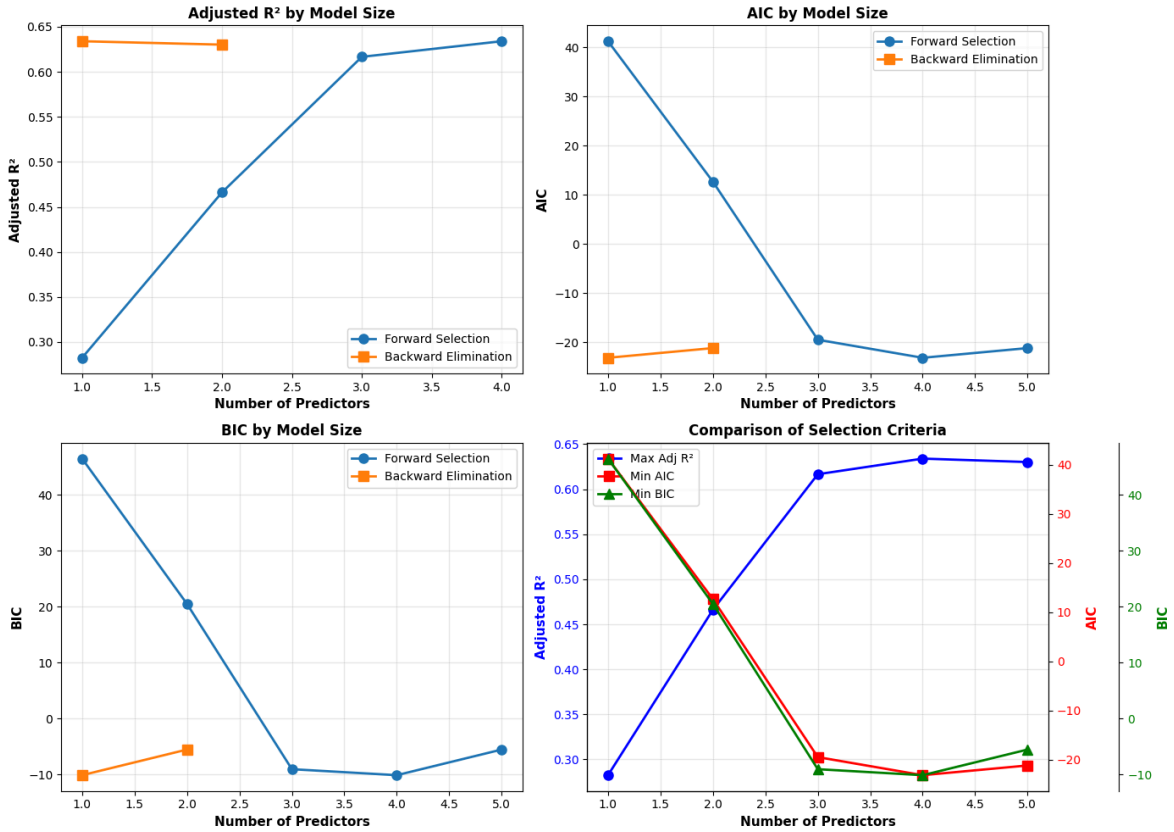
plt.tight_layout()
plt.show()
```

```
================================================================================
SUMMARY OF MODEL SELECTION RESULTS
================================================================================

1. FORWARD SELECTION RESULTS:
   - Best model by Adj R²: X4 + X3 + X2 + X5
   - Best model by AIC: X4 + X3 + X2 + X5 + X1
   - Best model by BIC: X4 + X3 + X2 + X5 + X1

2. BACKWARD ELIMINATION RESULTS:
   - Best model by Adj R²: X2 + X3 + X4 + X5
   - Best model by AIC: X2 + X3 + X4 + X5
   - Best model by BIC: X2 + X3 + X4 + X5

3. EXHAUSTIVE SEARCH RESULTS:
   - Best model by Adj R²: X2 + X3 + X4 + X5
   - Best model by AIC: X2 + X3 + X4 + X5
   - Best model by BIC: X2 + X3 + X4 + X5
   - Best model by Mallows' Cp: X1 + X2 + X3 + X4 + X5
```



In [14]:
```
print("\n" + "=" * 80)
print("FINAL MODEL DIAGNOSTICS AND COMPARISON")
print("=" * 80)

# recommended model vs full model
rec_vars = ["X2", "X3", "X4", "X5"]
full_vars = ["X1", "X2", "X3", "X4", "X5"]

stats_rec = calculate_model_stats(X[rec_vars], y, rec_vars)
stats_full = calculate_model_stats(X[full_vars], y, full_vars)

# keep fitted values and residuals handy
y_hat_rec = stats_rec["y_pred"]
resid_rec = stats_rec["residuals"]

y_hat_full = stats_full["y_pred"]
resid_full = stats_full["residuals"]

# ------------------------------------------------
# Recommended model output
# ------------------------------------------------
print("\nRECOMMENDED MODEL: Y ~ X2 + X3 + X4 + X5")
print("-" * 80)
print(f"Intercept: {stats_rec['intercept']:.6f}")
print("Coefficients:")
for i, v in enumerate(rec_vars):
    print(f"  {v}: {stats_rec['coefficients'][i]:10.6f}")

print(f"Adjusted R²: {stats_rec['adj_r_squared']:.6f}")
print(f"AIC: {stats_rec['aic']:.4f}")
print(f"BIC: {stats_rec['bic']:.4f}")

# ------------------------------------------------
# Full model output
```

```
# ------------------------------------------------
print("\nFULL MODEL: Y ~ X1 + X2 + X3 + X4 + X5")
print("-" * 80)
print(f"Intercept: {stats_full['intercept']:.6f}")
print("Coefficients:")
for i, v in enumerate(full_vars):
    print(f"  {v}: {stats_full['coefficients'][i]:10.6f}")

print(f"Adjusted R²: {stats_full['adj_r_squared']:.6f}")
print(f"AIC: {stats_full['aic']:.4f}")
print(f"BIC: {stats_full['bic']:.4f}")

# ------------------------------------------------
# F-test: does X1 actually help?
# ------------------------------------------------
ssr_reduced = stats_rec["ss_residual"]
ssr_full = stats_full["ss_residual"]
k_full = stats_full["k"]

f_stat = ((ssr_reduced - ssr_full) / 1) / (ssr_full / (n - k_full - 1))
p_val = 1 - scipy_stats.f.cdf(f_stat, 1, n - k_full - 1)

print("\n" + "=" * 80)
print("F-TEST FOR SIGNIFICANCE OF X1")
print("=" * 80)
print(f"F-statistic: {f_stat:.6f}")
print(f"p-value: {p_val:.6f}")
print(
    "Conclusion:",
    "X1 is NOT statistically significant at α = 0.05"
    if p_val > 0.05
    else "X1 is statistically significant at α = 0.05"
)

# ================================================
# Diagnostic plots
# ================================================
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

# ---- Residuals vs fitted (recommended) ----
ax = axes[0, 0]
ax.scatter(y_hat_rec, resid_rec, alpha=0.6, edgecolors="k", linewidth=0.5)
ax.axhline(0, linestyle="--", linewidth=1.5)
ax.set_title("Residuals vs Fitted (Recommended)")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.grid(True, alpha=0.3)

# ---- Q-Q plot (recommended) ----
ax = axes[0, 1]
scipy_stats.probplot(resid_rec, dist="norm", plot=ax)
ax.set_title("Normal Q-Q Plot (Recommended)")
ax.grid(True, alpha=0.3)

# ---- Scale-location plot (recommended) ----
ax = axes[0, 2]
std_resid = resid_rec / np.std(resid_rec)
ax.scatter(y_hat_rec, np.sqrt(np.abs(std_resid)), alpha=0.6, edgecolors="k", lin
ax.set_title("Scale-Location (Recommended)")
ax.set_xlabel("Fitted values")
```

```
ax.set_ylabel("√|Standardized residuals|")
ax.grid(True, alpha=0.3)

# ---- Residuals vs fitted (full) ----
ax = axes[1, 0]
ax.scatter(y_hat_full, resid_full, alpha=0.6, edgecolors="k", linewidth=0.5)
ax.axhline(0, linestyle="--", linewidth=1.5)
ax.set_title("Residuals vs Fitted (Full)")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.grid(True, alpha=0.3)

# ---- Q-Q plot (full) ----
ax = axes[1, 1]
scipy_stats.probplot(resid_full, dist="norm", plot=ax)
ax.set_title("Normal Q-Q Plot (Full)")
ax.grid(True, alpha=0.3)

# ---- Coefficient comparison ----
ax = axes[1, 2]
coef_df = pd.DataFrame({
    "Predictor": full_vars,
    "Full Model": stats_full["coefficients"],
    "Recommended Model": [0] + list(stats_rec["coefficients"])
})

x = np.arange(len(coef_df))
w = 0.35

ax.bar(x - w / 2, coef_df["Full Model"], w, label="Full")
ax.bar(x + w / 2, coef_df["Recommended Model"], w, label="Recommended")

ax.axhline(0, linewidth=0.8)
ax.set_title("Coefficient Comparison")
ax.set_xlabel("Predictor")
ax.set_ylabel("Coefficient value")
ax.set_xticks(x)
ax.set_xticklabels(coef_df["Predictor"])
ax.legend()
ax.grid(True, axis="y", alpha=0.3)

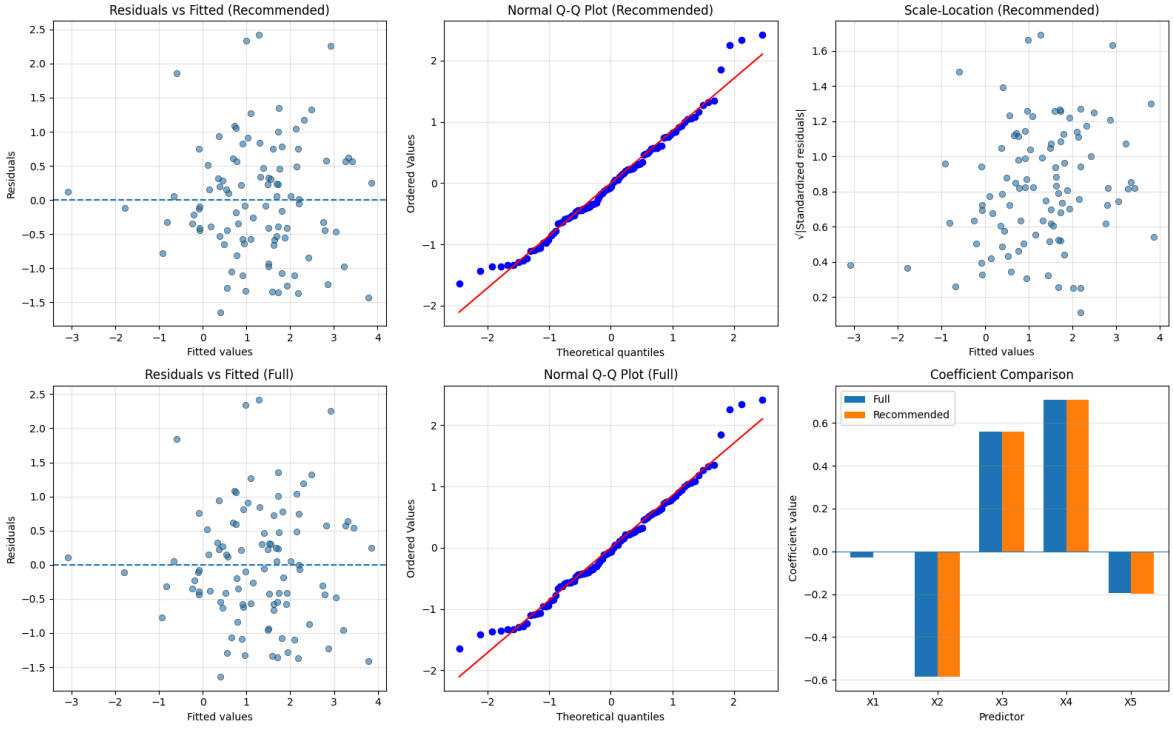
plt.tight_layout()
plt.show()
```

```
================================================================================
FINAL MODEL DIAGNOSTICS AND COMPARISON
================================================================================

RECOMMENDED MODEL: Y ~ X2 + X3 + X4 + X5
--------------------------------------------------------------------------------
Intercept: 1.189339
Coefficients:
  X2:  -0.586083
  X3:   0.559157
  X4:   0.710541
  X5:  -0.196639
Adjusted R²: 0.633974
AIC: -23.1710
BIC: -10.1452

FULL MODEL: Y ~ X1 + X2 + X3 + X4 + X5
--------------------------------------------------------------------------------
Intercept: 1.190212
Coefficients:
  X1:  -0.027785
  X2:  -0.584573
  X3:   0.560272
  X4:   0.710075
  X5:  -0.195747
Adjusted R²: 0.630170
AIC: -21.1952
BIC: -5.5642

================================================================================
F-TEST FOR SIGNIFICANCE OF X1
================================================================================
F-statistic: 0.022709
p-value: 0.880538
Conclusion: X1 is NOT statistically significant at α = 0.05
```



In [ ]:

Lecture 9: Volatility Modeling

MIT 18.S096

Dr. Kempthorne

Fall 2013

Outline

1. Volatility Modeling
   - Defining Volatility
   - Historical Volatility: Measurement and Prediction
   - Geometric Brownian Motion
   - Poisson Jump Diffusions
   - ARCH Models
   - GARCH Models

Defining Volatility

Basic Definition

- Annualized standard deviation of the change in price or value of a financial security.

Estimation/Prediction Approaches

- Historical/sample volatility measures.
- Geometric Brownian Motion Model
- Poisson Jump Diffusion Model
- ARCH/GARCH Models
- Stochastic Volatility (SV) Models
- Implied volatility from options/derivatives

Outline

1. Volatility Modeling
   - Defining Volatility
   - Historical Volatility: Measurement and Prediction
   - Geometric Brownian Motion
   - Poisson Jump Diffusions
   - ARCH Models
   - GARCH Models

Historical Volatility

Computing volatility from historical series of actual prices

- Prices of an asset at $(T+1)$ time points
$$\{P_t, t = 0, 1, 2, \ldots, T\}$$
- Returns of the asset for T time periods
$$R_t = log(P_t/P_{t-1}), t = 1, 2, \ldots, T$$
- $\{R_t\}$ assumed covariance stationary with
$$\sigma = \sqrt{var(R_t)} = \sqrt{E[(R_t - E[R_t])^2]}$$
with sample estimate:
$$\hat{\sigma} = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(R_t - \bar{R})^2}, \text{ with } \bar{R} = \frac{1}{T}\sum_{1}^{T} R_t.$$
- Annualized values
$$\widehat{vol} = \begin{cases} \sqrt{252}\hat{\sigma} & \text{(daily prices for 252 business days/year)} \\ \sqrt{52}\hat{\sigma} & \text{(weekly prices)} \\ \sqrt{12}\hat{\sigma} & \text{(monthly prices)} \end{cases}$$

Prediction Methods Based on Historical Volatility

Definition For time period t, define the **sample volatility**

$$\hat{\sigma}_t = \text{sample standard deviation of period } t \text{ returns}$$

- If t indexes months with daily data, then $\hat{\sigma}_t$ is the sample standard deviation of daily returns in month t.
- If t indexes days with daily data, then $\hat{\sigma}_t^2 = R_t^2$.
- With high-frequency data, daily σ_t is derived from cumulating squared intra-day returns.

Historical Average: $\tilde{\sigma}_{t+1}^2 = \frac{1}{t}\sum_1^t \hat{\sigma}_j^2$

(uses all available data)

Simple Moving Average: $\tilde{\sigma}_{t+1}^2 = \frac{1}{m}\sum_0^{m-1} \hat{\sigma}_{t-j}^2$

(uses last m single-period sample estimates)

Exponential Moving Average: $\tilde{\sigma}_{t+1}^2 = (1-\beta)\hat{\sigma}_t^2 + \beta\tilde{\sigma}_t^2 \quad 0 \le \beta \le 1$

(uses all available data)

Exponential Weighted Moving Average:

$\tilde{\sigma}_{t+1}^2 = \sum_{j=0}^{m-1}(\beta^j \hat{\sigma}_{t-j}^2)/[\sum_{j=0}^{m-1} \beta^j]$ (uses last m)

Predictions Based on Historical Volatility

Simple Regression:

$$\tilde{\sigma}^2_{t+1} = \gamma_{1,t}\hat{\sigma}^2_t + \gamma_{1,t}\hat{\sigma}^2_{t-1} + \cdots + \gamma_{p,t}\hat{\sigma}^2_{t-p+1} + u_t$$

Regression can be fit using all data or last m (rolling-windows).

Note: similar but different from auto-regression model of $\hat{\sigma}^2_t$

Trade-Offs

- Use more data to increase precision of estimators
- Use data closer to time t for estimation of σ_t.

Evaluate out-of-sample performance

- Distinguish assets and asset-classes
- Consider different sampling frequencies and different forecast horizons
- Apply performance measures (MSE, MAE, MAPE, etc.)

Benchmark Methodology: RiskMetrics

Outline

1. Volatility Modeling
   - Defining Volatility
   - Historical Volatility: Measurement and Prediction
   - Geometric Brownian Motion
   - Poisson Jump Diffusions
   - ARCH Models
   - GARCH Models

Geometric Brownian Motion (GBM)

For $\{S(t)\}$ the price of a security/portfolio at time t:

$$dS(t) = \mu S(t)dt + \sigma S(t)dW(t),$$

where

- σ is the volatility of the security's price
- μ is mean return (per unit time).
- $dS(t)$ infinitesimal increment in price
- $dW(t)$infinitesimal increment of a standard Brownian Motion/Wiener Process
  - Increments $[W(t') - W(t)]$ are Gaussian with mean zero and variance $(t' - t)$.
  - Increments on disjoint time intervals are independent.
    For $t_1 < t_2 < t_3 < t_4$,
    $[W(t_2) - W(t_1)]$ and $[W(t_4) - W(t_3)]$ are independent

Geometric Brownian Motion (GBM)

Sample Data from Process:

- Prices: $\{S(t), t = t_0, t_1, \ldots, t_n\}$
- Returns: $\{R_j = log[S(t_j)/S(t_{j-1})], j = 1, 2, \ldots, n\}$
  indep. r.v.'s: $R_j \sim N(\mu_* \Delta_j, \sigma^2 \Delta_j)$, where
  $\Delta_j = (t_j - t_{j-1})$ and $\mu_* = [\mu - \sigma^2/2]$

($\{log[S(t)]\}$ is Brownian Motion with drift μ and volatility σ^2.)

Maximum-Likelihood Parameter Estimation

- If $\Delta_j \equiv 1$, then
$$\begin{aligned} \hat{\mu}_* &= \bar{R} = \frac{1}{n}\sum_1^n R_t \\ \hat{\sigma}^2 &= \frac{1}{n}\sum_1^n (R_t - \bar{R})^2 \end{aligned}$$
- If Δ_j varies ... Exercise.

Geometric Brownian Motion

Garman-Klass Estimator:

- Sample information more than period-close prices, also have period-high, period-low, and period-open prices.
- Assume $\mu = 0$, $\Delta_j \equiv 1$ (e.g., daily) and let $f \in (0, 1)$ denote the fraction of the day prior to the market open.

$$
\begin{aligned}
C_j &= log[S(t_j)] \\
O_j &= log[S(t_{j-1} + f)]
\end{aligned}
$$

$$
H_j = \max_{t_{j-1}+f \leq t \leq t_j} log[S(t)]
$$

$$
L_j = \min_{t_{j-1}+f \leq t \leq t_j} log[S(t)]
$$

Garman-Klass Estimator

Using data from the first period:

- $\hat{\sigma}_0^2 = (C_1 - C_0)^2$: Close-to-Close squared return
  $E[\hat{\sigma}_0^2] = \sigma^2$, and $var[\hat{\sigma}_0^2] = 2(\sigma^2)^2 = 2\sigma^4$.
- $\hat{\sigma}_1^2 = \frac{(O_1 - C_0)^2}{f}$: Close-to-Open squared return
  $E[\hat{\sigma}_1^2] = \sigma^2$, and $var[\hat{\sigma}_1^2] = 2(\sigma^2)^2 = 2\sigma^4$.
- $\hat{\sigma}_2^2 = \frac{(C_1 - O_1)^2}{1-f}$: Open-to-Close squared return
  $E[\hat{\sigma}_2^2] = \sigma^2$, and $var[\hat{\sigma}_2^2] = 2(\sigma^2)^2 = 2\sigma^4$.
  Note: $\hat{\sigma}_1^2$ and $\hat{\sigma}_2^2$ are independent!

- $\hat{\sigma}_*^2 = \frac{1}{2}\hat{\sigma}_1^2 + \frac{1}{2}\hat{\sigma}_2^2$
  $E[\hat{\sigma}_*^2] = \sigma^2$, and $var[\hat{\sigma}_*^2] = \sigma^4$.

$\Longrightarrow eff(\hat{\sigma}_*^2) = \frac{var(\hat{\sigma}_0^2)}{var(\hat{\sigma}_*^2)} = 2.$

Parkinson (1976): With $f = 0$, defines
$\hat{\sigma}_3^2 = \frac{(H_1 - L_1)^2}{4(log2)}$ and shows $eff(\hat{\sigma}_3^2) \approx 5.2$.

Garman and Klass (1980) show that for any $0 < f < 1$:

- $\hat{\sigma}_4^2 = a \times \hat{\sigma}_1^2 + (1 - a)\hat{\sigma}_3^2$
  has minimum variance when $a \approx 0.17$, independent of f and $Eff(\hat{\sigma}_4^2) \approx 6.2$.
- "Best Analytic Scale-Invariant Estimator"'
  $\hat{\sigma}_{**}^2 = 0.511(u_1 - d_1)^2 - 0.019\{c_1(u_1 + d_1) - 2u_1 d_1\} - 0.383c_1^2$,
  where the normalized high/low/close are:
  $$\begin{aligned} u_j &= H_j - O_j \\ d_j &= L_j - O_j \\ c_j &= C_j - O_j \end{aligned}$$
  and $Eff(\hat{\sigma}_{**}^2) \approx 7.4$

- If $0 < f < 1$ then the opening price O_1 may differ from C_0 and the composite estimator is
$$\hat{\sigma}^2_{GK} = a\frac{(O_1 - C_0)^2}{f} + (1-a)\frac{\sigma^2_{**}}{(1-f)}$$
which has minimum variance when $a = 0.12$ and
$$Eff(\hat{\sigma}^2_{GK}) \approx 8.4.$$

Outline

1. Volatility Modeling
   - Defining Volatility
   - Historical Volatility: Measurement and Prediction
   - Geometric Brownian Motion
   - Poisson Jump Diffusions
   - ARCH Models
   - GARCH Models

Poisson Jump Diffusions

For $\{S(t)\}$ the stochastic proces for the price of the security/portfolio at time t,

$$\frac{dS(t)}{S(t)} = \mu dt + \sigma dW(t) + \gamma\sigma Z(t) d\Pi(t),$$

where

- $dS(t) =$ infinitesimal increment in price.
- $\mu =$ mean return (per unit time)
- $\sigma =$ diffusion volatility of the security's price process
- $dW(t)=$ increment of standard Wiener Process
- $d\Pi(t) =$ increment of a Poisson Process with rate λ, modeling the jump process.
- $(\gamma\sigma) \times Z(t)$, the magnitude a return jump/shock
  $Z(t)$ i.i.d $N(0,1)$ r.v.'s and
  $\gamma=$ scale(σ units) of jump magnitudes.

Poisson Jump Diffusions

Maximum-Likelihood Estimation of the PJD Model

- Model is a Poisson mixture of Gaussian Distributions.
- Moment-generating function derived as that of random sum of independent random variables.
- Likelihood function product of infinite sums
- EM Algorithm* expressible in closed form
  - Jumps treated as latent variables which simplify computations
  - Algorithm provides a posteriori estimates of number of jumps per time period.

* See Pickard, Kempthorne, Zakaria (1987).

Outline

1. Volatility Modeling
   - Defining Volatility
   - Historical Volatility: Measurement and Prediction
   - Geometric Brownian Motion
   - Poisson Jump Diffusions
   - ARCH Models
   - GARCH Models

ARCH Models

ARCH models are specified relative to the discrete-time process for the price of the security/portfolio: $\{S_t, t = 1, 2, \ldots\}$

Engle (1982) models the discrete returns of the process $y_t = log(S_t/S_{t-1})$ as

$$y_t = \mu_t + \epsilon_t,$$

where μ_t is the mean return, conditional on $\mathcal{F}_{t-1}$, the information available through time $(t-1)$, and

$$\epsilon_t = Z_t \times \sigma_t,$$

where Z_t i.i.d. with $E[Z_t] = 0$, and $var[Z_t] = 1$,

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \alpha_2 \epsilon_{t-2}^2 + \cdots + \alpha_p \epsilon_{t-p}^2$$

Parameter Constraints: $\alpha_j \geq 0, j = 0, 1, \ldots, p$

$\sigma_t^2 = var(R_t \mid \mathcal{F}_{t-1})$, "Conditional Heteroscedasticity" of returns .

ARCH Models

The ARCH model:

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \alpha_2 \epsilon_{t-2}^2 + \cdots + \alpha_p \epsilon_{t-p}^2$$

implies an AR model in ϵ_t^2. Add $(\epsilon_t^2 - \sigma_t^2) = u_t$ to both sides:

$$\epsilon_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \alpha_2 \epsilon_{t-2}^2 + \cdots + \alpha_p \epsilon_{t-p}^2 + u_t$$

where $u_t : E[u_t \mid \mathcal{F}_t] = 0$, and $var[u_t \mid \mathcal{F}_t] = var(\epsilon_t^2) = 2\sigma_t^4$.

Lagrange Multiplier Test

$$H_0 : \alpha_1 = \alpha_2 = \cdots = \alpha_p = 0$$

- Fit linear regression on squared residuals $\hat{\epsilon}_t = y_t - \hat{\mu}_t$. (i.e., Fit an AR(p) model to $[\hat{\epsilon}_t^2]$, $t = 1, 2, \ldots, n$)
- LM test statistic $= nR^2$, where R^2 is the R-squared of the fitted AR(p) model.

  Under H_0 the r.v. nR^2 is approx. χ^2 $(df = p)$
- Note: the linear regression estimates of parameters are not MLEs under Gaussian assumptions; they correspond to quasi-maximum likelihood estimates (QMLE).

Maximum Likelihood Estimation

ARCH Model:

$$\begin{aligned} y_t &= c + \epsilon_t \\ \epsilon_t &= z_t \sigma_t \\ \sigma_t^2 &= \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \cdots + \alpha_p \epsilon_{t-p}^2 \end{aligned}$$

$t = 0, 1, \ldots, T$

Likelihood:

$$\begin{aligned} L(c, \boldsymbol{\alpha}) &= p(y_1, \ldots, y_n \mid c, \alpha_0, \alpha_1, \ldots, \alpha_p) \\ &= \prod_{t=1}^{n} p(y_t \mid \mathcal{F}_{t-1}, c, \boldsymbol{\alpha}) \\ &= \prod_{t=1}^{n} \left[\frac{1}{\sqrt{2\pi\sigma_t^2}} exp\left(\frac{-1}{2} \frac{\epsilon_t^2}{\sigma_t^2} \right) \right] \end{aligned}$$

where $\epsilon_t = y_t - c$ and

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \cdots \alpha_p \epsilon_{t-p}^2.$$

Constraints:

- $\alpha_i \geq 0,\ i = 1, 2, \ldots, p$
- $(\alpha_1 + \cdots + \alpha_p) < 1.$

Outline

1. Volatility Modeling
   - Defining Volatility
   - Historical Volatility: Measurement and Prediction
   - Geometric Brownian Motion
   - Poisson Jump Diffusions
   - ARCH Models
   - GARCH Models

GARCH Models

Bollerslev (1986) extended ARCH models to:

GARCH(p,q) Model

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{p} \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^{q} \beta_j \sigma_{t-j}^2$$

Constraints: $\alpha_i \geq 0, \forall i$, and $\beta_j \geq 0, \forall j$

GARCH(1,1) Model

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

- Parsimonious
- Fits many financial time seriies

GARCH Models

The GARCH(1,1) model:

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

implies an ARMA model in ϵ_t^2. Eliminate $\sigma_{t'}^2$ using $(\epsilon_{t'}^2 - \sigma_{t'}^2) = u_{t'}$

$$\begin{aligned} \epsilon_t^2 - u_t &= \alpha_0 + \alpha_1 + \epsilon_{t-1}^2 + \beta_1(\epsilon_{t-1}^2 - u_{t-1}) \\ \epsilon_t^2 &= \alpha_0 + (\alpha_1 + \beta_1)\epsilon_{t-1}^2 + u_t - \beta_1 u_{t-1} \end{aligned}$$

where $u_t : E[u_t \mid \mathcal{F}_t] = 0$, and $var[u_t \mid \mathcal{F}_t] = var(\epsilon_t^2) = 2\sigma_t^4$.

$\Longrightarrow$ $GARCH(1,1)$ implies an $ARMA(1,1)$ with

$$u_t = (\epsilon_t^2 - \sigma_t^2) \sim WN(0, 2\sigma^4)$$

Stationarity of GARCH model deduced from ARMA model

$$\begin{aligned} A(L)\epsilon_t^2 &= B(L)u_t \\ \epsilon_t^2 &= [A(L)]^{-1}B(L)u_t. \end{aligned}$$

Covariance stationary:roots of $A(z)$ outside $\{|z| \leq 1\}$,i.e.,

$$|\alpha_1 + \beta_1| < 1$$

GARCH Models

Unconditional Volatility / Long-Run Variance

GARCH(1,1): Assuming stationarity, $0 < (\alpha_1 + \beta_1) < 1$

$$\sigma_*^2 = \alpha_0 + (\alpha_1 + \beta_1)\sigma_*^2$$

$$\Longrightarrow \quad \sigma_*^2 = \frac{\alpha_0}{(1-\alpha_1-\beta_1)}$$

GARCH(p,q) implies ARMA(max(p,q), q) model

- Stationary if $0 < (\sum_1^p \alpha_i + \sum_1^q \beta_j) < 1$
- Long-Run Variance:

$$\sigma_*^2 = \alpha_0 + (\sum_1^p \alpha_j + \sum_1^q \beta_1)\sigma_*^2$$

$$\Longrightarrow \quad \sigma_*^2 = \frac{\alpha_0}{[1-\sum_1^{max(p,q)}(\alpha_i+\beta_j)]}$$

GARCH Model Estimation

Maximum Likelihood Estimation

GARCH Model:

$$\begin{aligned} y_t &= c + \epsilon_t \\ \epsilon_t &= z_t \sigma_t \\ \sigma_t^2 &= \alpha_0 + \sum_1^p \alpha_i \epsilon_{t-i}^2 + \sum_1^q \beta_j \sigma_{t-j}^2 \end{aligned}$$

$t = 0, 1, \ldots, T$

Likelihood:

$$\begin{aligned} L(c, \boldsymbol{\alpha}, \boldsymbol{\beta}) &= p(y_1, \ldots, y_T \mid c, \alpha_0, \alpha_1, \ldots, \alpha_p, \beta_1, \ldots, \beta_q) \\ &= \prod_{t=1}^T p(y_t \mid \mathcal{F}_{t-1}, c, \boldsymbol{\alpha}, \boldsymbol{\beta}) \\ &= \prod_{t=1}^T [\frac{1}{\sqrt{2\pi\sigma_t^2}} exp(\frac{-1}{2} \frac{\epsilon_t^2}{\sigma_t^2})] \end{aligned}$$

where $\epsilon_t = y_t - c$ and

$$\sigma_t^2 = \alpha_0 + \sum_1^p \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^q \beta_j \sigma_{t-j}^2.$$

Constraints: $\alpha_i \geq 0, \forall i$, $\beta_j \geq 0, \forall j$, and $0 < (\sum_1^p \alpha_u + \sum_1^1 \beta_j) < 1$.

GARCH Model Estimation/Evaluation/Model-Selection

- Maximum-Likelihood Estimates: $\hat{c}, \hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\beta}}$
  $\Longrightarrow \hat{\epsilon}_t$ and $\hat{\sigma}_t^2$ $(t = T, T-1, \ldots)$
- Standardized Residuals
  $\hat{\epsilon}_t/\hat{\sigma}_t$: should be uncorrelated
- Squared Standardized Residuals
  $(\hat{\epsilon}_t/\hat{\sigma}_t)^2$: should be uncorrelated

Testing Normality of Residuals

- Normal QQ Plots
- Jaque-Bera test
- Shapiro-Wilk test
- MLE Percentiles Goodness-of-Fit Test
- Kolmogorov-Smirnov Goodness-of-Fit Test

Model Selection: Apply model-selection critera

- Akaike Information Criterion (AIC)
- Bayes Information Criterion (BIC)

Stylized Features of Returns/Volatility

Volatility Clustering

- Large ϵ_t^2 follow large ϵ_{t-1}^2
- Small ϵ_t^2 follow small ϵ_{t-1}^2

GARCH models can prescribe

- Large σ_t^2 follow large σ_{t-1}^2
- Small σ_t^2 follow small σ_{t-1}^2

Heavy Tails / Fat Tails

- Returns distribution has heavier tails (higher Kurtosis) than Gaussian
- GARCH(p,q) models are stochastic mixture of Gaussian distributions with higher kurtosis.

Engle, Bollerslev, and Nelson (1994)

Stylized Features of Returns/Volatility

Volatility Mean Reversion

GARCH(1,1) Model

- Long-run average volatility: $\sigma_*^2 = \frac{\alpha_0}{1-\alpha_1-\beta_1}$
- Mean-Reversion to Long-Run Average

$$\epsilon_t^2 = \alpha_0 + (\alpha_1 + \beta_1)\epsilon_{t-1}^2 + u_t - \beta_1 u_{t-1}$$

Substituting: $\alpha_0 = (1 - \alpha_1 - \beta_1)\sigma_*^2$

$$(\epsilon_t^2 - \sigma_*^2) = (\alpha_1 + \beta_1)(\epsilon_{t-1}^2 - \sigma_*^2) + u_t - \beta_1 u_{t-1}$$

$0 < (\alpha_1 + \beta_1) < 1 \Longrightarrow$ Mean Reversion!

Extended Ornstein-Uhlenbeck(OU) Process for ϵ_t^2 with MA(1) errors.

Extensions of GARCH Models

EGARCH Nelson (1992)

TGARCH Glosten, Jagannathan, Runkler (1993)

PGARCH Ding, Engle, Granger

GARCH-In-Mean

Non-Gaussian Distributions

- $t-$Distributions
- Generalized Error Distributions (GED)

MIT OpenCourseWare
http://ocw.mit.edu

18.S096 Topics in Mathematics with Applications in Finance

Fall 2013

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

FINANCIAL ECONOMETRICS

MODULE 5 | LESSON 1

ARCH MODEL

Reading Time	60 minutes
Prior Knowledge	Basics of time series, ARIMA
Keywords	Conditional Means, Conditional Variance, Volatility Clustering, ARCH(1) Model

In the last module, we used the ARIMA model to analyze Google stock price. One key issue we observed was that Google stock price does not exhibit constant variance throughout our modeling time period. This issue does not just happen to Google's stock price. It is very common among many financial asset returns. In this module, we are going to address this phenomenon and find a model to analyze it. We will introduce the ARCH model and GARCH model to handle this issue. They are popular methods in finance to model volatility for many asset returns. In this lesson, we will start with learning the ARCH model and its properties. We will simulate the ARCH process to give a visual example and learn how to use ACF and PACF plots to identify the ARCH process.

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from arch.univariate import ARCH, ConstantMean, Normal
from scipy import stats

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
# note that !pip install arch was previously run in this notebook to make the fo
```

In [2]:
```
# Download the dataset
m5_data = pd.read_csv("M5. goog_eur_10.csv")

# Convert date variable to date format and set index
m5_data["Date2"] = pd.to_datetime(m5_data["Date"], format="%m/%d/%Y")
goog = m5_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
```

1. Common Features of Financial Asset Returns

There are several special features from financial asset returns. The existence of these features motivates us to use ARCH and GARCH models to analyze financial asset returns. In this section, we are going to go through these features first so that you will have a

better understanding of why we need to learn the ARCH and GARCH models. The first feature we'll talk about is the reason to use asset returns to conduct a research project instead of asset prices.

1.1 Asset Return Time Series Data is More Stable Than Asset Price Time Series Data

In a lot of financial research, we usually use asset returns instead of asset prices to conduct analysis for a financial asset. Here are some reasons to use asset returns:

> **a.** An asset return provides same-scale time series data for all financial assets. It is easier for comparison among several financial assets with different price levels.

> **b.** Asset return time series is usually more stable than asset price time series. Let's use Google's stock price and return as an example.

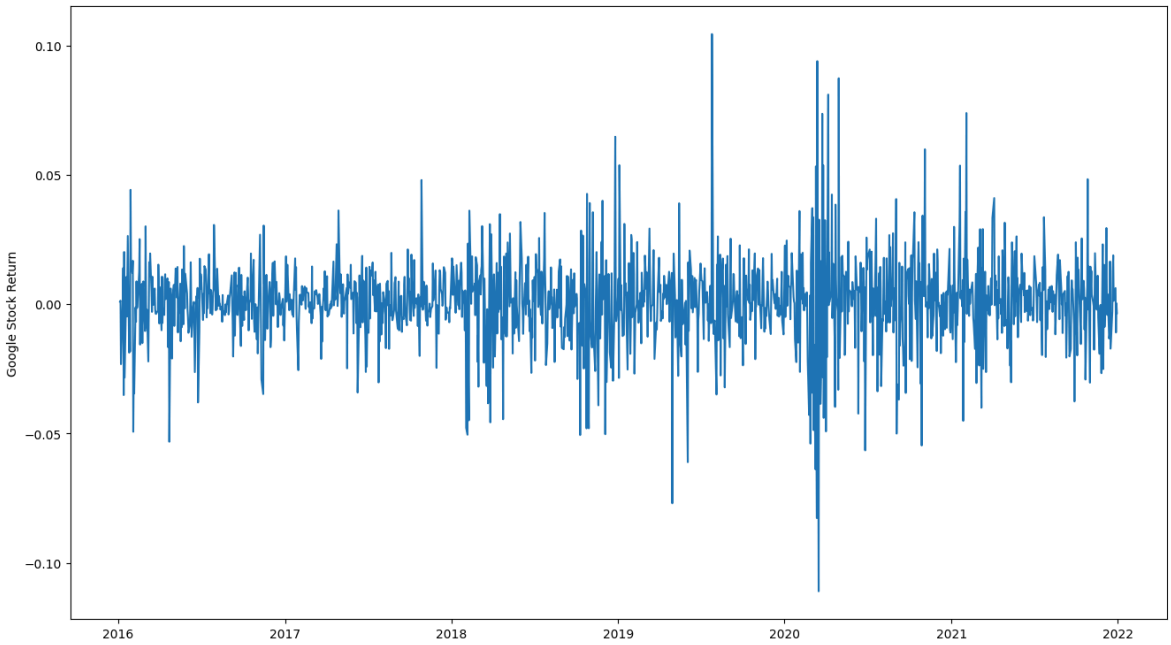
Figure 1: Google Daily Stock Price and Return Historical Chart 2016 – 2021

In [3]:
```
# Plot Google Daily stock price and return

# Google Daily stock price
plt.plot(goog.GOOGLE)
plt.ylabel("Google Stock Price")
plt.show()

# Google Daily stock price return
goog["GOOGLE_R"] = goog.GOOGLE.pct_change()
plt.plot(goog.GOOGLE_R)
plt.ylabel("Google Stock Return")
plt.show()
```





In figure 1, the upper chart is Google's daily stock price chart. The bottom chart is Google's stock daily return chart. We can see that Google's price plot has an upward trend whereas Google's return plot oscillates around 0 and is stable. We can say the return on average is 0.

> **c.** If the asset return is small, we can approximate the return by using the difference of the natural log of price. Here is the formula:

$$r_t = \frac{p_t - p_{t-1}}{p_{t-1}} \approx log(p_t) - log(p_{t-1})$$

1.2 The Volatility of Asset Returns Can Vary During Different Time Periods

In portfolio management and risk management, managers control risk by monitoring the volatility of the underlying assets. Variance and standard deviations are common metrics to measure volatility. We can see from figure 1 that the local variance of Google stock returns changes substantially across the time period. This time series data exhibits heteroskedasticity. In the return plot, we can see there are pockets of time in the whole period where the movement of returns are more volatile than others. These high volatility pockets do not spread randomly during the whole time period. We call these **volatility clustering**. We notice that there is autocorrelation of asset returns. A highly volatile return one day would be followed by another highly volatile return the next day. It means that the past volatilities will still have an impact on today's volatility. The highly volatile periods tend to group together. This is a key property that later we would like to model.

1.3 Asset Return Distribution Has Heavier Tails Than Normal Distribution

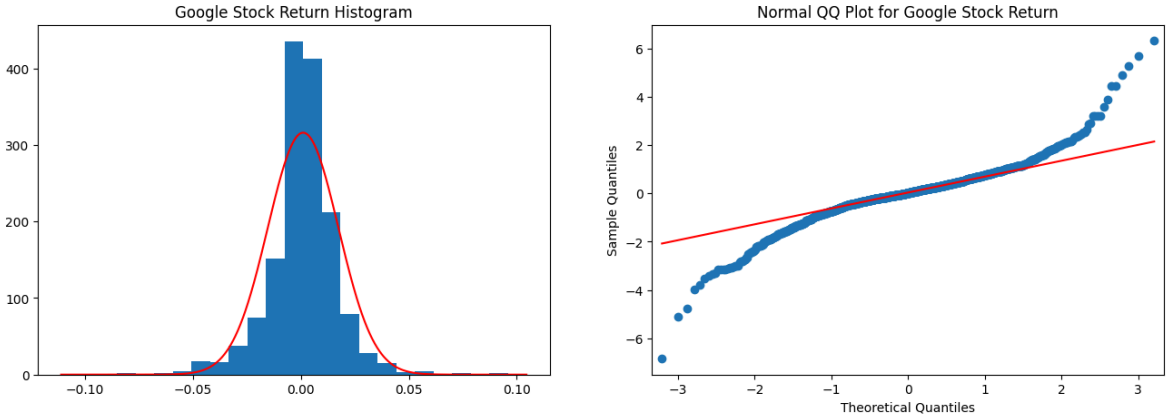
We have discussed many times in the previous lessons that financial asset returns often show a distribution with heavier tails than normal distribution. Let's take a look at Google's stock return distribution to see if the same feature shows up.

Figure 2: Histogram and Normal QQ Plot for Google Stock Returns

In [4]:
```
# Histogram and Normal QQ Plot for Google Stock Returns
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
goog_r = goog.GOOGLE_R.dropna()

# Histogram with density
x = np.linspace(min(goog_r), max(goog_r), len(goog_r))
(mu, sigma) = stats.norm.fit(goog_r)
values, bins, _ = ax1.hist(goog_r, bins=25)  # Histogram
ax1.plot(x, stats.norm.pdf(x, mu, sigma) * sum(values * np.diff(bins)), "r")  #
ax1.set(title="Google Stock Return Histogram")

# Normal QQ plot
sm.qqplot(goog_r, stats.norm, fit=True, line="q", ax=ax2)
ax2.set(title="Normal QQ Plot for Google Stock Return")
plt.show()
```



In figure 2, we show Google's stock return histogram and QQ plot. Both plots demonstrate that Google's stock return exhibits fat tail characteristic as well.

2. Conditional Means and Conditional Variances

We have talked about special features for financial asset returns time series. Before we dive into models to formally analyze those features, let's start by learning some concepts that will prepare us for explaining the models later.

We talked about conditional probability in the last module. Now let's learn how to obtain conditional means and conditional variances.

Assume X and Y are two discrete random variables.

The **conditional mean** of Y given X is as follows:

$$\mu_{Y|X} = \sum f(Y|X)Y = E(Y|X)$$

where $f(Y|X)$ is the conditional probability of Y given X.

The **conditional variance** of Y given X is as follows:

$$\sigma^2_{Y|X} = \sum Y^2 f(Y|X) - \mu^2_{Y|X} = E(Y^2|X) - \mu^2_{Y|X}$$

We can see that the calculation for conditional means and conditional variances are very similar to calculation for regular means and variances. The difference is we use conditional probability to calculate conditional means and conditional variances.

Why do we need to know conditional means and conditional variances in the time series context? The central idea is how we can incorporate the concept of using past value information to predict today's value in a model. We actually did that for the ARMA model in the last module. We used the concept of conditional mean to run an ARMA model. Let's use the following example to illustrate the idea.

Consider an AR(p) model with a constant conditional variance $Var(X_t|X_{t-1}, X_{t-2}, \cdots, X_{t-p}) = \sigma^2$. We can write the AR($p$) model as follows:

$$X_t = f(X_{t-1}, X_{t-2}, \cdots, X_{t-p}) + e_t$$

where white noise e_t has mean = 0 and constant conditional variance σ^2, $f()$ is the conditional mean for X_t given $X_{t-1}, X_{t-2}, \cdots, X_{t-p}$ and σ^2 is the constant conditional variance for X_t given $X_{t-1}, X_{t-2}, \cdots, X_{t-p}$.

Now if we want to introduce non-constant variance like we observed from the Google data in the last section to the above AR(p) model, we can rewrite the model as:

$$X_t = f(X_{t-1}, X_{t-2}, \cdots, X_{t-p}) + \sigma(X_{t-1}, X_{t-2}, \cdots, X_{t-p})_t \; e_t$$

Where white noise e_t has conditional mean = 0 and conditional variance = 1, $\sigma(X_{t-1}, X_{t-2}, \cdots, X_{t-p})_t$ is the standard deviation for X_t given $X_{t-1}, X_{t-2}, \cdots, X_{t-p}$. This standard deviation will change when $X_{t-1}, X_{t-2}, \cdots, X_{t-p}$ change. Since $\sigma(X_{t-1}, X_{t-2}, \cdots, X_{t-p})_t$ is standard deviation, it should be non-negative. Later, we will provide regularity conditions to make sure this function is non-negative.

What we just described is a general variance function model for conditional variance. ARCH and GARCH are all types of variance function models.

In the following sections, we are going to focus our analysis on asset returns. The common observation is that the conditional mean of asset return is 0, so we will drop the conditional mean part from the above model to focus on the conditional variance part in the following sections. We will start with a simple ARCH(1) model in this lesson. Then, we will introduce a general ARCH(m) model. In the next lesson, we will expand the ARCH model and introduce the GARCH model.

3. ARCH(1) Model

Now let's formally introduce our first conditional variance model, ARCH(1) model. ARCH stands for autoregressive conditional heteroskedasticity. If we have a stationary asset return time series r_t, we can define ARCH(1) model for r_t as follows:

$$r_t = \sigma_t e_t$$

Where $e_t \sim iidN(0,1)$ and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2$$

with $\alpha_0 > 0$ and $0 \leq \alpha_1 < 1$.

The design of the above variance equation makes sure the variance is non-negative. The requirement of $\alpha_1 < 1$ also ensures r_t is stationary with finite variance. Also in many cases, instead of assuming e_t to be standard normal distribution, we can assume e_t follows a Student's t-distribution. Lastly, the 1 in ARCH(1) refers to the lag 1 asset return in the variance equation.

Now let's review the properties of ARCH(1) model/process.

3.1 ARCH(1) is Strictly Stationary

From $r_t = \sigma_t e_t$ and $\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2$, we know

$$r_t^2 = \alpha_0 e_t^2 + \alpha_1 e_t^2 r_{t-1}^2$$

By iterating the above definition, we can show that

$$r_t = e_t \sqrt{\alpha_0 \left(1 + \sum_{j=1}^{\infty} \alpha_1^j e_{t-1}^2 \cdots e_{t-j}^2\right)}$$

We will leave the proof as one open-ended question.

3.2 r_t is Conditionally Normally Distributed

The conditional mean of r_t is given by:

$$E(r_t | r_s, s < t) = E(\sigma_t e_t | r_s, s < t) = \sigma_t E(e_t) = \sigma_t \cdot 0 = 0$$

Conditional variance of r_t is as follows:

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2$$

Hence $r_t | r_{t-1} \sim N(0, \alpha_0 + \alpha_1 r_{t-1}^2)$.

We can see from the above conditional variance equation, the current variance of r_t depends on the past value of r_{t-1}. It means when r_{t-1} has a large value at time $t-1$, the variance at time t will be large too. It then implies that r_t will have large volatility at time t. This model set up is able to capture the volatility clustering phenomenon we observed in the Google stock return data.

3.3 r_t is White Noise

The unconditional mean of r_t is as follows:

$$E[E(r_t|r_s, s < t)] = 0$$

The unconditional variance of r_t can be derived by taking the expectation of the equation from Section 3.1.

$$\begin{aligned} Var(r_t) = E(r_t^2) &= E(e_t^2(\alpha_0(1 + \sum_{j=1}^{\infty} \alpha_1^j e_{t-1}^2 \cdots e_{t-j}^2))) \\ &= \alpha_0(1 + \alpha_1^1 + \alpha_1^2 + \cdots + \alpha_1^\infty) \\ &= \frac{\alpha_0}{1 - \alpha_1}, \quad \text{where } 0 \leq \alpha_1 < 1 \end{aligned}$$

The autocovariance of r_t is:

$$\begin{aligned} cov(r_{t+h}, r_t) = E(r_{t+h} r_t) &= E[E(r_{t+h} r_t | r_s, s < t + h)] \\ &= E[r_t E(r_{t+h} | r_s, s < t + h)] \\ &= E[r_t \cdot 0] \\ &= 0, \quad \text{for all } h > 0 \end{aligned}$$

Hence, the unconditional distribution of r_t can be written as follows:

$$r_t \sim \text{white noise}\Big(0, \frac{\alpha_0}{1 - \alpha_1}\Big)$$

We can see from the above result, r_t is unconditionally homoscedastic when $0 \leq \alpha_1 < 1$. So when r_t is stationary ($0 \leq \alpha_1 < 1$), r_t is conditionally heteroskedastic but unconditionally homoscedastic.

Although we are using asset returns for our ARCH model example, the original ARCH model was used for modeling squared error terms from a time series model. That's why the ARCH model has this nice unconditional property as white noise.

3.4 r_t Is Not i.i.d.

Although r_t has 0 covariance to its past values, r_t^2 does depend on the squared past returns. It can be shown as follows:

$$\begin{aligned} E(r_t^2 | r_{t-1}) &= E(\alpha_0 e_t^2 + \alpha_1 e_t^2 r_{t-1}^2 | r_{t-1}) \\ &= \alpha_0 + \alpha_1 r_{t-1}^2 \end{aligned}$$

So we can see ARCH(1) is a case where the elements in the process have 0 covariance, but the elements are not independent. This conclusion also confirms our previous knowledge that two variables with 0 covariance do not mean they are independent.

3.5 r_t^2 IS A non-Gaussian AR(1) Process

From the ARCH(1) definition, we can take the square of r_t and rewrite the equations as follows:

$$r_t^2 = \sigma_t^2 e_t^2$$

$$\alpha_0 + \alpha_1 r_{t-1}^2 = \sigma_t^2$$

We then subtract the second equation from the first equation to get the following equation:

$$r_t^2 - (\alpha_0 + \alpha_1 r_{t-1}^2) = \sigma_t^2(e_t^2 - 1)$$

$$\Rightarrow r_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \nu_t$$

where $\nu_t = \sigma_t^2(e_t^2 - 1)$ is non-normal white noise. $(e_t^2 - 1)$ follows a shifted Chi-Square distribution since e_t^2 is the square of a N(0,1) random variable.

Also if $E(r_t^4) < \infty$, then r_t^2 has the same ACF as AR(1) process.

The ACF for r_t^2 is as follows:

$$\rho_{r^2}(h) = \alpha_1^{|h|}$$

for all h and $\alpha_1 < 1$.

With the knowledge that the squared asset return is also a stationary AR(1) process, we can use ACF and PACF plots of asset return and squared asset return to look for an ARCH(1) model. If the asset return time series plot exhibits a white noise pattern and the squared asset return PACF plot shows cut off pattern after lag 1, we can use an ARCH(1) model for modeling variance of the asset return. We will talk more about this in later sections.

3.6 r_t Has Heavier Tails Than Standard Normal Distribution If $\alpha_1 < \frac{1}{3}$

When $3\alpha_1 < 1$ and $E(r_t^4) < \infty$, we can show that kurtosis of r_t is greater than 3. It means that r_t has heavier tails than standard normal distribution. It is similar to our empirical observations that asset return usually has more outliers than a time series generated from normal distribution.

3.7 ARCH(m) Process

We have introduced ARCH(1) process in previous sections. We can extend the process to include return values from several pass time periods to form an ARCH(m) process. An ARCH(m) process is the process to predict current variance with squared asset return values going back to m time periods. Here is the formal definition for ARCH(m) process:

$$r_t = \sigma_t e_t$$

Where $e_t \sim iidN(0,1)$ and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \cdots + \alpha_m r_{t-m}^2$$

With some restrictions on ARCH(m) coefficients, the squared asset returns will be AR(m).

4. ARCH(1) Simulation

In this section, let's take a look at simulation data to get a visual understanding of an ARCH(1) process.

Figure 3: ARCH(1) Simulation n=1000, $\alpha_0 = 5$ and $\alpha_1 = 0.5$

In [5]:
```
# ARCH(1) Simulation n=1000, alpha_0=5 and alpha_1=0.5

# simulated process parameters
rs = np.random.RandomState([12345, 77777])
dist = Normal(seed=rs)
vol = ARCH(p=1)
repro_mod = ConstantMean(None, volatility=vol, distribution=dist)
params = pd.Series({"mu": 0.0, "omega": 5, "alpha[1]": 0.5, "beta[1]": 0.0})

# model simulation
arch1_sim = repro_mod.simulate(params, nobs=1000)
plt.plot(arch1_sim.data)
plt.show()
```

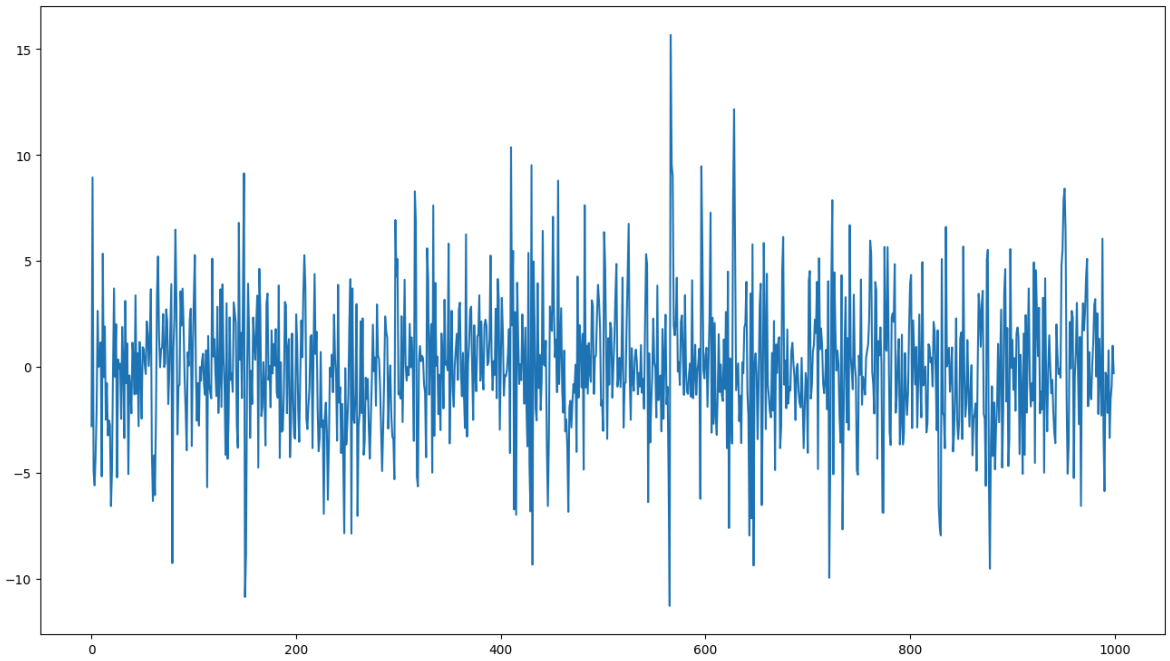


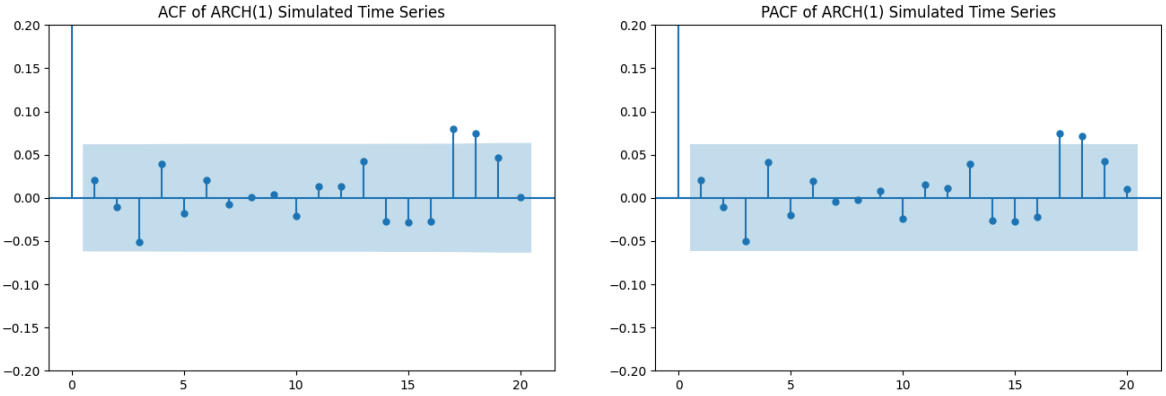
Figure 3 shows a simulated ARCH(1) process. From the graph, we can see the time series oscillates around 0 with occasional bursts of volatility. We can also see there are clusters of high volatility during the whole time period, like around $t = 600$. These features are all in agreement with the common features of financial asset returns we discussed earlier.

Figure 4: ACF and PACF of ARCH(1) Simulated Time Series

In [6]:
```
# ACF and PACF of ARCH(1) Simulated Time Series
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(arch1_sim.data.dropna(), lags=20, ax=ax1)
ax1.set(title="ACF of ARCH(1) Simulated Time Series")
ax1.set_ylim([-0.2, 0.2])

sm.graphics.tsa.plot_pacf(arch1_sim.data.dropna(), lags=20, ax=ax2)
```

```
ax2.set(title="PACF of ARCH(1) Simulated Time Series")
ax2.set_ylim([-0.2, 0.2])
plt.show()
```

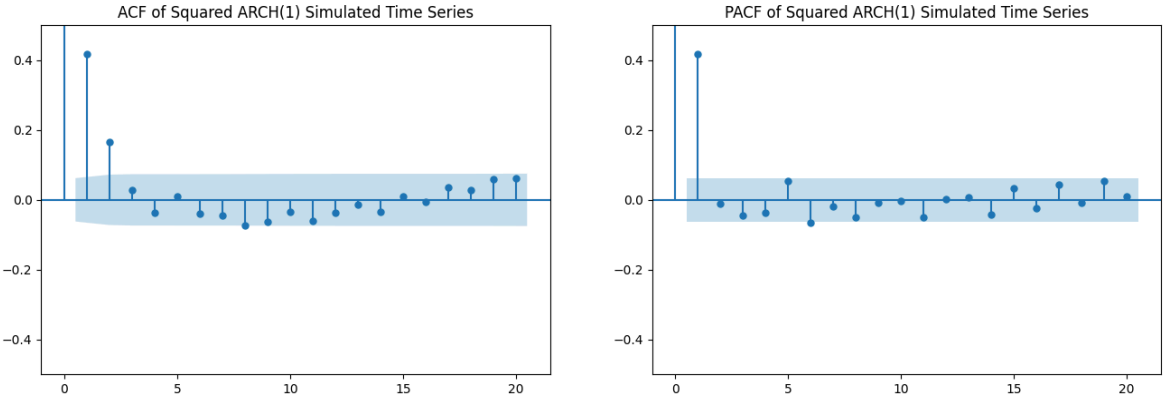


The ACF and PACF plots in figure 4 shows that the simulated time series do not exhibit any correlations and behave quite like white noise.

Figure 5: ACF and PACF of Squared ARCH(1) Simulated Time Series

In [7]:
```
# ACF and PACF of Squared ARCH(1) Simulated Time Series
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(arch1_sim.data.dropna() ** 2, lags=20, ax=ax1)
ax1.set(title="ACF of Squared ARCH(1) Simulated Time Series")
ax1.set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_pacf(arch1_sim.data.dropna() ** 2, lags=20, ax=ax2)
ax2.set(title="PACF of Squared ARCH(1) Simulated Time Series")
ax2.set_ylim([-0.5, 0.5])
plt.show()
```



However, in figure 5, the PACF plot of squared simulated time series shows a drop after lag 1. The ACF shows a gradual decline. The two graphs suggest the properties of the AR(1) process for the squared simulated time series.

5. Conclusion

In this lesson, we went through the definition and properties of an ARCH(1) model and introduced a more general ARCH(m) process too. In the next lesson, we will introduce another general form of the ARCH process—the GARCH process. The GARCH process is

widely used in finance research. We will go through the basics and demonstrate how to apply the GARCH model to a real finance problem.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Bayesian Updating with Discrete Priors
Class 11, 18.05
Jeremy Orloff and Jonathan Bloom

1 Learning Goals

1. Be able to apply Bayes' theorem to compute probabilities.
2. Be able to define and to identify the roles of prior probability, likelihood (Bayes term), posterior probability, data and hypothesis in the application of Bayes' Theorem.
3. Be able to use a Bayesian update table to compute posterior probabilities.

2 Review of Bayes' theorem

Recall that Bayes' theorem allows us to 'invert' conditional probabilities. If $\mathcal{H}$ and $\mathcal{D}$ are events, then:

$$P(\mathcal{H} \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid \mathcal{H})P(\mathcal{H})}{P(\mathcal{D})}$$

Our view is that Bayes' theorem forms the foundation for inferential statistics. We will begin to justify this view today.

2.1 The base rate fallacy

When we first learned Bayes' theorem we worked an example about screening tests showing that $P(\mathcal{D}|\mathcal{H})$ can be very different from $P(\mathcal{H}|\mathcal{D})$. In the appendix we work a similar example. If you are not comfortable with Bayes' theorem you should read the example in the appendix now.

3 Terminology and Bayes' theorem in tabular form

We now use a coin tossing problem to introduce terminology and a tabular format for Bayes' theorem. This will provide a simple, uncluttered example that shows our main points.

Example 1. There are three types of coins which have different probabilities of landing heads when tossed.

- Type A coins are fair, with probability 0.5 of heads
- Type B coins are bent and have probability 0.6 of heads
- Type C coins are bent and have probability 0.9 of heads

Suppose I have a drawer containing 5 coins: 2 of type A, 2 of type B, and 1 of type C. I reach into the drawer and pick a coin at random. Without showing you the coin I flip it once and get heads. What is the probability it is type A? Type B? Type C?

Solution: Let A, B, and C be the event that the chosen coin was type A, type B, and type C. Let $\mathcal{D}$ be the event that the toss is heads. The problem asks us to find

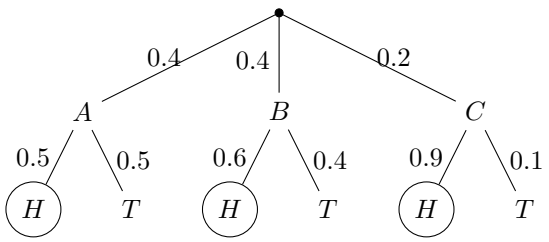
$$P(A|\mathcal{D}), \quad P(B|\mathcal{D}), \quad P(C|\mathcal{D}).$$

Before applying Bayes' theorem, let's introduce some terminology.

- Experiment: pick a coin from the drawer at random, flip it, and record the result.
- Data: the result of our experiment. In this case the event $\mathcal{D}$ = 'heads'. We think of $\mathcal{D}$ as data that provides evidence for or against each hypothesis.
- Hypotheses: we are testing three hypotheses: the coin is type A, B or C.
- Prior probability: the probability of each hypothesis prior to tossing the coin (collecting data). Since the drawer has 2 coins of type A, 2 of type B and 1 of type C we have
$$P(A) = 0.4, \qquad P(B) = 0.4, \qquad P(C) = 0.2.$$
- Likelihood: (This is the same likelihood we used for the MLE.) The likelihood function is $P(\mathcal{D}|\mathcal{H})$, i.e., the probability of the data assuming that the hypothesis is true. Most often we will consider the data as fixed and let the hypothesis vary. For example, $P(\mathcal{D}|A)$ = probability of heads if the coin is type A. In our case the likelihoods are
$$P(\mathcal{D}|A) = 0.5, \qquad P(\mathcal{D}|B) = 0.6, \qquad P(\mathcal{D}|C) = 0.9.$$
The name likelihood is so well established in the literature that we have to teach it to you. However in colloquial language likelihood and probability are synonyms. This leads to the likelihood function often being confused with the probability of a hypothesis. Because of this we'd prefer to use the name Bayes' term. However since we are stuck with 'likelihood' we will try to use it very carefully and in a way that minimizes any confusion.
- Posterior probability: the probability (posterior to) of each hypothesis given the data from tossing the coin.
$$P(A|\mathcal{D}), \qquad P(B|\mathcal{D}), \qquad P(C|\mathcal{D}).$$
These posterior probabilities are what the problem asks us to find.

We now use Bayes' theorem to compute each of the posterior probabilities. We are going to write this out in complete detail so we can pick out each of the parts (Remember that the data $\mathcal{D}$ is that the toss was heads.)

First we organize the probabilities into a tree:



Probability tree for choosing and tossing a coin.

Bayes' theorem says, e.g. $P(A|\mathcal{D}) = \dfrac{P(\mathcal{D}|A)P(A)}{P(\mathcal{D})}$. The denominator $P(\mathcal{D})$ is computed using the law of total probability:

$$P(\mathcal{D}) = P(\mathcal{D}|A)P(A) + P(\mathcal{D}|B)P(B) + P(\mathcal{D}|C)P(C) = 0.5 \cdot 0.4 + 0.6 \cdot 0.4 + 0.9 \cdot 0.2 = 0.62.$$

Now each of the three posterior probabilities can be computed:

$$P(A|\mathcal{D}) = \frac{P(\mathcal{D}|A)P(A)}{P(\mathcal{D})} = \frac{0.5 \cdot 0.4}{0.62} = \frac{0.2}{0.62}$$
$$P(B|\mathcal{D}) = \frac{P(\mathcal{D}|B)P(B)}{P(\mathcal{D})} = \frac{0.6 \cdot 0.4}{0.62} = \frac{0.24}{0.62}$$
$$P(C|\mathcal{D}) = \frac{P(\mathcal{D}|C)P(C)}{P(\mathcal{D})} = \frac{0.9 \cdot 0.2}{0.62} = \frac{0.18}{0.62}$$

Notice that the total probability $P(\mathcal{D})$ is the same in each of the denominators and that it is the sum of the three numerators. We can organize all of this very neatly in a Bayesian update table:

hypothesis	prior	likelihood	Bayes numerator	posterior (numerator/$P(\mathcal{D})$)
$\mathcal{H}$	$P(\mathcal{H})$	$P(\mathcal{D}\|\mathcal{H})$	$P(\mathcal{D}\|\mathcal{H})P(\mathcal{H})$	$P(\mathcal{H}\|\mathcal{D})$
A	0.4	0.5	0.2	0.3226
B	0.4	0.6	0.24	0.3871
C	0.2	0.9	0.18	0.2903
total	1	NO SUM	$P(\mathcal{D}) = 0.62$	1

The Bayes numerator is the product of the prior and the likelihood. We see in each of the Bayes' formula computations above that the posterior probability is obtained by dividing the Bayes numerator by $P(\mathcal{D}) = 0.62$. We also see that the law of law of total probability says that $P(\mathcal{D})$ is the sum of the entries in the Bayes numerator column.

Bayesian updating: The process of going from the prior probability $P(\mathcal{H})$ to the posterior $P(\mathcal{H}|\mathcal{D})$ is called Bayesian updating. Bayesian updating uses the data to alter our understanding of the probability of each of the possible hypotheses.

3.1 Important things to notice

1. There are two types of probabilities: Type one is the standard probability of data, e.g. the probability of heads is $p = 0.9$. Type two is the probability of the hypotheses, e.g. the probability the chosen coin is type A, B or C. This second type has prior (before the data) and posterior (after the data) values.

2. The posterior (after the data) probabilities for each hypothesis are in the last column. We see that coin B is now the most probable, though its probability has decreased from a prior probability of 0.4 to a posterior probability of 0.39. Meanwhile, the probability of type C has increased from 0.2 to 0.29.

3. The Bayes numerator column determines the posterior probability column. To compute the latter, we simply divided each numerator by $P(\mathcal{D})$, i.e. rescaled the Bayes numerators so that they sum to 1.

4. If all we care about is finding the most likely hypothesis, the Bayes numerator works as well as the normalized posterior.

5. The likelihood column does not sum to 1. The likelihood function is *not* a probability function.

6. The posterior probability represents the outcome of a 'tug-of-war' between the likelihood and the prior. When calculating the posterior, a large prior may be deflated by a small likelihood, and a small prior may be inflated by a large likelihood.

7. The maximum likelihood estimate (MLE) for Example 1 is hypothesis C, with a likelihood $P(\mathcal{D}|C) = 0.9$. The MLE is useful, but you can see in this example that it is not the entire story, since type B has the greatest posterior probability.

Terminology in hand, we can express Bayes' theorem in various ways:

$$P(\mathcal{H}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathcal{H})P(\mathcal{H})}{P(\mathcal{D})}$$

$$P(\text{hypothesis}|\text{data}) = \frac{P(\text{data}|\text{hypothesis})P(\text{hypothesis})}{P(\text{data})}$$

With the data fixed, the denominator $P(\mathcal{D})$ just serves to normalize the total posterior probability to 1. So we can also express Bayes' theorem as a statement about the proportionality of two functions of $\mathcal{H}$ (i.e, of the last two columns of the table).

$$P(\text{hypothesis}|\text{data}) \propto P(\text{data}|\text{hypothesis})P(\text{hypothesis})$$

This leads to the most elegant form of Bayes' theorem in the context of Bayesian updating:

$$\boxed{\text{posterior} \propto \text{likelihood} \times \text{prior}}$$

3.2 Prior and posterior probability mass functions

Earlier in the course we saw that it is convenient to use random variables and probability mass functions. To do this we had to assign values to events (head is 1 and tails is 0). We will do the same thing in the context of Bayesian updating.

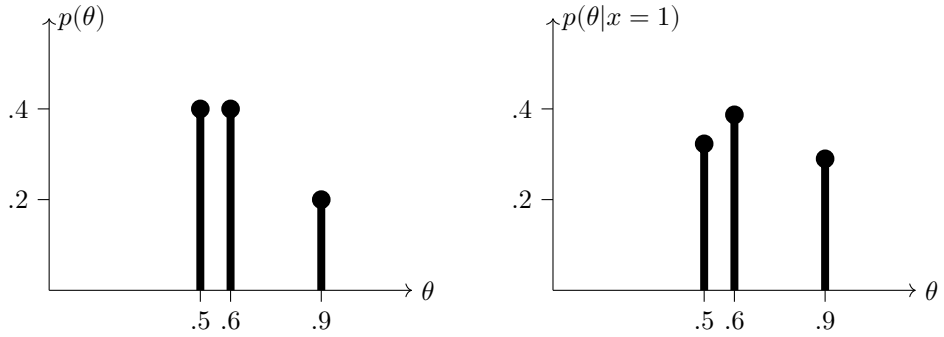
Our standard notations will be:

- θ is the value of the hypothesis.
- $p(\theta)$ is the prior probability mass function of the hypothesis.
- $p(\theta|\mathcal{D})$ is the posterior probability mass function of the hypothesis given the data.
- $p(\mathcal{D}|\theta)$ is the likelihood function. (This is not a pmf!)

In Example 1 we can represent the three hypotheses A, B, and C by $\theta = 0.5, 0.6, 0.9$. For the data we'll let $x = 1$ mean heads and $x = 0$ mean tails. Then the prior and posterior probabilities in the table define the prior and posterior probability mass functions.

Hypothesis	θ	prior pmf $p(\theta)$	posterior pmf $p(\theta\|x=1)$
A	0.5	$P(A) = p(0.5) = 0.4$	$P(A\|\mathcal{D}) = p(0.5\|x=1) = 0.3226$
B	0.6	$P(B) = p(0.6) = 0.4$	$P(B\|\mathcal{D}) = p(0.6\|x=1) = 0.3871$
C	0.9	$P(C) = p(0.9) = 0.2$	$P(C\|\mathcal{D}) = p(0.9\|x=1) = 0.2903$

Here are plots of the prior and posterior pmf's from the example.



Prior pmf $p(\theta)$ and posterior pmf $p(\theta|x=1)$ for Example 1

If the data was different then the likelihood column in the Bayesian update table would be different. We can plan for different data by building the entire likelihood table ahead of time. In the coin example there are two possibilities for the data: the toss is heads or the toss is tails. So the full likelihood table has two likelihood columns:

hypothesis	likelihood $p(x\|\theta)$	
θ	$p(x=0\|\theta)$	$p(x=1\|\theta)$
0.5	0.5	0.5
0.6	0.4	0.6
0.9	0.1	0.9

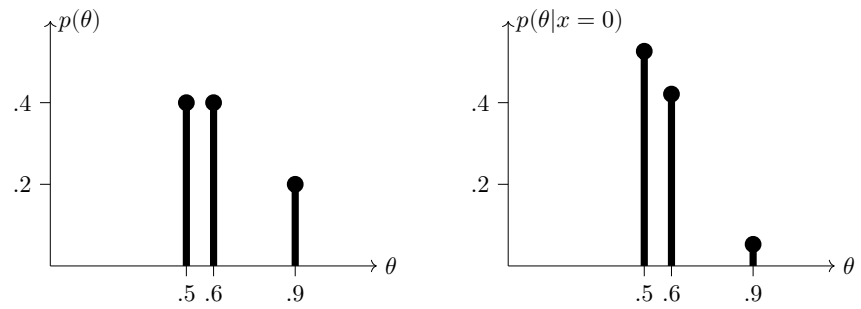
Important convention. Notice that in the above table we used the value of θ as the hypothesis. Of course, hypothesizing '$\theta = 0.5$' is exactly the same as hypothesizing 'the coin is type A'. It is also useful in settings where we haven't named all the possible hypotheses.

Example 2. Using the notation $p(\theta)$, etc., redo Example 1 assuming the flip was tails.

Solution: Since the data has changed, the likelihood column in the Bayesian update table is now for $x = 0$. That is, we must take the $p(x=0|\theta)$ column from the likelihood table.

hypothesis	prior	likelihood	Bayes numerator	posterior
θ	$p(\theta)$	$p(x=0\,\|\,\theta)$	$p(x=0\,\|\,\theta)p(\theta)$	$p(\theta\,\|\,x=0)$
0.5	0.4	0.5	0.2	0.5263
0.6	0.4	0.4	0.16	0.4211
0.9	0.2	0.1	0.02	0.0526
total	1	NO SUM	0.38	1

Now the probability that $\theta = 0.5$, i.e. the coin is type A, has increased from 0.4 to 0.5263, while the probability that $\theta = 0.9$, i.e the coin is type C, has decreased from 0.2 to only 0.0526. Here are the corresponding plots:



Prior pmf $p(\theta)$ and posterior pmf $p(\theta|x=0)$

3.3 Food for thought.

Suppose that in Example 1 you didn't know how many coins of each type were in the drawer. You picked one at random and got heads. How would you go about deciding which hypothesis (coin type) if any was most supported by the data?

4 Updating again and again

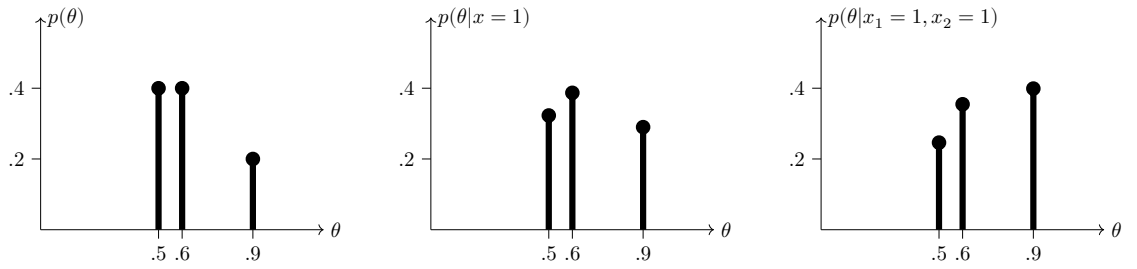
In life we are continually updating our beliefs with each new experience of the world. In Bayesian inference, after updating the prior to the posterior, we can take more data and update again! For the second update, the posterior from the first data becomes the prior for the second data.

Example 3. Suppose you have picked a coin as in Example 1. You flip it once and get heads. Then you flip the same coin and get heads again. What is the probability that the coin was type A? Type B? Type C?

Solution: As we update several times the table gets big, so we use a smaller font to fit it in:

hypothesis	prior	likelihood 1	Bayes numerator 1	likelihood 2	Bayes numerator 2	posterior 2
θ	$p(\theta)$	$p(x_1=1\|\theta)$	$p(x_1=1\|\theta)p(\theta)$	$p(x_2=1\|\theta)$	$p(x_2=1\|\theta)p(x_1=1\|\theta)p(\theta)$	$p(\theta\|x_1=1,x_2=1)$
0.5	0.4	0.5	0.2	0.5	0.1	0.2463
0.6	0.4	0.6	0.24	0.6	0.144	0.3547
0.9	0.2	0.9	0.18	0.9	0.162	0.3990
total	1	NO SUM		NO SUM	0.406	1

Note that the second Bayes numerator is computed by multiplying the first Bayes numerator and the second likelihood; since we are only interested in the final posterior, there is no need to normalize until the last step. As shown in the last column and plot, after two heads the type C hypothesis has finally taken the lead!



prior $p(\theta)$, first posterior $p(\theta|x_1 = 1)$, second posterior $p(\theta|x_1 = 1, x_2 = 1)$

5 Appendix: the base rate fallacy

Example 4. A screening test for a disease is both sensitive and specific. By that we mean it is usually positive when testing a person with the disease and usually negative when testing someone without the disease. Let's assume the true positive rate is 99% and the false positive rate is 2%. Suppose the prevalence of the disease in the general population is 0.5%. If a random person tests positive, what is the probability that they have the disease?

Solution: As a review we first do the computation using trees. Next we will redo the computation using tables.

Let's use notation established above for hypotheses and data: let $\mathcal{H}_+$ be the hypothesis (event) that the person has the disease and let $\mathcal{H}_-$ be the hypothesis they do not. Likewise, let $\mathcal{T}_+$ and $\mathcal{T}_-$ represent the data of a positive and negative screening test respectively. We are asked to compute $P(\mathcal{H}_+|\mathcal{T}_+)$.

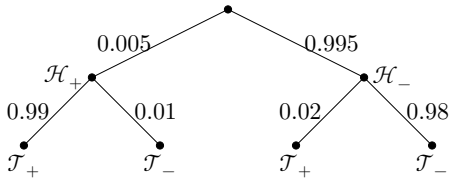
We are given

$$P(\mathcal{T}_+|\mathcal{H}_+) = 0.99, \quad P(\mathcal{T}_+|\mathcal{H}_-) = 0.02, \quad P(\mathcal{H}_+) = 0.005.$$

From these we can compute the false negative and true negative rates:

$$P(\mathcal{T}_-|\mathcal{H}_+) = 0.01, \quad P(\mathcal{T}_-|\mathcal{H}_-) = 0.98$$

All of these probabilities can be displayed quite nicely in a tree.



Bayes' theorem yields

$$P(\mathcal{H}_+|\mathcal{T}_+) = \frac{P(\mathcal{T}_+|\mathcal{H}_+)P(\mathcal{H}_+)}{P(\mathcal{T}_+)} = \frac{0.99 \cdot 0.005}{0.99 \cdot 0.005 + 0.02 \cdot 0.995} = 0.19920 \approx 20\%$$

Now we redo this calculation using a Bayesian update table:

hypothesis	prior	likelihood	Bayes numerator	posterior
$\mathcal{H}$	$P(\mathcal{H})$	$P(\mathcal{T}_+\|\mathcal{H})$	$P(\mathcal{T}_+\|\mathcal{H})P(\mathcal{H})$	$P(\mathcal{H}\|\mathcal{T}_+)$
$\mathcal{H}_+$	0.005	0.99	0.00495	0.19920
$\mathcal{H}_-$	0.995	0.02	0.01990	0.80080
total	1	NO SUM	$P(\mathcal{T}_+) = 0.02485$	1

The table shows that the posterior probability $P(\mathcal{H}_+|\mathcal{T}_+)$ that a person with a positive test has the disease is about 20%. This is far less than the sensitivity of the test (99%) but much higher than the prevalence of the disease in the general population (0.5%).

MIT OpenCourseWare
https://ocw.mit.edu

18.05 Introduction to Probability and Statistics
Spring 2022

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms.

FINANCIAL ECONOMETRICS

MODULE 5 | LESSON 2

GARCH MODEL

Reading Time	60 minutes
Prior Knowledge	Basics of Time Series, ARIMA model, ARCH model
Keywords	GARCH(1,1), GARCH(p,q), Engle's ARCH Lagrange Multiplier (LM) Test, Nyblom Stability Test, Sign Bias Test, Adjusted Pearson Goodness-of-Fit Test

In the last lesson, we discussed using the ARCH process to model non-constant variances in financial asset returns. We talked about the common features of financial asset returns, especially the existence of time-varying volatility in the data. We introduced the ARCH process to model non-constant variance specifically. We went through the key properties of the ARCH process. In this lesson, we are going to continue our journey of modeling non-constant variance. We are going to expand ARCH(1) model and introduce a general model: GARCH. We will go through the basics of GARCH and apply GARCH to Google stock returns.

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from arch import arch_model
from scipy import stats
from statsmodels.stats.diagnostic import acorr_ljungbox

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
```

In [2]:
```
# Download the dataset
m5_data = pd.read_csv("M5. goog_eur_10.csv")

# Convert date variable to date format
m5_data["Date2"] = pd.to_datetime(m5_data["Date"], format="%m/%d/%Y")
goog = m5_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")

goog["GOOGLE_R"] = np.log(goog["GOOGLE"]).diff().dropna()
```

1. GARCH Model

1.1 GARCH(1,1) Model

GARCH stands for generalized autoregressive conditional heteroskedasticity, and it is an extension of the ARCH model. In the variance function of the ARCH model, a lagged variance term is added to the right of the equation to become a GARCH model. We can formally define GARCH(1,1) as follows:

$$r_t = \sigma_t e_t$$

Where $e_t \sim iidN(0,1)$ and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

with $\alpha_0 > 0,\ 0 \leq \alpha_1 < 1$ and $0 \leq \beta_1 < 1$

The requirements of $\alpha_0 > 0,\ 0 \leq \alpha_1 < 1$ and $0 \leq \beta_1 < 1$ will make sure r_t is stationary and the variance is non-negative and finite.

Structurally, the relationship between GARCH and ARCH models is similar to that between ARMA and AR models. For ARCH(1) model, we can write it as follows:

$$r_t = e_t \sqrt{(\alpha_0 + \alpha_1 r_{t-1}^2)}$$

In the above ARCH(1) model, we see we use r_{t-1} to predict r_t. This concept is the same as for AR(1) model, so we sometimes call this term the AR part of the ARCH model.

Similarly, let's write a GARCH(1,1) model in the following form:

$$r_t = e_t \sqrt{(\alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2)}$$

In the GARCH(1,1) formula, we not only have r_{t-1} to predict r_t but also have σ_{t-1} to predict r_t. The reason why GARCH(1,1) is like ARMA(1,1) is because we have r_{t-1} like AR part and σ_{t-1} like MA part for GARCH.

So why do we need the GARCH model? Even though the ARCH model seems to be able to cover all the features of asset returns, it usually requires many parameters to properly model the asset return. In order to tackle this issue, the GARCH model was developed so that there is no need to include too many terms in a model.

1.2 Properties of GARCH(1,1) Model

Most of the properties for the GARCH(1,1) model are similar to the ARCH(1,1) model. In this section, we will highlight the key properties for GARCH(1,1).

1.2.1 r_t is Conditionally Normally Distributed

The above statement follows the same proof as we describe in ARCH(1) model. We can summarize it in the following formula:

$$r_t | r_{t-1} \sim N\left(0, \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2\right)$$

1.2.2 r_t^2 is An ARMA(1,1) Process

We know that GARCH(1,1) is defined as follows:

$$r_t = \sigma_t e_t \tag{1}$$

and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2 \tag{2}$$

If we subtract the equation (1) and equation (2), we can get the following new equation:

$$r_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2 + \nu_t$$

where $\nu_t = \sigma_t^2(e_t^2 - 1)$ and ν_t is white noise.

Since $\nu_{t-1} = \sigma_{t-1}^2(e_{t-1}^2 - 1)$ and $r_{t-1}^2 = \sigma_{t-1}^2 e_{t-1}^2$, we can get $\sigma_{t-1}^2 = r_{t-1}^2 - \nu_{t-1}$. Plug this last result into the new equation from the subtraction of two equations from the definition, we can get the following equation:

$$r_t^2 = \alpha_0 + (\alpha_1 + \beta_1) r_{t-1}^2 + \nu_t - \beta_1 \nu_{t-1}$$

The above equation is an ARMA(1,1) with the squared asset return as the AR part and white noise as the MA part. To ensure stationarity, $0 < \alpha_1 + \beta_1 < 1$.

1.2.3 r_t is White Noise

Using the same calculation from the ARCH model, we can show that GARCH(1,1) is white noise. We can write unconditional r_t distribution as follows:

$$r_t \sim \text{white noise}\left(0, \frac{\alpha_0}{1 - (\alpha_1 + \beta_1)}\right)$$

where $0 < \alpha_1 + \beta_1 < 1$

This shows similar results as the ARCH(1) model. r_t is unconditionally homoscedastic when $\alpha_1 + \beta_1 < 1$. So when r_t is stationary $(0 \leq \alpha_1 < 1)$, r_t is conditionally heteroskedastic but unconditionally homoscedastic. Also, even though we use asset return as an example for GARCH(1,1) here, it was originally applied to the squared error term from a time series regression model.

1.2.4 GARCH(1,1) Has Infinite Memory

By iterating the variance equation from GARCH(1,1), we can write the equation as follows:

$$\begin{aligned}
\sigma_{t^2} &= \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2 \\
&= \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1(\alpha_0 + \alpha_1 r_{t-2}^2 + \beta_1 \sigma_{t-2}^2) \\
&= \alpha_0 + \alpha_0\beta_1 + \alpha_1 r_{t-1}^2 + \alpha_1\beta_1 r_{t-2}^2 + \beta_1^2(\alpha_0 + \alpha_1 r_{t-3}^2 + \beta_1 \sigma_{t-3}^2) \\
&= \alpha_0(1 + \beta_1 + \beta_1^2 + \beta_1^3 + \cdots) + \alpha_1 \sum_{i=1}^{\infty} \beta_1^{i-1} r_{t-i}^2
\end{aligned}$$

We can see that σ_{t^2} can be expanded as the infinite sum of past asset returns. This means that all the past asset returns still have influence on today's conditional variance. Alternatively, we can say GARCH(1,1) has infinite memory. All the coefficients should decay in order to maintain stationarity of the GARCH(1,1) process. Because of this feature, the GARCH model is usually better to model time series data than the ARCH(1) model.

1.3 GARCH(p, q) Model

As with other time series models we have talked about so far, we can easily extend GARCH(1,1) model to GARCH(p, q) model.

We can define GARCH(p, q) as follows:

$$r_t = \sigma_t e_t$$

where $e_t \sim iidN(0,1)$ and the variance equation is:

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \cdots + \alpha_p r_{t-p}^2 + \beta_1 \sigma_{t-1}^2 + \beta_2 \sigma_{t-2}^2 + \cdots + \beta_q \sigma_{t-q}^2$$

where $\alpha_0 > 0,\ \alpha_i \geq 0$ for $i = 1, \cdots, p$ and $\beta_j \geq 0$ for $j = 1, \cdots, q$

We can now rewrite the variance equation with a backshift operator as follows:

$$\sigma_t^2 = \alpha_0 + \alpha(B) r_t^2 + \beta(B) \sigma_t^2$$

Where

$$\alpha(B) = \alpha_1 B + \alpha_2 B^2 + \cdots + \alpha_p B^p$$

and

$$\beta(B) = \beta_1 B + \beta_2 B^2 + \cdots + \beta_q B^q$$

For the above characteristic equations, the absolute values of the roots of B must be greater than 1 to ensure that r_t is stationary.

2. Model Diagnostics

There are a number of tests that are usually used to check for ARCH and GARCH models. We will cover some of them in this section.

2.1 Ljung-Box Test

We introduced this test in last module. The Ljung-Box test is used to see if there is serial correlation in a time series. We use this test if there is an overall correlation in the residuals or squared residuals from a time series model, including ARCH and GARCH models. The null hypothesis (H_0) is there is no serial correlation in the time series. Hence, if the p-value < 0.05, we will reject the null hypothesis, and the time series has a serial correlation issue.

2.2 Information Criteria

In the last module, we also introduced Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC). These are the metrics used to evaluate overall model specifications. The lower the numbers of AIC and/or BIC, the better the model specifications. There are other information criteria, but we usually focus on AIC or BIC.

2.3 Log Likelihood Number

We also use the log likelihood number to evaluate the goodness-of-fit of a model if it is fitted with the maximum likelihood method. The lower the log likelihood number, the better the model fit.

2.4 Engle's ARCH Lagrange Multiplier (LM) Test

Engle's ARCH LM test, or Engle's ARCH test, determines if there is a serial correlation (autocorrelation) issue in the squared residuals of a time series model. The null hypothesis (H_0) is that there is no serial correlation in the squared residuals.

2.5 Nyblom Stability Test

This is the test to check if there is a regime change for the time series over time. The test checks if there is any structural change among variables or if coefficients are stable with no variation overtime. The null hypothesis (H_0) is that there is no regime change and the variances for coefficients are 0.

2.6 Sign Bias Test

This is the test to check if shocks not modeled by the model have an impact on volatilities. The sign bias test is used to determine the impact of large and small shocks on volatility. The negative sign bias test is to check if negative shocks not modeled by the model have an impact on volatility. The positive sign bias test is to check if positive shocks not modeled by the model have an impact on volatility. The null hypothesis (H_0) is that there is no impact from shocks not modeled by the model.

2.7 Adjusted Pearson Goodness-of-Fit Test

This test compares the empirical distribution of standard residuals with the selected theoretical distribution. It is a chi-squared test. The null hypothesis (H_0) is that the empirical distribution of the standard residuals is equal to theoretical distribution.

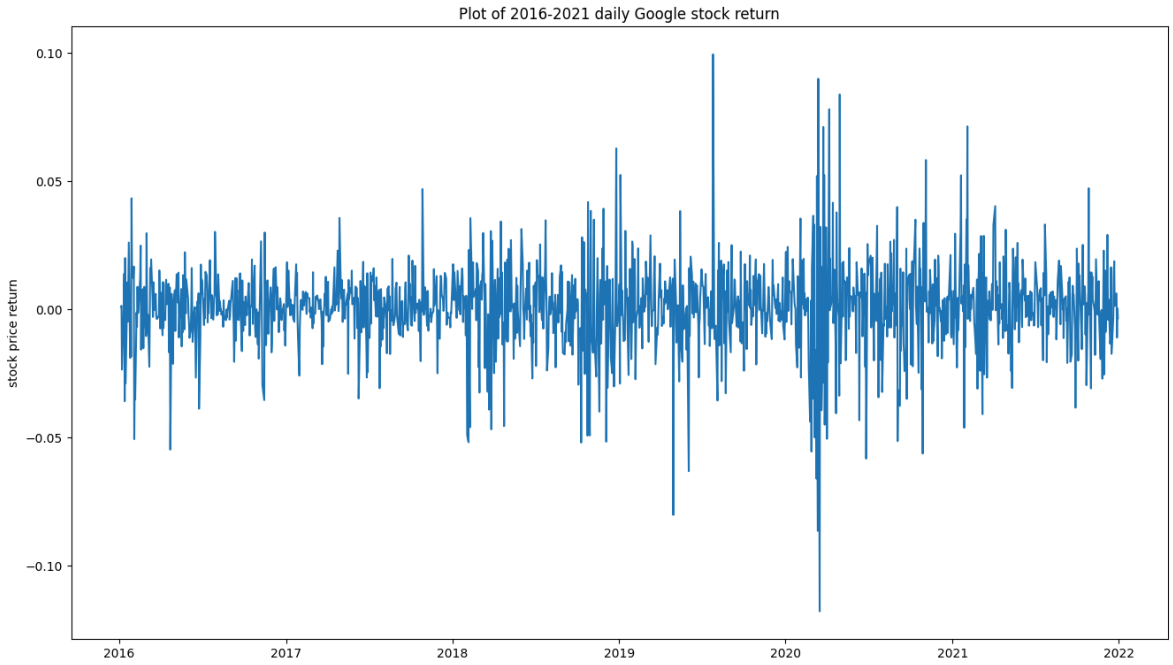
3. GARCH Application: Google Stock Return

In this section, we are going to use Google stock return data from 2016 to 2021 to run a GARCH model. We'll start by looking into the data first before building the model.

3.1 Data Exploration

Figure 1: Google Stock Returns from 2016 to 2021

In [3]:
```
# Google Stock Returns from 2016 to 2021
plt.plot(goog.GOOGLE_R)
plt.ylabel("stock price return")
plt.title("Plot of 2016-2021 daily Google stock return")
plt.show()
```



In figure 1, we can see Google stock returns are oscillating around 0. We can see there are periods of time when the volatilities of returns are higher than other times. This pattern of volatility in Google stock return data is the phenomenon that the GARCH model tries to capture.

Figure 2: Histogram of Google Stock Returns

In [4]:
```
# Histogram Histogram of Google Stock Returns
plt.figure(figsize=(11, 5))
goog_r = goog.GOOGLE_R.dropna()

x = np.linspace(min(goog_r), max(goog_r), len(goog_r))
values, bins, _ = plt.hist(goog_r, bins=25)  # Histogram
```

```
(mu, sigma) = stats.norm.fit(goog_r)
plt.plot(x, stats.norm.pdf(x, mu, sigma) * sum(values * np.diff(bins)), "r")  #

plt.title("Google Stock Return Histogram")
plt.show()
```

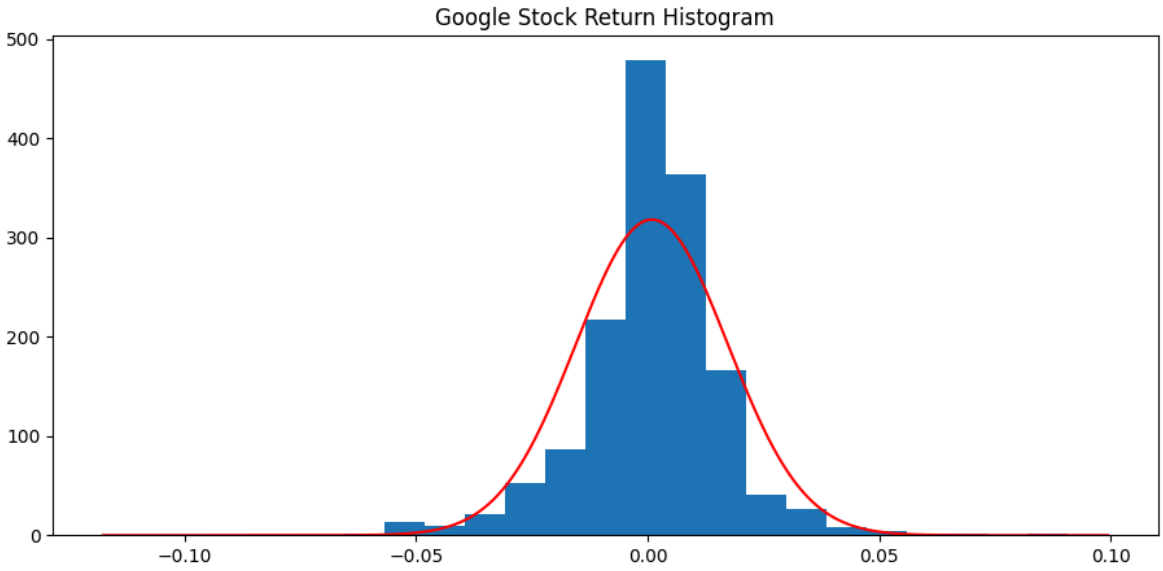
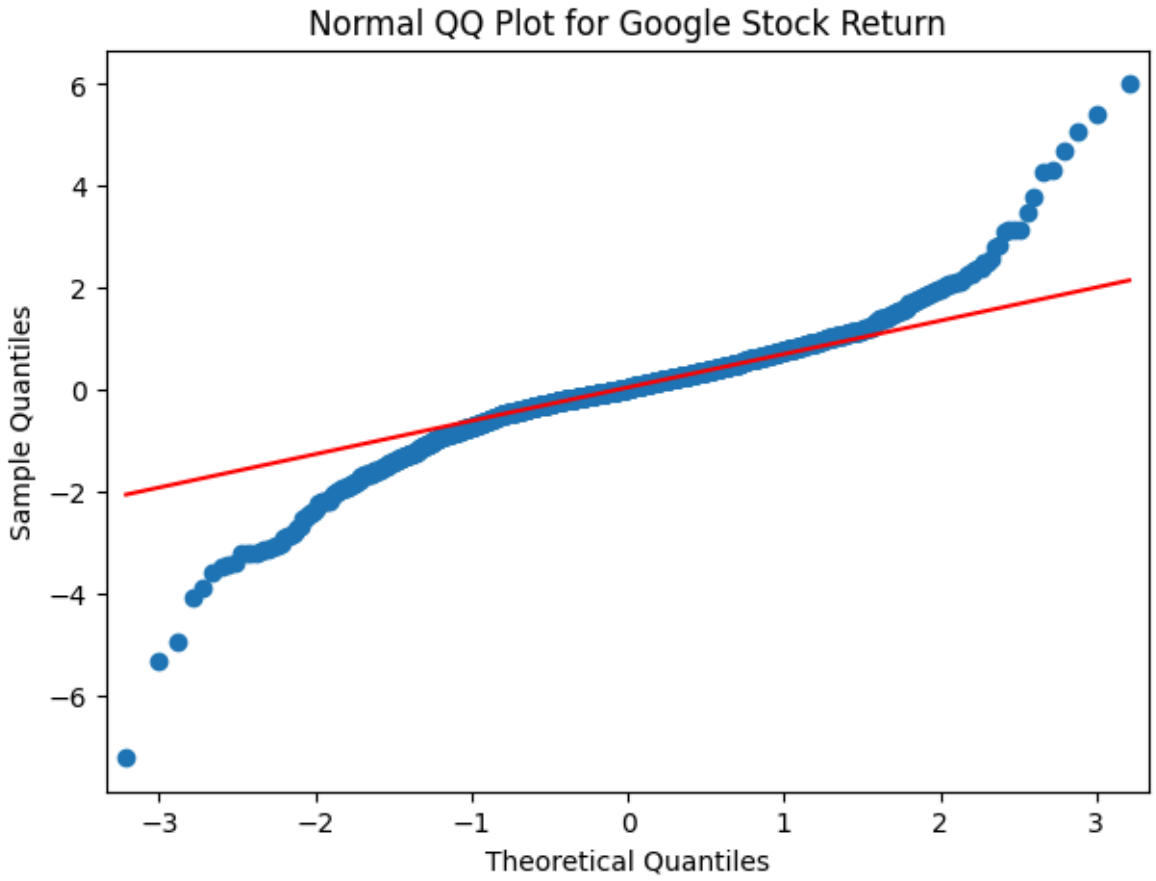


Figure 2 shows the histogram of Google's stock returns and a normal distribution curve overlaid on it. We can see Google's stock return distribution has a higher peak and fatter tail than normal distribution.

Figure 3: Normal QQ Plot for Google Stock Returns

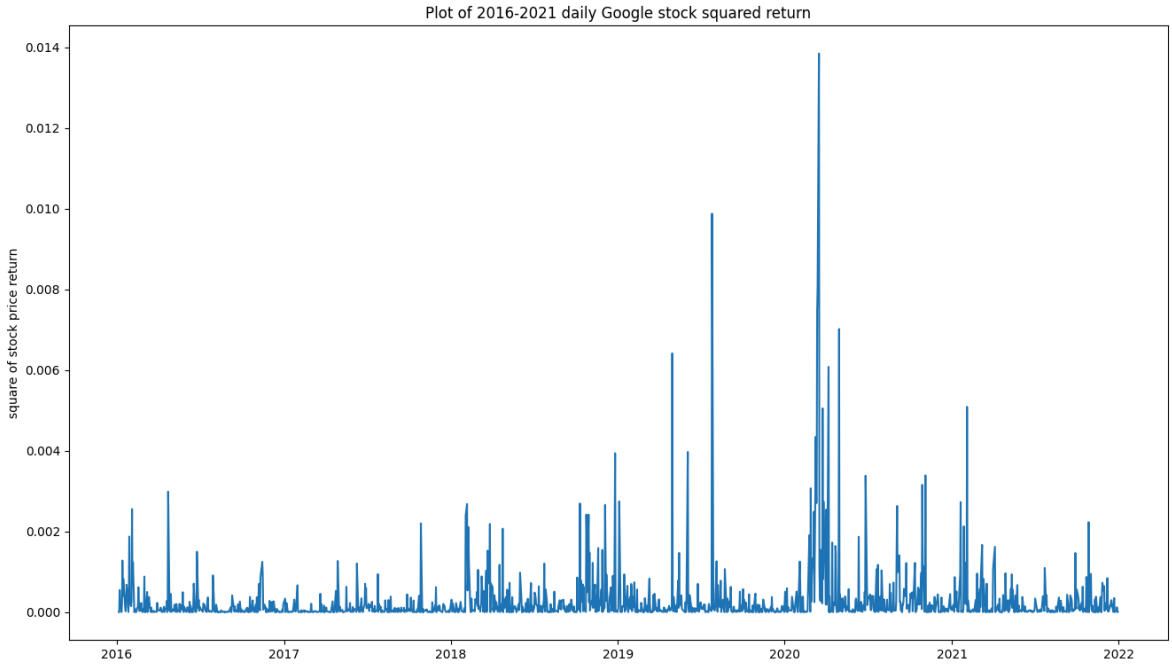
In [5]:
```
# Normal QQ Plot for Google Stock Returns
qq = sm.qqplot(goog_r, stats.norm, fit=True, line="q")
qq.set_size_inches((7, 5))
plt.title("Normal QQ Plot for Google Stock Return")
plt.show()
```



In figure 3, we can clearly see that Google stock returns have a fat-tailed distribution compared to normal distribution.

Figure 4: Google Stock Squared Returns from 2016 to 2021

In [6]:
```
# Google Stock Squared Returns
plt.plot(goog.GOOGLE_R**2)
plt.ylabel("square of stock price return")
plt.title("Plot of 2016-2021 daily Google stock squared return")
plt.show()
```



From figure 4, we can see there are serial correlations to the squared return for Google stock. The high returns are usually followed by more volatilities.

Figure 5: ACF and PACF Plots for Google Stock Returns and Squared Returns

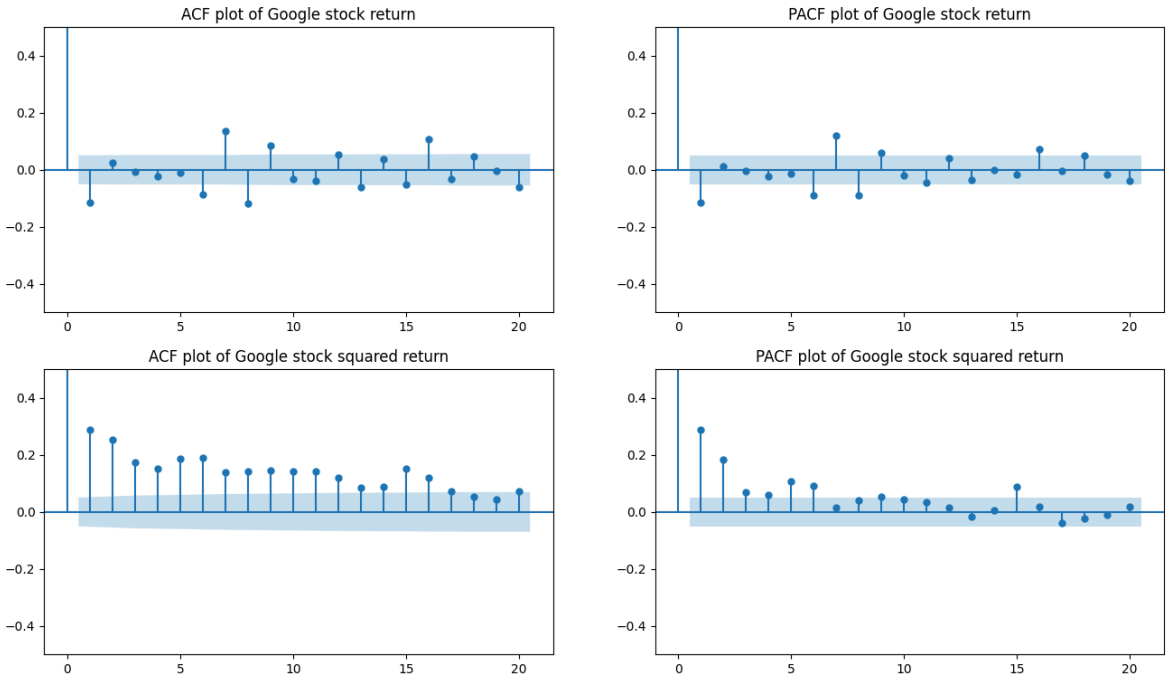
In [7]:
```
# ACF and PACF Plots for Google Stock Returns and Squared Returns
fig, ax = plt.subplots(2, 2)

sm.graphics.tsa.plot_acf(goog.GOOGLE_R.dropna(), lags=20, ax=ax[0, 0])
ax[0, 0].set(title="ACF plot of Google stock return")
ax[0, 0].set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_pacf(goog.GOOGLE_R.dropna(), lags=20, ax=ax[0, 1])
ax[0, 1].set(title="PACF plot of Google stock return")
ax[0, 1].set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_acf(goog.GOOGLE_R.dropna() ** 2, lags=20, ax=ax[1, 0])
ax[1, 0].set(title="ACF plot of Google stock squared return")
ax[1, 0].set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_pacf(goog.GOOGLE_R.dropna() ** 2, lags=20, ax=ax[1, 1])
ax[1, 1].set(title="PACF plot of Google stock squared return")
ax[1, 1].set_ylim([-0.5, 0.5])
plt.show()
```



In figure 5, ACF and PACF plots for Google stock returns and squared returns all show patterns like the ARMA model. Hence, we can start with the GARCH(1,1) model for model specifications.

3.2 Model Estimation and Diagnostics

Let's start with the GARCH(1,1) model using Google stock return data. Figure 6 shows the GARCH(1,1) model result.

Figure 6: GARCH(1,1) Model with Normal White Noise

In [8]:
```
# GARCH(1,1) Model with Normal White Noise
garch11_spec = arch_model(
    goog.GOOGLE_R.dropna(),
    vol="GARCH",
    p=1,
    q=1,
    mean="AR",
    dist="Normal",
    rescale=True,
)
garch11_fit = garch11_spec.fit()
garch11_fit.summary()
```

```
Iteration:      1,   Func. Count:      6,   Neg. LLF: 18102370089.838078
Iteration:      2,   Func. Count:     14,   Neg. LLF: 199289710361.31866
Iteration:      3,   Func. Count:     23,   Neg. LLF: 2905.730302628701
Iteration:      4,   Func. Count:     31,   Neg. LLF: 2886.925378396835
Iteration:      5,   Func. Count:     38,   Neg. LLF: 2728.1622986296334
Iteration:      6,   Func. Count:     43,   Neg. LLF: 2727.9777720000106
Iteration:      7,   Func. Count:     48,   Neg. LLF: 2727.9469065715693
Iteration:      8,   Func. Count:     53,   Neg. LLF: 2727.9393448273977
Iteration:      9,   Func. Count:     58,   Neg. LLF: 2727.937888260677
Iteration:     10,   Func. Count:     63,   Neg. LLF: 2727.9370769987204
Iteration:     11,   Func. Count:     68,   Neg. LLF: 2727.937067484191
Iteration:     12,   Func. Count:     72,   Neg. LLF: 2727.9370674840043
Optimization terminated successfully    (Exit mode 0)
            Current function value: 2727.937067484191
            Iterations: 12
            Function evaluations: 72
            Gradient evaluations: 12
```

Out[8]:

AR - GARCH Model Results

Dep. Variable:	GOOGLE_R	**R-squared:**	0.000
Mean Model:	AR	**Adj. R-squared:**	0.000
Vol Model:	GARCH	**Log-Likelihood:**	-2727.94
Distribution:	Normal	**AIC:**	5463.87
Method:	Maximum Likelihood	**BIC:**	5485.14
		No. Observations:	1504
Date:	Sun, Dec 07 2025	**Df Residuals:**	1503
Time:	06:22:37	**Df Model:**	1

Mean Model

	coef	std err	t	P>|t|	95.0% Conf. Int.
Const	0.1114	4.019e-02	2.772	5.578e-03	[3.262e-02, 0.190]

Volatility Model

	coef	std err	t	P>|t|	95.0% Conf. Int.
omega	0.1250	9.366e-02	1.335	0.182	[-5.858e-02, 0.309]
alpha[1]	0.0822	3.786e-02	2.170	2.999e-02	[7.959e-03, 0.156]
beta[1]	0.8671	6.959e-02	12.461	1.223e-35	[0.731, 1.004]

Covariance estimator: robust

In [9]:

```
# Diagnostic tests for GARCH(1,1) Model with Normal White Noise
print("GARCH(1,1) Model with Normal White Noise\n")

# Ljung-Box test and the Box-Pierce test
print("Ljung-Box and Box-Pierce tests on stanrdized residuals")
print(acorr_ljungbox(garch11_fit.std_resid, boxpierce=True))

print("\nLjung-Box and Box-Pierce tests on stanrdized squared residuals")
print(acorr_ljungbox(garch11_fit.std_resid**2, boxpierce=True))

# ARCH LM test for conditional heteroskedasticity
print("\nARCH LM test for conditional heteroskedasticity")
print(garch11_fit.arch_lm_test(standardized=True))
```

```
GARCH(1,1) Model with Normal White Noise

Ljung-Box and Box-Pierce tests on stanrdized residuals
      lb_stat  lb_pvalue    bp_stat  bp_pvalue
1    0.453025   0.500902   0.452122   0.501329
2    0.576445   0.749595   0.575215   0.750056
3    1.467876   0.689704   1.463686   0.690677
4    2.000287   0.735706   1.993976   0.736867
5    6.544690   0.256763   6.517257   0.259087
6    7.905635   0.245099   7.870972   0.247710
7   13.250117   0.066244  13.183515   0.067761
8   17.537745   0.024972  17.442673   0.025816
9   17.725621   0.038494  17.629176   0.039728
10  18.162461   0.052284  18.062536   0.053918

Ljung-Box and Box-Pierce tests on stanrdized squared residuals
     lb_stat  lb_pvalue   bp_stat  bp_pvalue
1   0.006414   0.936170  0.006401   0.936234
2   0.101005   0.950752  0.100741   0.950877
3   0.103420   0.991424  0.103148   0.991457
4   0.115280   0.998401  0.114961   0.998410
5   0.636968   0.986248  0.634224   0.986383
6   0.641445   0.995669  0.638677   0.995720
7   1.616170   0.978013  1.607577   0.978350
8   2.954319   0.937191  2.936840   0.938258
9   3.117530   0.959452  3.098859   0.960246
10  3.119505   0.978470  3.100818   0.978951

ARCH LM test for conditional heteroskedasticity
ARCH-LM Test
H0: Standardized residuals are homoskedastic.
ARCH-LM Test
H1: Standardized residuals are conditionally heteroskedastic.
Statistic: 15.7513
P-value: 0.8968
Distributed: <scipy.stats._distn_infrastructure.rv_continuous_frozen object at 0x
704493092650>
```

Figure 6 shows the GARCH(1,1) model with normal white noise. Everything looks good from the model results except that we can see one of the coefficient estimates is not significant. Now let's look on some diagnostic plots.

Figure 7: Model Diagnostic Plots for the GARCH(1,1) Model with Normal White Noise

In [ ]:
```python
# Model Diagnostic Plots for the GARCH(1,1) Model with Normal White Noise
fig, ax = plt.subplots(4, 3, figsize=(16, 12))

# Figure Row 1 Column 1
ax[0, 0].plot(goog.GOOGLE_R)
ax[0, 0].plot(2.0 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].plot(-2.0 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].tick_params(labelrotation=45)
ax[0, 0].set_title("Series with 2 conditional SD")
ax[0, 0].set_ylabel("Return")

# Figure Row 1 Column 2
VaR_1 = stats.norm.ppf(0.99)
```

```
ax[0, 1].plot(goog.GOOGLE_R)
ax[0, 1].plot(VaR_1 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].plot(-VaR_1 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].tick_params(labelrotation=45)
ax[0, 1].set_title("Series with 1% VaR Limits")
ax[0, 1].set_ylabel("Return")

# Figure Row 1 Column 3
ax[0, 2].plot(garch11_fit.conditional_volatility / 100.0)
ax[0, 2].set_title("Conditional SD")
ax[0, 2].tick_params(labelrotation=45)

# Figure Row 2 Column 1
sm.graphics.tsa.plot_acf(garch11_fit.resid / 100.0, lags=20, ax=ax[1, 0])
ax[1, 0].set_title("ACF of Observations")
ax[1, 0].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 2
sm.graphics.tsa.plot_acf(garch11_fit.resid**2, lags=20, ax=ax[1, 1])
ax[1, 1].set_title("ACF of Squared Observations")
ax[1, 1].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 3
sm.graphics.tsa.plot_acf(np.abs(garch11_fit.resid), lags=20, ax=ax[1, 2])
ax[1, 2].set_title("ACF of Absolute Observations")
ax[1, 2].set_ylim([-0.3, 0.3])

# Figure Row 3 Column 1
ax[2, 0].xcorr(
    garch11_fit.resid**2,
    garch11_fit.resid,
    usevlines=True,
    maxlags=30,
    normed=True,
    lw=2,
)
ax[2, 0].set_title("Cross-Correlation of Squared Observations \n vs Actual Obser

# Figure Row 3 Column 2
standaraized_residuals = garch11_fit.std_resid
min_val = np.min(standaraized_residuals)
max_val = np.max(standaraized_residuals)
empirical_density = np.linspace(min_val, max_val, len(standaraized_residuals))
ax[2, 1].plot(empirical_density, stats.norm.pdf(empirical_density), lw=1)
ax[2, 1].set_title("Empirical density of \n standardized residuals")

# Figure Row 3 Column 3
sm.qqplot(garch11_fit.resid, stats.norm, fit=True, line="q", ax=ax[2, 2])
ax[2, 2].set_title("Normal QQ-plot")

# Figure Row 4 Column 1
sm.graphics.tsa.plot_acf(garch11_fit.std_resid, lags=20, ax=ax[3, 0])
ax[3, 0].set_title("ACF of Standardized Residuals")
ax[3, 0].set_ylim([-0.12, 0.12])

# Figure Row 4 Column 2
sm.graphics.tsa.plot_acf((garch11_fit.std_resid) ** 2, lags=20, ax=ax[3, 1])
ax[3, 1].set_title("ACF of Squared Standardized Residuals")
ax[3, 1].set_ylim([-0.12, 0.12])
```

```
ax[3, 2].axis("off")
fig.tight_layout()
plt.show()
```

From figure 7, we can see that normal QQ plot for residuals is still showing fat-tailed distribution. With this piece of information, let's try a GARCH(1,1) model with Student's t-distribution to see if we can improve the model.

Figure 8: GARCH(1,1) Model with Student's t White Noise

In [ ]:
```
# GARCH(1,1) Model with Student's t White Noise
garch11_t_spec = arch_model(
    goog.GOOGLE_R.dropna(),
    vol="GARCH",
    p=1,
    q=1,
    mean="AR",
    dist="StudentsT",  # power=2.0,
    rescale=True,
)
garch11_t_fit = garch11_t_spec.fit()
garch11_t_fit.summary()
```

In [ ]:
```
# Diagnostic tests for GARCH(1,1) Model with Student's t White Noise
print("GARCH(1,1) Model with StudentsT White Noise\n")

# Ljung-Box test and the Box-Pierce test
print("Ljung-Box and Box-Pierce tests on standardized residuals")
print(acorr_ljungbox(garch11_t_fit.std_resid, boxpierce=True))

print("\nLjung-Box and Box-Pierce tests on standardized squared residuals")
print(acorr_ljungbox(garch11_t_fit.std_resid**2, boxpierce=True))

# ARCH LM test for conditional heteroskedasticity
print("\nARCH LM test for conditional heteroskedasticity for")
print(garch11_t_fit.arch_lm_test(standardized=True))
```

In figure 8, now all the coefficient estimates are significant. Let's check the model diagnostic plots.

Figure 9: GARCH(1,1) Model with Student's t White Noise

In [ ]:
```
# Model Diagnostic Plots for the GARCH(1,1) Model with Student's t White Noise
fig, ax = plt.subplots(4, 3, figsize=(16, 12))

# Figure Row 1 Column 1
ax[0, 0].plot(goog.GOOGLE_R)
ax[0, 0].plot(2.0 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].plot(-2.0 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].tick_params(labelrotation=45)
ax[0, 0].set_title("Series with 2 conditional SD")
ax[0, 0].set_ylabel("Return")

# Figure Row 1 Column 2
VaR_1 = stats.t(df=len(goog.GOOGLE_R) - 1).ppf(0.99)
# VaR_1 = stats.norm.ppf(0.99)
ax[0, 1].plot(goog.GOOGLE_R)
```

```
ax[0, 1].plot(VaR_1 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].plot(-VaR_1 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].tick_params(labelrotation=45)
ax[0, 1].set_title("Series with 1% VaR Limits")
ax[0, 1].set_ylabel("Return")

# Figure Row 1 Column 3
ax[0, 2].plot(garch11_t_fit.conditional_volatility / 100.0)
ax[0, 2].set_title("Conditional SD")
ax[0, 2].tick_params(labelrotation=45)

# Figure Row 2 Column 1
sm.graphics.tsa.plot_acf(garch11_t_fit.resid / 100.0, lags=20, ax=ax[1, 0])
ax[1, 0].set_title("ACF of Observations")
ax[1, 0].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 2
sm.graphics.tsa.plot_acf(garch11_t_fit.resid**2, lags=20, ax=ax[1, 1])
ax[1, 1].set_title("ACF of Squared Observations")
ax[1, 1].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 3
sm.graphics.tsa.plot_acf(np.abs(garch11_t_fit.resid), lags=20, ax=ax[1, 2])
ax[1, 2].set_title("ACF of Absolute Observations")
ax[1, 2].set_ylim([-0.3, 0.3])

# Figure Row 3 Column 1
ax[2, 0].xcorr(
    garch11_t_fit.resid**2,
    garch11_t_fit.resid,
    usevlines=True,
    maxlags=30,
    normed=True,
    lw=2,
)
ax[2, 0].set_title("Cross-Correlation of Squared Observations \n vs Actual Obser

# Figure Row 3 Column 2
standaraized_residuals = garch11_t_fit.std_resid
min_val = np.min(standaraized_residuals)
max_val = np.max(standaraized_residuals)
empirical_density = np.linspace(min_val, max_val, len(standaraized_residuals))
ax[2, 1].plot(empirical_density, stats.norm.pdf(empirical_density), lw=1)
ax[2, 1].set_title("Empirical density of \n standardized residuals")

# Figure Row 3 Column 3
sm.qqplot(garch11_t_fit.resid, stats.t, fit=True, line="q", ax=ax[2, 2])
ax[2, 2].set_title("StudentsT QQ-plot")

# Figure Row 4 Column 1
sm.graphics.tsa.plot_acf(garch11_t_fit.std_resid, lags=20, ax=ax[3, 0])
ax[3, 0].set_title("ACF of Standardized Residuals")
ax[3, 0].set_ylim([-0.12, 0.12])

# Figure Row 4 Column 2
sm.graphics.tsa.plot_acf((garch11_t_fit.std_resid) ** 2, lags=20, ax=ax[3, 1])
ax[3, 1].set_title("ACF of Squared Standardized Residuals")
ax[3, 1].set_ylim([-0.12, 0.12])

ax[3, 2].axis("off")
```

```
fig.tight_layout()
plt.show()
```

From figure 9, we can see the StudentsT QQ plot shows the model is a good fit. Hence, GARCH(1,1) with Student's t white noise fits the model pretty well.

3.3 Model Forecast

Figure 10: 20 Steps Ahead Forecast for GARCH(1,1) Model with Student's t White Noise

In [ ]:
```
# 20 Steps Ahead Forecast for GARCH(1,1) Model with Student's t White Noise

# set horizon and forecast
horizon = 20
garch_forecast = garch11_t_fit.forecast(
    reindex=False, horizon=horizon, method="simulation"
)

# reindex data
googr = goog.GOOGLE_R.copy()
googr.index = list(range(len(goog_r) + 1))

# forecast mean
forc_mean = pd.Series(garch_forecast.mean.dropna().squeeze())
forc_mean.index = list(range(len(googr), len(googr) + horizon))

# volatility forecast
variance_fct = pd.DataFrame(data={"Forecast": garch_forecast.variance.values[0]}
variance_fct.index = list(range(len(googr), len(googr) + horizon))
std_fct = [np.sqrt(variance_fct.values[i] * (i + 1)) for i in range(len(variance
volatility_fct = pd.DataFrame(std_fct)

# upper/lower bands
upper_band = googr.values[-1] * (1.0 + 2 * volatility_fct)
upper_band.index = variance_fct.index
lower_band = googr.values[-1] * (1.0 - 2 * volatility_fct)
lower_band.index = variance_fct.index

# Plot
plt.figure(figsize=(16, 7))
plt.xlim(1400, len(googr) + 21)
plt.ylim(-0.05, 0.05)
googr.plot(ylabel="Google Log Return", title="Forecast Volatility")
plt.plot((upper_band + lower_band) / 2, "r--")
plt.fill_between(
    upper_band.index.tolist(),
    upper_band.values.T[0],
    lower_band.values.T[0],
    color="orange",
    alpha=0.5,
)
plt.show()
```

In figure 10, we plot the 20 steps ahead forecast for Google stock return data based on the GARCH(1,1) with Student's t white noise. As the forecast goes farther out, we can see

that the 1-sigma bands get wider.

4. Conclusion

In this lesson, we introduced the GARCH model. We first gave the definition of GARCH(1,1) model. We then went through the properties of the GARCH(1,1) model. We also briefly expanded the GARCH model to a higher order GARCH(p, q) model. Next, we talked about estimation of the GARCH model and the tests for model diagnostics. We finished the lesson with a GARCH application to Google stock returns. In the next lesson, we will go over some details about estimation methods for the GARCH model.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Bayesian Updating: Probabilistic Prediction
Class 12, 18.05
Jeremy Orloff and Jonathan Bloom

1 Learning Goals

1. Be able to use the law of total probability to compute prior and posterior predictive probabilities.

2 Introduction

In the previous class we looked at updating the probability of hypotheses based on data. We can also use the data to update the probability of each possible outcome of a future experiment. In this class we will look at how this is done.

2.1 Probabilistic prediciton; words of estimative probability (WEP)

There are many ways to word predictions:

- Prediction: "It will rain tomorrow."
- Prediction using words of estimative probability (WEP): "It is likely to rain tomorrow."
- Probabilistic prediction: "Tomorrow it will rain with probability 60% (and not rain with probability 40%)."

Each type of wording is appropriate at different times.

In this class we are going to focus on probabilistic prediction and precise quantitative statements. You can see https://en.wikipedia.org/wiki/Words_of_Estimative_Probability for an interesting discussion about the appropriate use of words of estimative probability. The article also contains a list of *weasel words* such as 'might', 'cannot rule out', 'it's conceivable' that should be avoided as almost certain to cause confusion.

There are many places where we want to make a probabilistic prediction. Examples are

- Medical treatment outcomes
- Weather forecasting
- Climate change
- Sports betting
- Elections
- ...

These are all situations where there is uncertainty about the outcome and we would like as precise a description of what could happen as possible.

3 Predictive Probabilities

Probabilistic prediction simply means assigning a probability to each possible outcomes of an experiment.

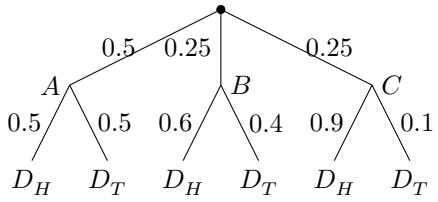
Recall the coin example from the previous class notes: there are three types of coins which are indistinguishable apart from their probability of landing heads when tossed.

- Type A coins are fair, with probability 0.5 of heads
- Type B coins have probability 0.6 of heads
- Type C coins have probability 0.9 of heads

You have a drawer containing 4 coins: 2 of type A, 1 of type B, and 1 of type C. You reach into the drawer and pick a coin at random. We let A stand for the event 'the chosen coin is of type A'. Likewise for B and C.

3.1 Prior predictive probabilities

Before taking data we can compute the probability that our chosen coin will land heads (or tails) if flipped. Let D_H be the event it lands heads and let D_T the event it lands tails. We can use the law of total probability to determine the probabilities of these events. Either by drawing a tree or directly proceeding to the algebra, we get:



$$\begin{aligned} P(D_H) &= P(D_H|A)P(A) + P(D_H|B)P(B) + P(D_H|C)P(C) \\ &= 0.5 \cdot 0.5 + 0.6 \cdot 0.25 + 0.9 \cdot 0.25 = 0.625 \\ P(D_T) &= P(D_T|A)P(A) + P(D_T|B)P(B) + P(D_T|C)P(C) \\ &= 0.5 \cdot 0.5 + 0.4 \cdot 0.25 + 0.1 \cdot 0.25 = 0.375 \end{aligned}$$

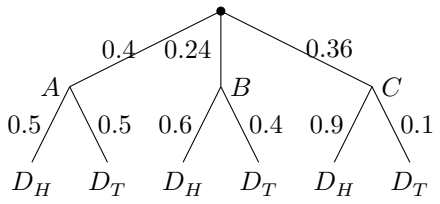
Definition: These probabilities give a (probabilistic) prediction of what will happen if the coin is tossed. Because they are computed before we collect any data they are called prior predictive probabilities.

3.2 Posterior predictive probabilities

Suppose we flip the coin once and it lands heads. We now have data D, which we can use to update the prior probabilities of our hypotheses to posterior probabilities. Last class we learned to use a Bayes table to facilitate this computation:

hypothesis	prior	likelihood	Bayes numerator	posterior
H	$P(H)$	$P(D\|H)$	$P(D\|H)P(H)$	$P(H\|D)$
A	0.5	0.5	0.25	0.4
B	0.25	0.6	0.15	0.24
C	0.25	0.9	0.225	0.36
total	1	NO SUM	0.625	1

Having flipped the coin once and gotten heads, we can compute the probability that our chosen coin will land heads (or tails) if flipped a second time. We proceed just as before, but using the posterior probabilities $P(A|D)$, $P(B|D)$, $P(C|D)$ in place of the prior probabilities $P(A)$, $P(B)$, $P(C)$.



$$\begin{aligned} P(D_H|D) &= P(D_H|A)P(A|D) + P(D_H|B)P(B|D) + P(D_H|C)P(C|D) \\ &= 0.5 \cdot 0.4 + 0.6 \cdot 0.24 + 0.9 \cdot 0.36 = 0.668 \\ P(D_T|D) &= P(D_T|A)P(A|D) + P(D_T|B)P(B|D) + P(D_T|C)P(C|D) \\ &= 0.5 \cdot 0.4 + 0.4 \cdot 0.24 + 0.1 \cdot 0.36 = 0.332 \end{aligned}$$

Definition: These probabilities give a (probabilistic) prediction of what will happen if the coin is tossed again. Because they are computed after collecting data and updating the prior to the posterior, they are called posterior predictive probabilities.

Note that heads on the first toss increases the probability of heads on the second toss.

3.3 Review

Here's a succinct description of the preceding sections that may be helpful:

Each hypothesis gives a different probability of heads, so the total probability of heads is a weighted average. For the prior predictive probability of heads, the weights are given by the prior probabilities of the hypotheses. For the posterior predictive probability of heads, the weights are given by the posterior probabilities of the hypotheses.

Remember: Prior and posterior probabilities are for hypotheses. Prior predictive and posterior predictive probabilities are for outcomes. To keep this straight, remember that the predictive probabilities are used to predict future outcomes, i.e. data.

MIT OpenCourseWare
https://ocw.mit.edu

18.05 Introduction to Probability and Statistics
Spring 2022

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms.

```
!python --version
```

```
Python 3.12.12
```

```
# !pip install arviz pymc pytensor
```

```
import arviz as az
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pymc as pm
from pymc import GARCH11
from pytensor import shared
import pytensor
pytensor.config.mode = 'NUMBA'
```

```
# Download the dataset
m5_data = pd.read_csv("M5. goog_eur_10.csv")

# Convert date variable to date format
m5_data["Date2"] = pd.to_datetime(m5_data["Date"], format="%m/%d/%Y")
goog = m5_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
goog["GOOGLE_R"] = np.log(goog.GOOGLE).diff().dropna()
```

1. Bayesian Statistics

1.1 What is Bayesian Statistics?

Bayesian statistics uses Bayes' theorem as the foundation to apply probabilities to data analysis. From the method we have been learning so far, we assume parameters like mean, variance, and standard deviations are unknown but fixed parameters. This method is called **frequentist statistics**. Maximum likelihood estimation belongs to frequentist statistics. On the other hand, in Bayesian statistics, mean, variance and standard deviations are random variables with probability distributions. This is the main difference between frequentist statistics and Bayesian statistics. To run Bayesian statistics, a researcher has to either use experience or other techniques to assume a parameter's probability distribution before running the analysis. We call these probability distributions **prior probability distributions** or **priors**. We then combine the information from data to derive **posterior probability distributions for parameters** or **posteriors**. The required readings in this lesson will provide definitions and applications of these concepts in more detail. It is good that now the parameters are not unknown fixed values but rather random variables. This change can make analysis more flexible. However, this method sometimes is criticized for subjective parameter probability distribution choice by the researcher before running the data analysis. There is a lot of debate about advantages and disadvantages of frequentist statistics vs. Bayesian statistics. For students who are interested in knowing more detail, there is a lot of material online.

1.2 Basics of Bayesian Updating, Prior Probability Distributions, Posterior Probability Distributions, and Likelihood Function

Before continuing this lesson, please read the required readings for this lesson. The required readings will cover the concept of Bayesian updating, prior probability distributions (priors), posterior probability distributions (posteriors), and likelihood functions in detail. In the following sections of this lesson, we will use these concepts extensively so it is important that you are familiar with these concepts before proceeding to the next section.

1.3 Sampling from Posterior Probability Distribution

We learned in the last section how to derive posterior probability distributions for parameters we are interested in analyzing. We know a posterior distribution is the multiplication of the likelihood function and the prior distribution. Oftentimes, a posterior distribution formula is too complicated to manipulate or is in high dimension. Hence, we have to conduct sampling to obtain a series of sample data points from the posterior distribution to investigate its marginal distribution density, mean, and other parameters.

The most popular sampling method is the **Markov-Chain-Monte-Carlo (MCMC)** method. Monte Carlo is a random sampling method when the samples are independent. Markov-Chains address random samples that are autocorrelated to each other. Hence, MCMC method is a random sampling method for autocorrelated samples. There are several sampling algorithms under the MCMC method. The two most popular ones are the **Metropolis-Hastings algorithm** and the **Gibbs Sampling algorithm**. For those who are interested in the details of these algorithms, there are plenty of materials to explore online. For the following sections, the software we are using to conduct Bayesian estimation on GARCH models uses the Metropolis-Hastings algorithm.

One final note about sampling before we move on to Bayesian estimation of GARCH models: The MCMC method can be sensitive to the initial values used to start sampling. Therefore, sometimes, we would use the result from maximum likelihood estimation as the starting values for Bayesian estimation. We will demonstrate this point when we show how to use Bayesian estimation in a later section.

2. Bayesian Estimation on GARCH Model

In this section, we are going to go over the steps to use Bayesian method to fit a GARCH model. Specifically, we will focus on a GARCH model with Student's t innovations (errors). From the last lesson, we see that one common feature of financial asset returns is their heavy tail distribution. Hence, it is reasonable to use Student's t-distribution to model asset returns.

Ardia and Hoogerheide (2010) demonstrated the steps using Bayesian estimation to fit a GARCH model with Student's t innovations. They wrote an R package (bayesGARCH) to automate the sampling process by using the Metropolis-Hastings algorithm. In the next section, we will use our Google stock return as an example to show the process. Before diving into the example, let's go over the theory for GARCH model with Student's t innovations first.

$$r_t = \epsilon_t \sqrt{\frac{\nu-2}{\nu}\omega_t \sigma_t^2}$$

where

$t = 1, \cdots, T$
$\epsilon_t \sim \text{iid normal}(0,1)$
$\omega_t \sim \text{iid inverse gamma}(\frac{\nu}{2}, \frac{\nu}{2})$
$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta \sigma_{t-1}^2$ where $\alpha_0 > 0,\ \alpha_1, \beta \geq 0$ and $\nu > 2$.

The restrictions above will ensure that the variance will be positive and finite. However, they do not ensure stationary. We know from the last lesson that $\alpha_1 + \beta < 1$ is needed to ensure stationary. We will use the result in our application later.

Another point we would like to explain is the inclusion of ω_t. The reason we have ω_t as a prior multiplied with the variance of r_t in the model is that the marginal distribution of r_t will become a Student's t-distribution with degrees of freedom ν.

The priors for $\alpha_0, \alpha_1, \beta$ are assumed to be truncated normal distributions with the domains greater than 0 since these restrictions ensure positive and finite variance. The prior for ν is shifted exponential distribution.

The joint posterior distribution will look like:

$$p(\alpha_0, \alpha_1, \beta, \nu, \omega_t | r_t) = \frac{l(r_t|\alpha_0, \alpha_1, \beta, \nu, \omega_t)\ p(\alpha_0, \alpha_1, \beta, \nu, \omega_t)}{p(r_t)}$$

Then, MCMC will sample from the joint posterior distribution for each parameter. From the sampled data, we can generate the density for each parameter. As we mentioned in the last section, MCMC sampling can be sensitive to initial values. One method to avoid extreme values from initial values is to drop the first n numbers of sampled observations. This is called a **burn-in** period. We can indicate to the package how many initial sampled observations we would like to drop from our sample series. We will elaborate more on this topic in the next section.

In this section, we will use Google stock return to fit a GARCH model by using the Bayesian estimation method. We will use Python package: PyMC to run this model. The PyMC package has inbuilt GARCH(1,1) distribution that will make the code easier to implement. We should bear in mind that the inbuilt GARCH distribution is with Normal innovations. However, from the Ardia and Hoogerheide (2006) we can learn that the priors for Bayesian GARCH(1,1) with Normal innovations remain the same. Hence, we provide this example for demonstration purposes.

We will use MCMC to generate two data sample series with 3000 samples in each series. In MCMC, a sample series is called a **chain**. Why do we need to sample two data series? We generate two chains because we need to use them to check if the sample series are close enough to the posterior that we want to analyze. This is called **convergence diagnostics** for MCMC sampling. There are several methods to conduct convergence diagnostics. We will use the Gelman-Rubin Diagnostic. The diagnostic provides a "Scale Reduction Factor" to indicate if several series converge. Usually, a value below 1.2 is okay. Since this diagnostic requires at least two chains, we will create two chains from MCMC. Figure 1 shows the MCMC sampling result for the joint posterior distribution for Google stock return GARCH model.

Figure 1: First Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

```
# First Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

# starting parameters = blank canvas model(0.000001, 0.000001, 0.000001)
alpha_mu = shared(np.array([0.000001, 0.000001], dtype=np.float64))
alpha_sigma = shared(np.array([[1000.0, 0.0], [0.0, 1000.0]], dtype=np.float64))

beta_mu = shared(np.array(0.000001, dtype=np.float64))
beta_sigma = shared(np.array(1000.0, dtype=np.float64))
```

```
# initial volatility
ivolatility = shared(np.array(0.000001, dtype=np.float64))
ivolatility_vol = shared(np.array(10.0, dtype=np.float64))

# construct MCMC model
mcmc0 = pm.Model()

with mcmc0:
    # create multivariate normal: p(α) ∝ ϕN2(α|μα,Σα) 1 {α ∈ R^2+}
    mvn = pm.MvNormal("mvNormal", mu=alpha_mu, cov=alpha_sigma, shape=2)

    # restrict the two coefficients to be greater than zero
    alp0 = pm.Deterministic("alpha0", pm.math.switch(mvn[0] > 0, mvn[0], -np.inf))
    alp1 = pm.Deterministic("alpha1", pm.math.switch(mvn[1] > 0, mvn[1], -np.inf))

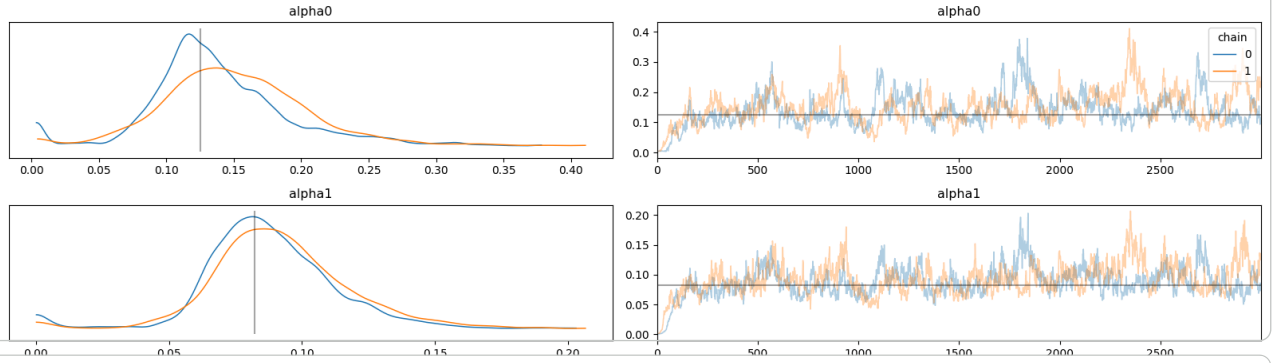
    # create univariate truncated normal: p(β) ∝ ϕN1(β|μβ,Σβ) 1{β ∈ R+}
    nTruncated = pm.TruncatedNormal("beta", mu=beta_mu, sigma=beta_sigma, lower=0)
    volTruncated = pm.TruncatedNormal(
        "volatility", mu=ivolatility, sigma=ivolatility_vol, lower=0
    )

    # likelihood
    likelihood = GARCH11(
        "GARCH",
        omega=alp0,
        alpha_1=alp1,
        beta_1=nTruncated,
        initial_vol=volTruncated,
        observed=goog.GOOGLE_R.dropna() * 100,
    )
```

```
# Plot first round MCMC model posteriors
with mcmc0:
    step_mcmc0 = pm.Slice()
    trace_mcmc0 = pm.sample(
        3000,
        cores=2,
        step=step_mcmc0,
        tune=0,  # tune=0 => no burn-in
        return_inferencedata=True,
        random_seed=12345,
    )
    az.plot_trace(
        trace_mcmc0,
        var_names=["alpha0", "alpha1", "beta"],
        # plot vertical lines at GARCH(1,1) MLE param results
        lines=[
            ("alpha0", {}, [0.124993]),
            ("alpha1", {}, [0.082160]),
            ("beta", {}, [0.867127]),
        ],
        compact=False,
        legend=True,
        figsize=(16, 7),
    )
    plt.tight_layout()
    plt.show()
```

```
Progress                                Draws   Tuning   Steps out   Steps in   Sampling Speed   Elapsed   Remaining
3000    False    0           18         18.89 draws/s    0:02:38   0:00:00
3000    False    0           10         18.82 draws/s    0:02:39   0:00:00

ERROR:pymc.stats.convergence:The effective sample size per chain is smaller than 100 for some parameters.  A higher number i
```




```
# First Round MCMC Convergence Test Result and Parameter Estimations
az.summary(trace_mcmc0, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
alpha0	0.145094	0.053294	0.059351	0.262754	0.007721	0.004627	47.599936	51.832998	1.068832
alpha1	0.089880	0.025170	0.048496	0.141094	0.003876	0.002370	47.796746	48.588985	1.066640
beta	0.852806	0.042358	0.757265	0.916286	0.006640	0.004178	44.814853	48.402712	1.074747

Last column in Figure 2 for *r_hat* represents the Gelman-Rubin statistic. It tests for lack of convergence by comparing the variance between multiple chains to the variance within each chain. If convergence has been achieved, the between-chain and within-chain variances should be identical. If *r_hat* < 1.2 for all model parameters, we can have some confidence that convergence has been reached.

Let's look at the estimations for the parameters. Figure 2 shows the estimated parameters from the first round MCMC sampling result. Let's focus on the means of the coefficients. We see *alpha0*, *alpha1* and *beta* all look reasonable within our restrictions for stationary and positive and finite variance. But so far the model represents so called 'blank canvas' model i.e. we have not accounted for any meaningful starting values. In this initial model the starting values are all near zero for all *alpha0*, *alpha1* and *beta* priors.

We discussed in the previous section that the MCMC can be sensitive to starting value. If starting value is way off, it can impact the final result of MCMC. In this case, we can assign starting values to parameters before the MCMC starts sampling. In that way, we restrain MCMC from going wild when sampling. So where can we find initial values for parameters? In the previous lesson, we estimated the parameters using maximum likelihood. We can use these estimates as starting values for MCMC. Otherwise, this step is a trial-and-error process. You might need to run MCMC a number of times for MCMC to converge to a solution and find reasonable coefficient estimates.

For the next round of MCMC, we adjust starting values for parameters in the code and rerun MCMC. The following figures show the final results of the second run MCMC sampling.

Figure 3: Second Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

```
# Second Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

# starting parameters = MLE(0.124993, 0.082160, 0.867127)
alpha_mu = shared(np.array([0.124993, 0.082160], dtype=np.float64))
alpha_sigma = shared(np.array([[1000.0, 0.0], [0.0, 1000.0]], dtype=np.float64))

beta_mu = shared(np.array(0.867127, dtype=np.float64))
beta_sigma = shared(np.array(1000.0, dtype=np.float64))

# initial volatility
ivolatility = shared(
    np.array(1.63865, dtype=np.float64)
)  # np.std(goog["GOOGLE_R"].dropna()*100)
ivolatility_vol = shared(np.array(10.0, dtype=np.float64))

# construct MCMC model
mcmc = pm.Model()

with mcmc:
    # create a multivariate normal: p(α) ∝ ϕN2(α|μα,Σα) 1 {α ∈ R^2+}
    mvn = pm.MvNormal("mvNormal", mu=alpha_mu, cov=alpha_sigma, shape=2)

    # restrict the two coefficients to be greater than zero
```

```
    alp0 = pm.Deterministic("alpha0", pm.math.switch(mvn[0] > 0, mvn[0], -np.inf))
    alp1 = pm.Deterministic("alpha1", pm.math.switch(mvn[1] > 0, mvn[1], -np.inf))

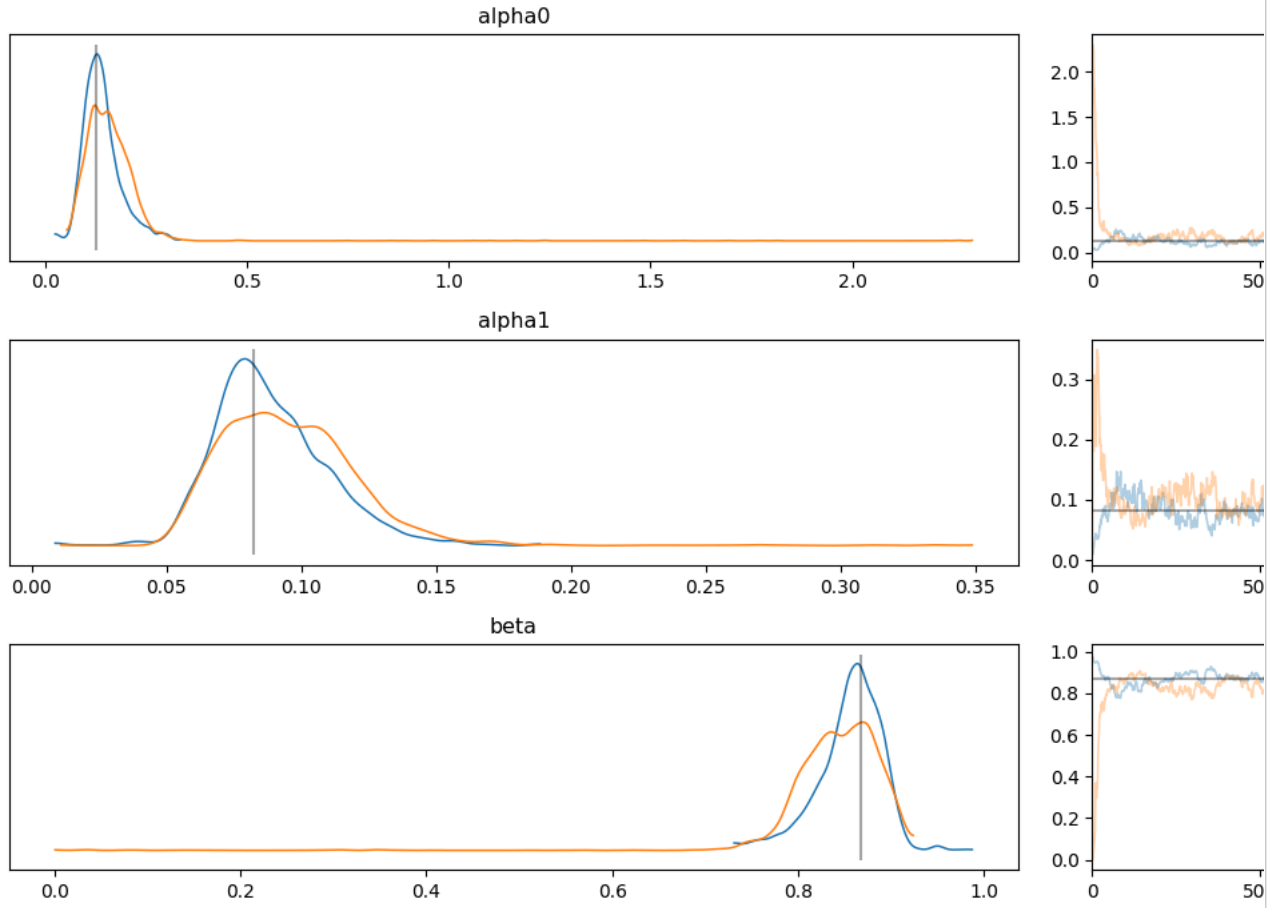
    # create univariate truncated normal: p(β) ∝ ϕN1(β|μβ,Σβ) 1{β ∈ R+}
    nTruncated = pm.TruncatedNormal("beta", mu=beta_mu, sigma=beta_sigma, lower=0)
    volTruncated = pm.TruncatedNormal(
        "volatility", mu=ivolatility, sigma=ivolatility_vol, lower=0
    )

    # likelihood
    likelihood = GARCH11(
        "garch",
        omega=alp0,
        alpha_1=alp1,
        beta_1=nTruncated,
        initial_vol=volTruncated,
        observed=goog.GOOGLE_R.dropna() * 100,
    )
```

```
# Plot Second round MCMC model posteriors
with mcmc:
    step_mmc = pm.Slice()
    trace_mmc = pm.sample(
        3000,
        cores=2,
        step=step_mmc,
        tune=0,  # no burn-in
        return_inferencedata=True,
        random_seed=12345,
    )
    az.plot_trace(
        trace_mmc,
        var_names=["alpha0", "alpha1", "beta"],
        lines=[
            ("alpha0", {}, [0.124993]),
            ("alpha1", {}, [0.082160]),
            ("beta", {}, [0.867127]),
        ],
        compact=False,
        legend=True,
        figsize=(16, 7),
    )
    plt.tight_layout()
    plt.show()
```

Progress	Draws	Tuning	Steps out	Steps in	Sampling Speed	Elapsed	Remaining
	3000	False	0	11	18.86 draws/s	0:02:38	0:00:00
	3000	False	0	14	18.83 draws/s	0:02:39	0:00:00

```
ERROR:pymc.stats.convergence:The effective sample size per chain is smaller than 100 for some parameters.  A higher number i
```




```
Start coding or generate with AI.
```

In figure 3, we can see that after adding starting values to the code, the trace plot for all priors converges better in this round of MCMC. Let's check the Gelman-Rubin convergence test and other parameter estimations.

```
# Second Round MCMC Sampling Result: Parameter estimation
az.summary(trace_mmc, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
alpha0	0.151440	0.091908	0.067844	0.240691	0.007156	0.043919	82.938764	221.227579	1.029334
alpha1	0.092149	0.024706	0.054129	0.132585	0.002455	0.002861	92.183544	318.266329	1.024602
beta	0.848298	0.051242	0.782080	0.913977	0.004892	0.015678	75.874103	231.988360	1.030617

We can see from figure 4 that all the parameters have converged pretty well after adding starting values to the code and *r_hat* improved.

However, we can still see from figure 3 that the first 250 data points seem to be very volatile. We can treat them as the burn-in period and drop those sampled data points. At this point, we will also combine chains. Let's check out the MCMC result after these steps.

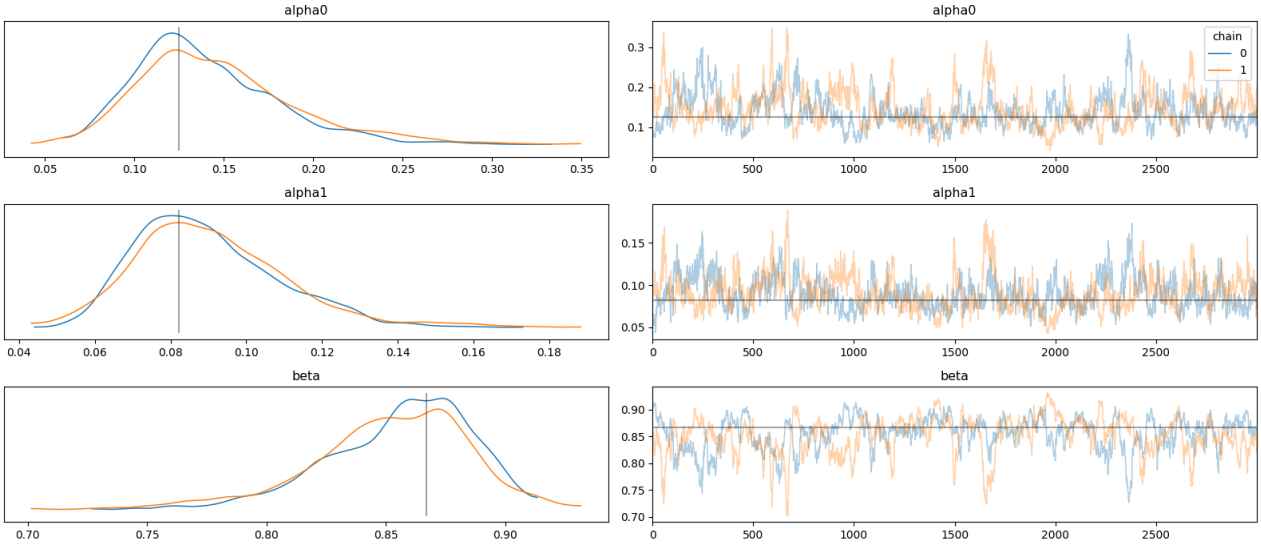
Figure 5: Second Round MCMC Sampling Result: Parameter Trace Plots and Density Plots After Adjustment

```
# Second Round MCMC Sampling Result: Parameter Trace Plots and Density Plots After Adjustment
with mcmc:
    step_mmc_a = pm.Slice()
    trace_mmc_a = pm.sample(
        3000,
        cores=2,
        step=step_mmc_a,
        tune=250,  # burn-in = 250 steps
        return_inferencedata=True,
```

```
    random_seed=12345,
)
az.plot_trace(
    trace_mmc_a,
    var_names=["alpha0", "alpha1", "beta"],
    lines=[
        ("alpha0", {}, [0.124993]),
        ("alpha1", {}, [0.082160]),
        ("beta", {}, [0.867127]),
    ],
    compact=False,
    legend=True,
    figsize=(16, 7),
)
plt.tight_layout()
plt.show()
```

Progress	Draws	Tuning	Steps out	Steps in	Sampling Speed	Elapsed	Remaining
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3250	False	1	1	26.62 draws/s	0:02:02	0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3250	False	0	7	25.76 draws/s	0:02:06	0:00:00

```
ERROR:pymc.stats.convergence:The effective sample size per chain is smaller than 100 for some parameters.  A higher number i
```



From figure 5, we can see the trace plots for 3 parameters all oscillate within a narrower range compared to the trace plots in figure 1. On the left, we can see that the marginal densities of *alpha0* and *alpha1* seem to skew to the right while the marginal density for beta skew to the left. Let's check the final parameter estimates.

```
Start coding or generate with AI.
```

Figure 6: First Round MCMC Sampling Result: Parameter Estimation with Adjustment

```
# First Round MCMC Sampling Result: Parameter Estimation with Adjustment
az.summary(trace_mmc_a, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
alpha0	0.143620	0.043415	0.074571	0.231585	0.004067	0.002347	108.231247	235.146465	1.008775
alpha1	0.090170	0.019715	0.057861	0.128158	0.001783	0.001058	119.996201	301.232062	1.003836
beta	0.852987	0.032861	0.786890	0.907703	0.003265	0.001904	97.762053	211.792705	1.007301

```
Start coding or generate with AI.
```

In figure 6, we can see that all estimated coefficients are well behaved and confined in the restrictions to meet stationarity and positive and finite variance requirements. In figure 7 below, we can compare the estimated coefficients from MLE and Bayesian estimations. We can see the estimated coefficients are similar. We achieved this comparable result partly because we used MLE estimates as our starting values for MCMC sampling. By carefully selecting starting values for MCMC, we can control the behavior of the estimated coefficients.

Bayesian Updating with Continuous Priors
Class 13, 18.05
Jeremy Orloff and Jonathan Bloom

1 Learning Goals

1. Understand a parameterized family of distributions as representing a continuous range of hypotheses for the observed data.
2. Be able to state Bayes' theorem and the law of total probability for continous densities.
3. Be able to apply Bayes' theorem to update a prior probability density function to a posterior pdf given data and a likelihood function.
4. Be able to interpret and compute posterior predictive probabilities.

2 Introduction

Up to now we have only done Bayesian updating when we had a finite number of hypothesis, e.g. our dice example had five hypotheses (4, 6, 8, 12 or 20 sides). Now we will study Bayesian updating when there is a continuous range of hypotheses. The Bayesian update process will be essentially the same as in the discrete case. As usual when moving from discrete to continuous we will need to replace the probability mass function by a probability density function, and sums by integrals.

The first few sections of this note are devoted to working with pdfs. In particular we will cover the law of total probability and Bayes' theorem. We encourage you to focus on how these are essentially identical to the discrete versions. After that, we will apply Bayes' theorem and the law of total probability to Bayesian updating.

3 Examples with continuous ranges of hypotheses

Here are three standard examples with continuous ranges of hypotheses.

Example 1. Suppose you have a system that can succeed or fail with probability p. Then we can hypothesize that p is anywhere in the range $[0, 1]$. That is, we have a continuous range of hypotheses. We will often model this example with a 'bent' coin with unknown probability p of heads.

Example 2. The lifetime of a certain isotope is modeled by an exponential distribution $\exp(\lambda)$. In principle, the mean lifetime $1/\lambda$ can be any real number in $(0, \infty)$.

Example 3. We are not restricted to a single parameter. In principle, the parameters μ and σ of a normal distribution can be any real numbers in $(-\infty, \infty)$ and $(0, \infty)$, respectively. If we model gestational length for single births by a normal distribution, then from millions of data points we know that μ is about 40 weeks and σ is about one week.

In all of these examples we modeled the random process giving rise to the data by a distribution with parameters –called a parametrized distribution. Every possible choice of the parameter(s) is a hypothesis, e.g. we can hypothesize that the probability of succcess in Example 1 is $p = 0.7313$. We have a continuous set of hypotheses because we could take any value between 0 and 1.

4 Notational conventions

4.1 Parametrized models

As in the examples above our hypotheses often take the form a certain parameter has value θ. We will often use the letter θ to stand for an arbitrary hypothesis. This will leave symbols like p, f, and x to take there usual meanings as pmf, pdf, and data. Also, rather than saying 'the hypothesis that the parameter of interest has value θ' we will simply say the hypothesis θ.

4.2 Big and little letters

We have two parallel notations for outcomes and probability:

1. (Big letters) Event A, probability function $P(A)$.
2. (Little letters) Value x, pmf $p(x)$ or pdf $f(x)$.

These notations are related by $P(X = x) = p(x)$, where x is a value the discrete random variable X and '$X = x$' is the corresponding event.

We carry these notations over to the probabilities used in Bayesian updating.

1. (Big letters) From hypotheses $\mathcal{H}$ and data $\mathcal{D}$ we compute several associated probabilities

$$P(\mathcal{H}),\ P(\mathcal{D}),\ P(\mathcal{H}|\mathcal{D}),\ P(\mathcal{D}|\mathcal{H}).$$

In the coin example we might have $\mathcal{H}_{0.6}$ = 'the chosen coin has probability 0.6 of heads', $\mathcal{D}$ = '3 flips landed HHT', so $P(\mathcal{D}|\mathcal{H}_{0.6}) = (0.6)^2(0.4)$

2. (Small letters) Hypothesis values θ and data values x both have probabilities or probability densities:

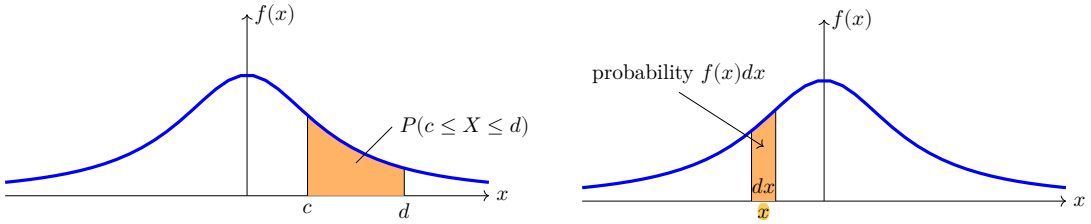
$$\begin{array}{cccc} p(\theta) & p(x) & p(\theta|x) & p(x|\theta) \\ f(\theta) & f(x) & f(\theta|x) & f(x|\theta) \end{array}$$

In the coin example we might have $\theta = 0.6$ and x is the sequence $1, 1, 0$. So, $p(x|\theta) = (0.6)^2(0.4)$. We might also write $p(x = 1, 1, 0|\theta = 0.6)$ to emphasize the values of x and θ.

Although we will still use both types of notation, from now on we will mostly use the small letter notation involving pmfs and pdfs. Hypotheses will usually be parameters represented by Greek letters $(\theta, \lambda, \mu, \sigma, \dots)$ while data values will usually be represented by English letters $(x, x_i, y, \dots)$.

5 Quick review of pdf and probability

Suppose X is a random variable with pdf $f(x)$. Recall $f(x)$ is a density; its units are probability/(units of x).



The probability that the value of X is in $[c, d]$ is given by

$$\int_c^d f(x)\,dx.$$

The probability that X is in an infinitesimal range dx around x is $f(x)\,dx$. In fact, the integral formula is just the 'sum' of these infinitesimal probabilities. We can visualize these probabilities by viewing the integral as area under the graph of $f(x)$.

In order to manipulate probabilities instead of densities in what follows, we will make frequent use of the notion that $f(x)\,dx$ is the probability that X is in an infinitesimal range around x of width dx. Please make sure that you fully understand this notion.

6 Continuous priors, discrete likelihoods

In the Bayesian framework we have probabilities of hypotheses –called prior and posterior probabilities– and probabilities of data given a hypothesis –called likelihoods. In earlier classes both the hypotheses and the data had discrete ranges of values. We saw in the introduction that we might have a continuous range of hypotheses. The same is true for the data, but for today we will assume that our data can only take a discrete set of values. In this case, the likelihood of data x given hypothesis θ is written using a pmf: $p(x|\theta)$.

We will use the following coin example to explain these notions. We will carry this example through in each of the succeeding sections.

Example 4. Suppose we have a bent coin with unknown probability θ of heads. In this case, we'll say the coin is of 'type θ' and we'll label the hypothesis that a random coin is of type θ by $\mathcal{H}_\theta$. The value of θ is random and could be anywhere between 0 and 1. For this and the examples that follow we'll suppose that the value of θ follows a distribution with continuous prior probability density $f(\theta) = 2\theta$. We have a discrete likelihood because tossing a coin has only two outcomes, $x = 1$ for heads and $x = 0$ for tails.

$$p(x = 1|\mathcal{H}_\theta) = \theta, \qquad p(x = 0|\mathcal{H}_\theta) = 1 - \theta.$$

As we stated earlier, we will often write θ for the hypothesis $\mathcal{H}_\theta$. So the above probabilities become

$$p(x = 1|\theta) = \theta, \qquad p(x = 0|\theta) = 1 - \theta.$$

Think: This can be tricky to wrap your mind around. We have a continuous range of types of coins –we identify the type by the value of the parameter θ. We are able to choose a coin at random and the type chosen has a probability density $f(\theta)$.

It may help to see that the discrete examples we did in previous classes are similar. In one example, we had three types of coin with probability of heads 0.5, 0.6, or 0.9. So, we called our hypotheses $H_{0.5}$, $H_{0.6}$, $H_{0.9}$ and these had prior probabilities $P(H_{0.5})$ etc. In other words, we had a type of coin with an unknown probability of heads, we had hypotheses about that probability and each of these hypotheses had a prior probability.

7 The law of total probability

The law of total probability for continuous probability distributions is essentially the same as for discrete distributions. We replace the prior pmf by a prior pdf and the sum by an integral. We start by reviewing the law for the discrete case.

Recall that for a discrete set of hypotheses $\mathcal{H}_1, \mathcal{H}_2, \ldots \mathcal{H}_n$ the law of total probability says

$$P(\mathcal{D}) = \sum_{i=1}^{n} P(\mathcal{D}|\mathcal{H}_i)P(\mathcal{H}_i). \tag{1}$$

This is the total prior probability of $\mathcal{D}$ because we used the prior probabilities $P(\mathcal{H}_i)$

In the little letter notation with $\theta_1, \theta_2, \ldots, \theta_n$ for hypotheses and x for data the law of total probability is written

$$p(x) = \sum_{i=1}^{n} p(x|\theta_i)p(\theta_i). \tag{2}$$

We also called this the prior predictive probability of the outcome x to distinguish it from the prior probability of the hypothesis θ.

Likewise, there is a law of total probability for continuous pdfs. We state it as a theorem using little letter notation.

Theorem. Law of total probability. Suppose we have a continuous parameter θ in the range $[a, b]$, and discrete random data x. Assume θ is itself random with density $f(\theta)$ and that x and θ have likelihood $p(x|\theta)$. In this case, the total probability of x is given by the formula.

$$p(x) = \int_a^b p(x|\theta)f(\theta)\,d\theta \tag{3}$$

Proof. Our proof will be by analogy to the discrete version: The probability term $p(x|\theta)f(\theta)\,d\theta$ is perfectly analogous to the term $p(x|\theta_i)p(\theta_i)$ in Equation 2 (or the term $P(\mathcal{D}|\mathcal{H}_i)P(\mathcal{H}_i)$ in Equation 1). Continuing the analogy: the sum in Equation 2 becomes the integral in Equation 3

As in the discrete case, when we think of θ as a hypothesis explaining the probability of the data we call $p(x)$ the prior predictive probability for x.

Example 5. (Law of total probability.) Continuing with Example 4. We have a bent coin with probability θ of heads. The value of θ is random with prior pdf $f(\theta) = 2\theta$ on $[0, 1]$.

Suppose I am about to flip the coin. What is the total probability of heads, i.e what is the prior predictive probability of heads?

Solution: In Example 4 we noted that the likelihoods are $p(x = 1|\theta) = \theta$ and $p(x = 0|\theta) = 1 - \theta$. So the total probability of $x = 1$ is

$$p(x = 1) = \int_0^1 p(x = 1|\theta)\, f(\theta)\, d\theta = \int_0^1 \theta \cdot 2\theta\, d\theta = \int_0^1 2\theta^2 d\theta = \frac{2}{3}.$$

Since the prior is weighted towards higher probabilities of heads, so is the total probability of heads.

8 Bayes' theorem for continuous probability densities

The statement of Bayes' theorem for continuous pdfs is essentially identical to the statement for pmfs. We state it including $d\theta$ so we have genuine probabilities:

Theorem. Bayes' Theorem. Use the same assumptions as in the law of total probability, i.e. θ is a continuous parameter with pdf $f(\theta)$ and range $[a, b]$; x is random discrete data; together they have likelihood $p(x|\theta)$. With these assumptions:

$$f(\theta|x)\, d\theta = \frac{p(x|\theta)f(\theta)\, d\theta}{p(x)} = \frac{p(x|\theta)f(\theta)\, d\theta}{\int_a^b p(x|\theta)f(\theta)\, d\theta}. \tag{4}$$

Proof. Since this is a statement about probabilities it is just the usual statement of Bayes' theorem. We hope this is clear.

It is important enough to spell out somewhat formally: Let Θ be the random variable that produces the value θ. Consider the events

$$\mathcal{H} = \text{'}\Theta \text{ is in an interval of width } d\theta \text{ around the value } \theta\text{'}$$

and

$$\mathcal{D} = \text{'the value of the data is } x\text{'}.$$

Then $P(\mathcal{H}) = f(\theta)\, d\theta$, $P(\mathcal{D}) = p(x)$, and $P(\mathcal{D}|\mathcal{H}) = p(x|\theta)$. Now our usual form of Bayes' theorem becomes

$$f(\theta|x)\, d\theta = P(\mathcal{H}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathcal{H})P(\mathcal{H})}{P(\mathcal{D})} = \frac{p(x|\theta)f(\theta)\, d\theta}{p(x)}$$

Looking at the first and last terms in this equation we see the new form of Bayes' theorem.

Finally, we firmly believe that it is more conducive to careful thinking about probability to keep the factor of $d\theta$ in the statement of Bayes' theorem. But because it appears in the numerator on both sides of Equation 4 many people drop the $d\theta$ and write Bayes' theorem in terms of densities as

$$f(\theta|x) = \frac{p(x|\theta)f(\theta)}{p(x)} = \frac{p(x|\theta)f(\theta)}{\int_a^b p(x|\theta)f(\theta)\, d\theta}.$$

9 Bayesian updating with continuous priors

Now that we have Bayes' theorem and the law of total probability we can finally get to Bayesian updating. Before continuing with Example 4, we point out two features of the Bayesian updating table that appears in the next example:

1. The table for continuous priors is very simple: since we cannot have a row for each of an infinite number of hypotheses we'll have just one row which uses a variable to stand for all hypotheses $\mathcal{H}_\theta$.

2. By including $d\theta$, all the entries in the table are probabilities and all our usual probability rules apply.

Example 6. (Bayesian updating.) Continuing Examples 4 and 5. We have a bent coin with unknown probability θ of heads. The value of θ is random with prior pdf $f(\theta) = 2\theta$. Suppose we flip the coin three times and get the sequence HTT. Compute the posterior pdf for θ.

Solution: We make the usual update table, with an added column giving the range of values that θ can take. We make the first row an abstract version of Bayesian updating and the second row is Bayesian updating for this particular example. In later examples we will skip that abstract version.

hypothesis	range	prior	likelihood	Bayes numerator	posterior
$\mathcal{H}_\theta$	θ range	$f(\theta)\,d\theta$	$p(x=1,1,0\|\theta)$	$p(x=1,1,0\|\theta)f(\theta)\,d\theta$	$f(\theta\|x=1,1,0)\,d\theta$
$\mathcal{H}_\theta$	$[0,1]$	$2\theta\,d\theta$	$\theta^2(1-\theta)$	$2\theta^3(1-\theta)\,d\theta$	$20\theta^3(1-\theta)\,d\theta$
total	$[0,1]$	$\int_a^b f(\theta)\,d\theta = 1$	no sum	$p(x=1,1,0)$ $= \int_0^1 2\theta^3(1-\theta)\,d\theta = 1/10$	1

Therefore the posterior pdf (after seeing HHT) is $\boxed{f(\theta|x) = 20\theta^3(1-\theta)}$.

We have a number of comments:

1. Since we used the prior probability $f(\theta)\,d\theta$, the hypothesis should have been:
'the unknown paremeter is in an interval of width $d\theta$ around θ'.
Even for us that is too much to write, so you will have to think it everytime we write that the hypothesis is θ or $\mathcal{H}_\theta$.

2. The posterior pdf for θ is found by removing the $d\theta$ from the posterior probability in the table.

$$f(\theta|x) = 20\theta^3(1-\theta).$$

3. (i) As always $p(x)$ is the total probability. Since we have a continuous distribution instead of a sum we compute an integral.

(ii) Notice that by including $d\theta$ in the table, it is clear what integral we need to compute to find the total probability $p(x)$.

4. The table organizes the continuous version of Bayes' theorem. Namely, the posterior pdf

is related to the prior pdf and likelihood function via:

$$f(\theta|x)\,d\theta = \frac{p(x|\theta)\,f(\theta)d\theta}{\int_a^b p(x|\theta)f(\theta)\,d\theta} = \frac{p(x|\theta)\,f(\theta)}{p(x)}\,d\theta$$

Removing the $d\theta$ in the numerator of both sides we have the statement in terms of densities.

5. Regarding both sides as functions of θ, we can again express Bayes' theorem in the form:

$$f(\theta|x) \propto p(x|\theta) \cdot f(\theta)$$

$$\text{posterior} \propto \text{likelihood} \times \text{prior}.$$

9.1 Flat priors

One important prior is called a flat or uniform prior. A flat prior assumes that every hypothesis is equally probable. For example, if θ has range $[0, 1]$ then $f(\theta) = 1$ is a flat prior.

Example 7. (Flat priors.) We have a bent coin with unknown probability θ of heads. Suppose we toss it once and get heads. Assume a flat prior and find the posterior probability for θ.

Solution: This is similar Example 6 with a different prior and data.

hypothesis θ	range θ	prior $f(\theta)\,d\theta$	likelihood $p(x=1\|\theta)$	Bayes numerator	posterior $f(\theta\|x=1)\,d\theta$
θ	$[0,1]$	$1 \cdot d\theta$	θ	$\theta\,d\theta$	$2\theta\,d\theta$
total	$[0,1]$	$\int_a^b f(\theta)\,d\theta = 1$	no sum	$p(x=1) = \int_0^1 \theta\,d\theta = 1/2$	1

9.2 Using the posterior pdf

Example 8. In the previous example the prior probability was flat. First show that this means that a priori the coin is equally like to be biased towards heads or tails. Then, after observing one heads, what is the (posterior) probability that the coin is biased towards heads?

Solution: Since the parameter θ is the probability the coin lands heads, the first part of the problem asks us to show $P(\theta > 0.5) = 0.5$ and the second part asks for $P(\theta > 0.5 \,|\, x = 1)$. These are easily computed from the prior and posterior pdfs respectively.

The prior probability that the coin is biased towards heads is

$$P(\theta > 0.5) = \int_{0.5}^1 f(\theta)\,d\theta = \int_{0.5}^1 1 \cdot d\theta = \theta|_{0.5}^1 = \frac{1}{2}.$$

The probability of $1/2$ means the coin is equally likely to be biased toward heads or tails. The posterior probabilitiy that it's biased towards heads is

$$P(\theta > 0.5|x = 1) = \int_{0.5}^1 f(\theta|x=1)\,d\theta = \int_{0.5}^1 2\theta\,d\theta = \theta^2|_{0.5}^1 = \frac{3}{4}.$$

We see that observing one heads has increased the probability that the coin is biased towards heads from 1/2 to 3/4.

10 Predictive probabilities

Just as in the discrete case we are also interested in using the posterior probabilities of the hypotheses to make predictions for what will happen next.

Example 9. (Prior and posterior prediction.) Continuing Examples 4, 5, 6: we have a coin with unknown probability θ of heads and the value of θ has prior pdf $f(\theta) = 2\theta$. Find the prior predictive probability of heads. Then suppose the first flip was heads and find the posterior predictive probabilities of both heads and tails on the second flip.

Solution: For notation let x_1 be the result of the first flip and let x_2 be the result of the second flip. The prior predictive probability is exactly the total probability computed in Examples 5 and 6.

$$p(x_1 = 1) = \int_0^1 p(x_1 = 1|\theta) f(\theta)\, d\theta = \int_0^1 2\theta^2\, d\theta = \frac{2}{3}.$$

The posterior predictive probabilities are the total probabilities computed using the posterior pdf. From Example 6 we know the posterior pdf is $f(\theta|x_1 = 1) = 3\theta^2$. So the posterior predictive probabilities are

$$p(x_2 = 1|x_1 = 1) = \int_0^1 p(x_2 = 1|\theta, x_1 = 1) f(\theta|x_1 = 1)\, d\theta = \int_0^1 \theta \cdot 3\theta^2\, d\theta = 3/4$$

$$p(x_2 = 0|x_1 = 1) = \int_0^1 p(x_2 = 0|\theta, x_1 = 1) f(\theta|x_1 = 1)\, d\theta = \int_0^1 (1-\theta) \cdot 3\theta^2\, d\theta = 1/4$$

(More simply, we could have computed $p(x_2 = 0|x_1 = 1) = 1 - p(x_2 = 1|x_1 = 1) = 1/4$.)

11 (Optional) From discrete to continuous Bayesian updating

This section is optional. In it we will try to develop intuition for the transition from discrete to continuous Bayesian updating. We'll walk a familiar road from calculus. Namely we will:

(i) divide the continuous range of hypotheses into a finite number of short intervals.

(ii) create the discrete updating table for the finite number of hypotheses.

(iii) consider how the table changes as the number of hypotheses goes to infinity.

In this way, we'll see the prior and posterior pmfs converge to the prior and posterior pdfs.

Example 10. To keep things concrete, we will work with the same prior and data as in Example 7. We have a 'bent' coin with a flat prior $f(\theta) = 1$. Our data is we tossed the coin once and got heads.

Our goal is to go from discrete to continuous by increasing the number of hypotheses.

4 hypotheses. Suppose we have four types of coins that have probability of heads 1/8, 3/8, 5/8 and 7/8 respectively. If one coin is chosen at random, our hypotheses for its type are

$$\mathcal{H}_1 : \theta = 1/8, \quad \mathcal{H}_2 : \theta = 3/8, \quad \mathcal{H}_3 : \theta = 5/8, \quad \mathcal{H}_4 : \theta = 7/8.$$

To get this, we divided $[0,1]$ into 4 equal intervals: $[0,1/4]$, $[1/4,1/2]$, $[1/2,3/4]$, $[3/4,1]$. Each interval has width $\Delta\theta = 1/4$. We put our the value of θ for our coin types at the centers of the four intervals.

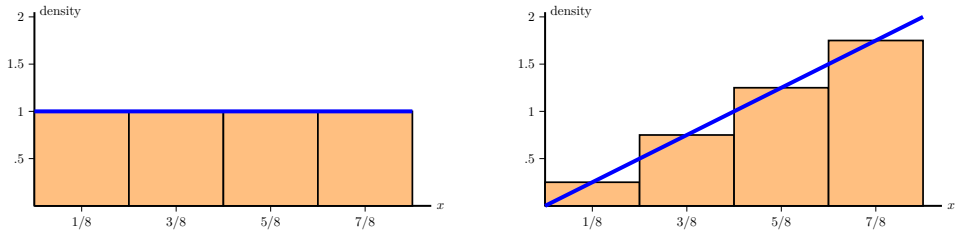
(Just as with forming Riemann sums in calculus, it's not important where in each interval we choose θ. The center is one easy choice.)

Let's name each of these values $\theta_j = j/8$, where $j = 1, 3, 5, 7$.

The flat prior gives each hypothesis a probability of $1/4 = 1 \cdot \Delta\theta$. We have the table:

hypothesis	prior	likelihood	Bayes num.	posterior
$\theta = \theta_1 = \frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{4} \cdot \frac{1}{8}$	0.0625
$\theta = \theta_2 = \frac{3}{8}$	$\frac{1}{4}$	$\frac{3}{8}$	$\frac{1}{4} \cdot \frac{3}{8}$	0.1875
$\theta = \theta_3 = \frac{5}{8}$	$\frac{1}{4}$	$\frac{5}{8}$	$\frac{1}{4} \cdot \frac{5}{8}$	0.3125
$\theta = \theta_4 = \frac{7}{8}$	$\frac{1}{4}$	$\frac{7}{8}$	$\frac{1}{4} \cdot \frac{7}{8}$	0.4375
Total	1	–	$\sum_{i=1}^{n} \theta_i \, \Delta\theta$	1

Here are the density histograms of the prior and posterior pmf. The prior and posterior pdfs from Example 7 are superimposed on the histograms in red.

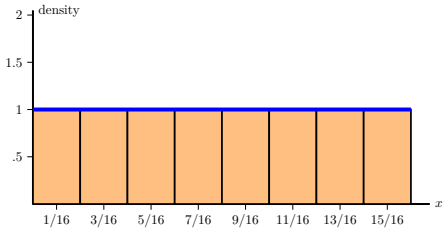


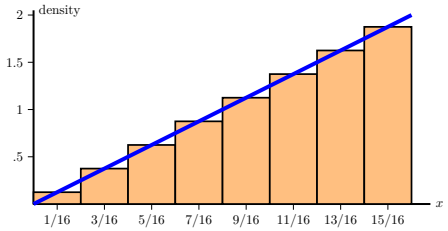
8 hypotheses. Next we slice [0,1] into 8 intervals each of width $\Delta\theta = 1/8$ and use the center of each slice for our 8 hypotheses θ_i.

θ_1: '$\theta = 1/16$', θ_2: '$\theta = 3/16$', θ_3: '$\theta = 5/16$', θ_4: '$\theta = 7/16$'
θ_5: '$\theta = 9/16$', θ_6: '$\theta = 11/16$', θ_7: '$\theta = 13/16$', θ_8: '$\theta = 15/16$'

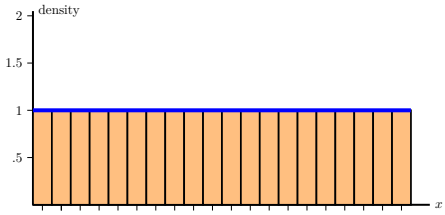
The flat prior gives each hypothesis the probablility $1/8 = 1 \cdot \Delta\theta$. Here are the table and density histograms.

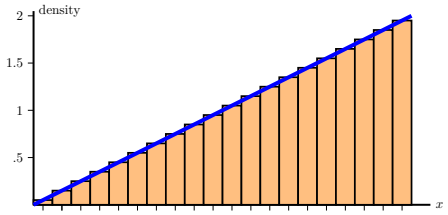
hypothesis	prior	likelihood	Bayes num.	posterior
$\theta = \theta_1 = \frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{8} \cdot \frac{1}{16}$	0.0156
$\theta = \theta_2 = \frac{3}{16}$	$\frac{1}{8}$	$\frac{3}{16}$	$\frac{1}{8} \cdot \frac{3}{16}$	0.0469
$\theta = \theta_3 = \frac{5}{16}$	$\frac{1}{8}$	$\frac{5}{16}$	$\frac{1}{8} \cdot \frac{5}{16}$	0.0781
$\theta = \theta_4 = \frac{7}{16}$	$\frac{1}{8}$	$\frac{7}{16}$	$\frac{1}{8} \cdot \frac{7}{16}$	0.1094
$\theta = \theta_5 = \frac{9}{16}$	$\frac{1}{8}$	$\frac{9}{16}$	$\frac{1}{8} \cdot \frac{9}{16}$	0.1406
$\theta = \theta_6 = \frac{11}{16}$	$\frac{1}{8}$	$\frac{11}{16}$	$\frac{1}{8} \cdot \frac{11}{16}$	0.1719
$\theta = \theta_7 = \frac{13}{16}$	$\frac{1}{8}$	$\frac{13}{16}$	$\frac{1}{8} \cdot \frac{13}{16}$	0.2031
$\theta = \theta_8 = \frac{15}{16}$	$\frac{1}{8}$	$\frac{15}{16}$	$\frac{1}{8} \cdot \frac{15}{16}$	0.2344
Total	1	–	$\sum_{i=1}^{n} \theta_i \, \Delta\theta$	1





20 hypotheses. Finally we slice [0,1] into 20 pieces. This is essentially identical to the previous two cases. Let's skip right to the density histograms.





Looking at the sequence of plots we see how the prior and posterior density histograms converge to the prior and posterior probability density functions.

MIT OpenCourseWare
https://ocw.mit.edu

18.05 Introduction to Probability and Statistics
Spring 2022

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms.

FINANCIAL ECONOMETRICS

MODULE 5 | LESSON 4

GARCH MODEL UNDER STATE SPACE MODEL CONSTRUCT

Reading Time	60 minutes
Prior Knowledge	Time Series analysis, GARCH
Keywords	State Space Model, Markov Process, Kalman Filter

In this lesson, we will use a state space approach to create a GARCH model. The state space model (SSM) was originally developed as an analytical method for aerospace-related research. However, it has gained a lot of interest in engineering and finance recently. We will give a brief description of the structure of state space model and its properties. We will then introduce a popular linear state space model: Kalman Filter. We will go through the basics of the Kalman Filter. Lastly, we will combine Kalman Filter with GARCH(1,1) model to give you insight into how the combined model works.

1. State Space Models

1.1. The Basics of a State Space Model

State space model (or dynamic linear model, DLM) is another popular time series model class. State space model (SSM) states that there are two types of time series variables: one that we can observe and one that we cannot observe. The ones that we cannot observe are called **states**. Those that we can observe are called **observations** instead. The states are generated by a system that describes how states evolve over time. We would like to find out the structure of this system. However, we only see observations. Observations are dependent on states with noise added. We use an observation equation to describe the relationship between observations and states. We also add a measurement noise to the observation equation to fill the gap between the observations and states. We can formalize the above concept in the following equations. Assume y_t is the time series observation variable and x_t is the time series state variable:

Observation Equation:

$$y_t = A_t x_t + v_t, \quad v_t \sim WN(0, V_t) \tag{1}$$

where v_t is the measurement noise.

State Equation:

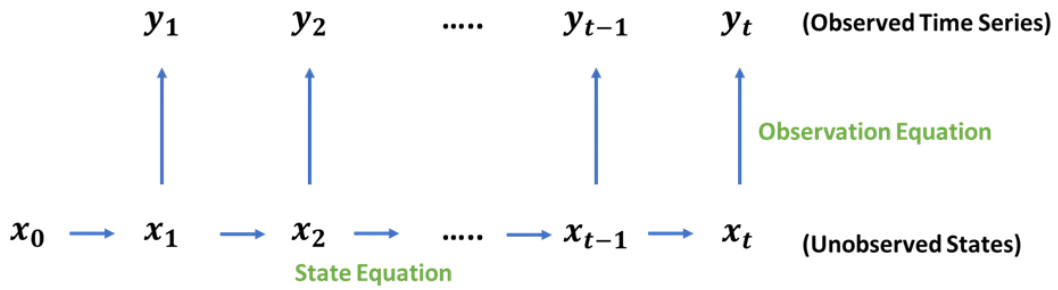
$$x_t = \Phi_t x_{t-1} + w_t, \quad w_t \sim WN(0, W_t) \tag{2}$$

where w_t is the system noise.

We also assume that v_t and v_s are independent when $t \neq s$, w_t and w_s are independent when $t \neq s$, and v_t and w_s are uncorrelated on all lags.

The above two equations form the basis of a state space model. We can also use the following graph to demonstrate the structure of a state space model.

Figure 1: Relationship between States and Observations in a State Space Model



From figure 1, we can see the evolution of the state variable is represented by a state equation. This is the system or the model created by theory or process. State equation(s) can be a single equation or a set of equations to describe how states are generated. Then, we have the observation equation to connect states from a state equation to observations with a measurement noise. As for the number of state variables and observation variables, there can be more than one state variable or observation variable in a state space model. The number of state variables also do not have to be the same as the number of observation variables. As for state space model design, it can be linear or non-linear. In equation (1) and equation (2), we can see that A_t and Φ_t both have time subscript. It means they can change by time. If this is the case, it is a time-varying state space model. If A_t and Φ_t are fixed regardless of time (no time subscript), it is a time-invariant state space model.

We can see state space model is a very general and flexible time series modeling format. You can construct a time series model for state equation and then add an observation equation to connect states with observations or data.

Once we build a state space model, we will use the model to achieve the following three goals:

> **a. Predicting:** forecast future values of states given all the information and data we have today
> **b. Filtering:** estimate current values of states given all the information and data we have today
> **c. Smoothing:** estimate pass values of states given all the information and data we have today

1.2. Properties of State Space Model

Now let's look at two key properties from the state space model.

a. x_t is a Markov Process

It means $P(x_t|x_{t-1}, x_{t-2}, \cdots, x_0) = P(x_t|x_{t-1})$. Markov process states that the probability of the value of a state at current time only depends on the value of the most recent state. The values of the state from older times do not have an impact on today's value. We can say that x_t is memoryless.

b. y_t is fully specified given x_t

This statement can be shown as follows:
$P(y_t|x_t, x_{t-1}, \cdots, x_0;\ y_{t-1}, y_{t-2}, \cdots, y_1) = P(y_t|x_t)$.

With the above two properties, we can write the joint distribution function for states and observations as follows:

$$P(x_{0:t}, y_{1:t}) = P(x_0)\prod_{i=1}^{t} P(x_i|x_{i-1})P(y_i|x_i)$$

For x_0, we usually have to assume its value or its distribution in order to proceed with the model building.

1.3 Benefits of State Space Model (SSM)

There are several reasons why state space modeling is gaining popularity among researchers. Firstly, according to the properties we explained above, we only need the most recent information to estimate the value of the current state. This property will reduce the amount of historical data needed and data storage space required to estimate SSM.

Secondly, as we saw above, SSM allows for time varying coefficients. It is very helpful if the modeling period happens to have regime change. Time varying SSM is good for capturing this change.

Other advantages of SSM include:

> a. Use SSM to handle missing values by using forecast function.
> b. Apply SSM to build a model for complex multivariate systems.
> c. Use SSM to build a model for non-stationary time series.

In the next section, we will introduce the Kalman filter. The Kalman filter is the most studied state space model, and we will use Kalman filter as the base state space model to build a GARCH(1,1) model.

1.4. Estimation of Parameters for State Space Model

Once the state space model is specified, we can estimate the coefficients. We have to estimate these parameters: A_t, Φ_t, V_t and W_t.

The most popular estimation method is MLE. However, Bayesian estimation is also another viable choice for parameter estimation. Actually, Bayesian estimation is a suitable approach for a state space model given that the nature of the question of SSM is to estimate the values of states given all the observations we know. In fact, we will use a Bayesian approach to discuss our next topic: Kalman filter.

2. Kalman Filter

2.1. Kalman Filter Model Setup

Kalman filter is a linear state space model with normal (Gaussian) noise. It has been very popular due to its recursive method that can avoid heavy computing and storage issues happening to other statistical analysis methods. In this section, we will study a time invariant multivariate Kalman filter model. The properties of the Kalman filter derived in this section still apply to Kalman filter models where some or all of the parameters are time varying. Let's write down the model and discuss its properties.

Assume we have x_t as a $r \times 1$ state vector and y_t as a $n \times 1$ observation vector. The model can be written as follows:

Observation Equation: $y_t = F'x_t + v_t,\ \ v_t$ is the measurement noise.

State Equation: $x_t = Gx_{t-1} + w_t,\ \ w_t$ is the system noise.

where $v_t \sim N(0, V)$ and $w_t \sim N(0, W)$.

We also assume the initial value of x_t to be x_0 and x_0 has the $x_0 \sim N(m_0, C_0)$ distribution and m_0 and C_0 are known.

Following the assumption from the last section, we still assume that v_t and v_s are independent when $t \neq s,\ w_t$ and w_s are independent when $t \neq s$, and v_t and w_s are uncorrelated on all lags.

In the model, F is a $r \times n$ matrix, and it is called an observation matrix or design matrix. G is a $r \times r$ matrix and it is called a state transition matrix.

Given so many variables we have mentioned so far, we will summarize these variables in the following list:

> x_t: $r \times 1$ state vector at time t;
> y_t: $n \times 1$ observation vector at time t;
> F: $r \times n$ observation matrix between observation and state at time t;
> G: $r \times r$ state transition matrix between current state at t and prior state at $t-1$;
> v_t: measurement noise from multivariate normal distribution;

> w_t: system noise from multivariate normal distribution;
> V: variance-covariance matrix for multivariate normal distribution where measurement noise is from;
> W: variance-covariance matrix for multivariate normal distribution where system noise is from;
> m_0: mean value of multivariate normal distribution from initial state, x_0;
> C_0: variance-covariance matrix of multivariate normal distribution from initial state, x_0.

2.2. Kalman Filter Model Derivation

In the last section, we went through the Kalman filter model setup and described all the variables in this model. In this section, we are going to explain how Kalman filter model works and what the key results are from this model. We are going to use a Bayesian approach to explain how a Kalman filter model works.

In the previous section, we mentioned that the concept of using all the information from the past and new data to infer the unobserved state in a state space model is very similar to the Bayesian updating process. Let's try to demonstrate what a state space model is looking for in terms of a Bayes theorem.

For a Kalman filter model, we can express it using the following Bayes theorem formula.

$$P(x_t|y_t, D_{t-1}) = \frac{P(y_t|x_t)P(x_t|D_{t-1})}{P(y_t)}$$

Where x_t is current state, y_t is current observation, D_{t-1} is all the pass information, including pass states and pass observations.

Here is the breakdown of prior probability, likelihood function, and posterior probability.

> Posterior probability: $P(x_t|y_t, D_{t-1})$
> Likelihood function: $P(y_t|x_t)$
> Prior probability: $P(x_t|D_{t-1})$

The above equation says the probability of getting the value of state x_t at time t, given we know the current observation y_t and all our prior knowledge D_{t-1}, is the multiplication of the likelihood of y_t happening given x_t and the probability of the state x_t happening given all our prior knowledge D_{t_1}. We also normalize the multiplication by dividing the probability of y_t happening. This is the process of updating the value of current state x_t with the new information y_t.

With the above Bayesian equation and some mathematical manipulations, we can derive the following results:

> **Prior:** $x_t|D_{t-1} = x_t|y_{1:t-1} \sim N(a_t, R_t)$
> Where $a_t = Gm_{t-1}$ and $R_t = G'C_{t-1}G + W$

> **Likelihood:** $y_t|x_t \sim N(Fx_t, V)$

> **One-Step Forecast:** $y_t|y_{t-1} \sim N(f_t, Q_t)$
> Where $f_t = Fa_t$ and $Q_t = F'R_tF + V$

> **Posterior:** $x_t|D_t = x_t|y_{1:t} \sim N(m_t, C_t)$
> Where $m_t = a_t + K_te_t$
> $C_t = R_t - K_t'Q_tK_t = (1 - K_tF)R_t$
> $K_t = R_tFQ_t^{-1}$ (K_t is called Kalman gain matrix)
> $e_t = y_t - f_t$ and $e_t \sim N(0, Q_t)$

The above posterior result is called Kalman filter recursion. Remember there are three goals for the state space model described before. This result is to achieve the filtering goal: Use all the past information and new data to estimate the current value of the state. We use the mean of the posterior distribution m_t as our estimated value of the current state x_t, given all the information we have.

Let's look at the above result intuitively. By looking at the mean of the posterior equation, we can see m_t is a linear combination of previous state m_{t-1} and new information y_t. We can view the estimated value of the current state as the weighted average of the value of the past state and new information. K_t (Kalman gain matrix) is one of the weights in m_t formula. We can see K_t as a tuning factor to decide if more weight should be put on new information or not. The higher the value of K_t, the more weight the model puts on new information. Also, the estimated current state is only dependent on the state value of the last period and new value; there is no need to have all the historical data to update the current state. This recursive method of updating current state value with new information and only the last period of the state makes the forecast of Kalman filter a lot easier than other methods.

2.3 Forecasting with Kalman Filter

One key reason to learn Kalman filter is to use it to conduct forecasting. Forecasting includes predicting x_{t+d} and y_{t+d} given $y_{1:t}$. We can apply the result from the last section to conduct forecasting.

When $d = 1$, the forecasting result can be derived from prior and one-step forecast equations from last section.

> **State at $t + 1$:**
> $x_{t+1} = Gm_t$ forecast state
> $R_{t+1} = G'C_tG + W$ forecast variance of state

> **Observation at $t + 1$:**
> $y_{t+1} = Fx_{t+1}$ forecast observation
> $Q_{t+1} = F'R_{t+1}F + V$ forecast variance of observation

When $d > 1$, we can use a recursive method to obtain the result as follows:

> **State at $t+d$:**
> $x_{t+d} = Gm_{t+d-1}$ forecast state
> $R_{t+d} = G'C_{t+d-1}G + W$ forecast variance of state

> **Observation at $t+d$:**
> $y_{t+d} = Fx_{t+d}$ forecast observation
> $Q_{t+d} = F'R_{t+d}F + V$ forecast variance of observation

3. Use Kalman Filter to Fit GARCH(1,1) Model

In this section, we are going to apply Kalman filter to build a GARCH(1,1) model. We have studied in the previous lessons that GARCH model has conditional heteroskedasticity feature on the observation level. Hence, we need to incorporate this feature to a Kalman filter model. We can write the model as follows:

$$
\begin{aligned}
y_t &= Fx_t + z_t \\
x_t &= Gx_{t-1} + w_t, \text{ where } w_t \sim N(0, W) \\
z_t &= s_t\epsilon_t, \text{ where } \epsilon_t \sim N(0,1) \\
s_t^2 &= \alpha_0 + \alpha_1 z_{t-1}^2 + \beta_1 s_{t-1}^2
\end{aligned}
$$

The first two equations from the above equation system are from the Kalman filter. The first one is the observation equation and the second one is the state equation. In the observation equation, we can see the measurement noise has a further specification from the third and fourth equations. The third and fourth equations are the section for GARCH. The measurement noise is now following standard normal GARCH(1,1) set up.

From the above system of equations for Kalman filter-GARCH(1,1) model, we can do some manipulation and obtain the posterior and one-step forecast result.

> **Posterior for state:** $x_t|y_{1:t} \sim N(m_t, C_t)$
> Where all the parameters are the same as standard Kalman filter model

> **One-step forecast for observation:** $y_t|y_{t-1} \sim N(f_t, Q_t)$
> Where $f_t = Fa_t$ and $Q_t = F'R_tF + s_t^2$

We can see from the above result, the only difference between the standard Kalman filter model and Kalman filter-GARCH(1,1) model is the s_t^2 in Q_t, the variance for observation.

Once we have the posterior and one-step forecast equations set up, we can use either Gibbs sampling or Metropolis-Hasting sampling method to sample unknown parameters. For unknown state vectors, we use a technique called forward filtering and backward sampling (FFBS) to sample. FFBS was proposed by Carter and Kohn as well as Fruhwirth-Schnatter. Most statistical software will provide FFBS sampling function for use. For more information on how to sample, Omar provides detailed steps to sample prior distribution and simulate data with FFBS on Kalman filter GARCH(1,1) model.

4. Shiny Application For GARCH(1,1) Model

In this module, we provided a Shiny application to build a GARCH(1,1) model on an asset return.

Click here to access the application.First, you choose an asset return for the model. There are three choices: Google stock price daily return, EUR/USD daily return, and U.S. 10-year Treasury bond yield daily return. You then use the error distribution for a GARCH(1,1) model. There are four choices: normal, Student's t, skew normal, and skew Student's t.

The data summary tab will show the time series plot, histogram, and QQ plot of the selected asset return. The squared return summary tab will show the time series plot for the squared return. This tab will also provide ACF and PACF plots for both daily returns and squared daily returns of the selected asset return. The last tab will present the GARCH(1,1) model result and model diagnostic information.

5. Conclusion

In this lesson, we went through the theory of state space model. We talked about the basics of state space model, its properties, its benefits, and its estimation method. We then discussed a popular linear state space model, Kalman Filter. We explained how Kalman Filter works, how to derive the model and how to use it to forecast. After learning Kalman Filter, we combined Kalman Filter with GARCH(1,1) model and gave a brief model setup for Kalman Filter-GARCH(1,1) model. This lesson concludes our study on conditional heteroskedasticity in time series data. In the next module, we will dive into the method to check for stationary for a time series. We will also look into the relationship of two time series.

References

- Carter, Chris, and Robert Kohn. "On Gibbs Sampling for State Space Models." Biometrika, vol. 81, 1994, pp. 541–553.
- Fruhwirth-Schnatter, Sylvia. "Data Augmentation and Dynamic Linear Models." Journal of Time Series Analysis, vol. 15, 1994, pp. 183–202.
- Omar, Zayd. State-Space Models with GARCH Errors: Application to Health Data. 2019. McGill University, Master's thesis.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

14.384 Time Series Analysis, Fall 2007
Professor Anna Mikusheva
Paul Schrimpf, scribe
Novemeber 29, 2007
revised December 3, 2009

Lecture 23-24

Intro to Bayes approach. Reasons to be Bayesian

Many ideas are borrowed from Lecture notes written by Frank Schorfheide.
Bayesian models has two pieces:

- A parametric model, giving a distribution, $f(\mathcal{Y}_T|\theta)$, for the data given parameters
- A prior distribution for the parameters, $p(\theta)$

From these, we can form the joint distribution of the data and parameters,

$$p(\mathcal{Y}_T, \theta) = f(\mathcal{Y}_T|\theta)p(\theta)$$

and the marginal distribution of the data

$$p(\mathcal{Y}_T) = \int f(\mathcal{Y}_T|\theta)p(\theta)d\theta$$

Finally, using Baye's rule, we can form the posterior distribution of the parameters given the data

$$p(\theta|\mathcal{Y}_T) = \frac{f(\mathcal{Y}_T|\theta)p(\theta)}{p(\mathcal{Y}_T)}$$

One can make inferences based on the posterior distribution. For example, we can report the mode (or the mean) as a parameter estimate. Any set of posterior measure biggen than $1-\alpha$ is called an $1-\alpha$ credible set. Hypotheses can be tested based on posterior odds.

Differences between Bayesian and Frequentist Approaches

Frequentist

- θ is fixed, but unknown
- Uncertainty comes from sampling uncertainty. That is, from the fact that we can get different samples.
- All probabilistic statements are statements about sampling uncertainty. For example,
  - $E_\theta\hat{\theta}(\mathcal{Y}_T) = \theta$ (unbiasedness) means in average over all possible repeated samples, one receives the true value
  - $P_\theta\{\theta \in C(\mathcal{Y}_T)\} = 1-\alpha$ coverage probability of confidence sets is a statement about the ratio (in repeated samples) of sets containing θ. Once we observe a sample, θ is either in the set or not; there is no probability, after realization of a sample. The coverage probability is a statement about ex-ante probability.

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

Bayesian

- θ is random
- $\mathcal{Y}_T$ is treated as fixed after observed
- uncertainty = "beliefs" about θ
- All probabilistic statements are about uncertainty about θ.
  - $P_\theta\{\theta \in C(\mathcal{Y}_T)\} = 1 - \alpha$ coverage probability is the probability that θ is in the set.

Reasons to be Bayesian

Reason 1 – Philosophical

Example 1. Two observations, $y_1, y_2 \sim$ iid.

$$P_\theta(y_i = \theta - 1) = \frac{1}{2} = P_\theta(y_i = \theta + 1)$$

Consider a confidence set:

$$C(y_1, y_2) = \begin{cases} \frac{y_1+y_2}{2} & y_1 \neq y_2 \\ y_1 - 1 & y_1 = y_2 \end{cases}$$

If we observe $y_1 \neq y_2$, which will happen $1/2$ the time, we know $\theta = C(y_1, y_2)$. Otherwise, we have a probability of $1/2$ that $\theta = C(y_1, y_2)$. From a frequentist perspective, then the coverage of this set is $1/2 + 1/2 * 1/2 = 75\%$. Now, suppose we observe $y_1 \neq y_2$. Then we know θ with certainty. Why would we then report a coverage of 75% (ex-ante coverage) rather than the ex-post accuracy of 100%? Frequentists average probabilities over all situations that may have been realized, but were not. Bayesians are conditioning on the realization. As this example shows, conditioning on observation may be justified.

This example might appear artificial. However, especially in time series, there are fundamental reasons to be Bayesian. Perhaps the frequentist perspective makes sense in a cross section. In a cross section, we can imagine taking different samples and repeating our "experiment" (idea of repeated samples). However, in time series we often have only one realization, and it is difficult to imagine where we would obtain another sample. For example, if our data is U.S. inflation, then we would need another world to get another sample.

Bayes is based on *Conditional principle*: if experiment is selected by some random mechanism independent of θ, then only the experiment actually performed is relevant, not the experiments that may have been performed.

Another philosophical reason to be Bayesian is that we do have prior beliefs about parameters and we should incorporate them in a coherent way. Some people criticize Bayesians for imposing too much parametric structure and prior beliefs. Bayesians argue that even frequentists implicitly impose priors by choosing which models to estimate and which results to report. At least the role of priors is more aparent in Bayesian econometrics.

Conjugate Priors

When a prior and a posterior are in the same family of distributions, then the prior is called the conjugate prior for the distribution of the data. There are a handful of well behaved cases, which are very convenient and easier to work with.

Example 2. *OLS* The model is

$$y_t = x_t\theta + u_t$$

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

with $u_t \sim iidN(0,1)$. In matrix form we'll write $Y = X\theta + U$. The distribution of the data is

$$f(Y|X,\theta) = (2\pi)^{-T/2} \exp\left(-\frac{1}{2}(Y - X\theta)'(Y - X\theta)\right)$$

If we choose a normal prior, $\theta \sim N(0, \tau^2 I_k)$,

$$p(\theta) = (2\pi\tau^2)^{-k/2} \exp\left(\frac{-1}{2\tau}\theta'\theta\right)$$

then the posterior is

$$\begin{aligned} p(\theta|Y,X) &\propto \exp\left(-\frac{1}{2}\left[-Y'X\theta - \theta'X'Y + \theta'X'X\theta + \frac{1}{\tau^2}\theta'\theta\right]\right) \\ &\propto \exp\left(-\frac{1}{2}\left[-Y'X\theta - \theta'X'Y + \theta'(X'X + \frac{I_k}{\tau^2})\theta\right]\right) \\ &\propto \exp\left(-\frac{1}{2}\left[\left(\theta - (X'X + \frac{I_k}{\tau^2})^{-1}X'Y\right)'(X'X + \frac{I_k}{\tau^2})\left(\theta - (X'X + \frac{I_k}{\tau^2})^{-1}X'Y\right)\right]\right) \end{aligned}$$

so $\theta|Y,X \sim N(\tilde{\theta}, \tilde{\Sigma})$ with

$$\begin{aligned} \tilde{\theta} &= (X'X + \frac{I_k}{\tau^2})^{-1}X'Y \\ \tilde{\Sigma} &= (X'X + \frac{I_k}{\tau^2})^{-1} \end{aligned}$$

Also, we see that as $\tau \to \infty$ (uninformative prior), $\tilde{\theta} \to (X'X)^{-1}X'Y = \hat{\theta}^{ML}$, and as $\tau \to 0$, $\tilde{\theta} \to 0$, the prior dominates. Furthermore, if we fix τ and $T \to \infty$ with $\frac{X'X}{T} \to Q_{XX}$, then $\tilde{\theta} \to \theta_0$, the frequentist limit. So, prior vanishes asymptotically. This result is more general.

Reason 2- priors vanish asymptotically

The prior vanishes asymptotically. All inference in "clean" situations asymptotically converge to frequentists'. This claim is based on the following theorem:

Theorem 3. *(Geweke, p.93) Suppose*

1. *Prior is absolutely continuous wrt to Lebesgue measure and prior puts positive probability on all sets with positive Lebesgue measure*
2. *Uniform convergence of likelihood* $\frac{1}{T}\log f(\mathcal{Y}_T|\theta) \to^{a.s.} l(\theta)$ *uniformly in* θ
3. $l(\theta)$ *is continuous and has a unique maximum at* θ^*

Then for any open neighborhood, $\varepsilon(\theta^*)$*,*

$$\lim_{T\to\infty} P(\theta \in \varepsilon(\theta^*)|\mathcal{Y}_T) = 1 \ a.s.$$

Theorem says that the posterior concentrates around the asymptotic limit of frequentist MLE, and in any "reasonable" situation Baeys estimate in large samples will be close to MLE. There's a similar theorem that shows asymptotic normality of the Bayesian estimator. These two theorems sort of say what happens when you use Bayesian methods in a frequentist world. One interpretation is that the prior vanishes asymptotically. However, you should be cautious about this theorem!

Cautions:

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

- The theorem is about asymptotics. However, the prior can influence inferences in finite samples.
- Condition 3 is an identification condition. If you are not identified, then where the Bayesian estimator converges depends on your prior.
- Prior should not restrict parameter space (condition 1)
- Condition 2 is like a LLN, it may not be satisfied with non-stationarity

Reason 3 – Decision Theory

All Bayes rules possess an optimality property in frequentists' world- they are admissible. And under some conditions all admissible rules are Bayes. For explaining what it means we have to introduce some new concepts.

Suppose we have a loss function $\mathcal{L}(a, \theta)$, where θ is a parameter and a is some action that we want to choose. For example, if we just want to estimate θ, we might have $a = \hat{\theta}$ and $\mathcal{L}(a, \theta) = (a - \theta)^2$. Our goal is to come up with a decision rule $a(\mathcal{Y}_T)$ that depends on the sample $\mathcal{Y}_T$, and give a small loss. Let our expected loss(called *risk*) for a given value of θ be

$$R_a(\theta) = E_\theta \mathcal{L}(a(\mathcal{Y}_T), \theta)$$

Note, that it is frequentist notion (expectation is taken over repeated samples)!!! In example above the risk is MSE. We would like $a(\mathcal{Y}_T)$ to minimize our expected loss. However, in general, the solution will depend on θ.

Definition 4. A decision rule, $a()$, is *admissible* if there exists no $\tilde{a}()$ such that $R_a(\theta) \geq R_{\tilde{a}}(\theta) \forall \theta$ with strict inequality for some θ_0.

Definition 5. A *Bayesian Decision Rule* minimizes a weighted risk (with weights $p(\theta)$):

$$\begin{aligned}
\min_a \int R_a(\theta) p(\theta) d\theta &= \min_a \int E_\theta \mathcal{L}(a(\mathcal{Y}_T), \theta) p(\theta) d\theta \\
&= \min_a \int \int \mathcal{L}(a(\mathcal{Y}_T), \theta) f(\mathcal{Y}_T|\theta) p(\theta) d\mathcal{Y}_T d\theta \\
&= \min_a \int \left[\int \mathcal{L}(a(\mathcal{Y}_T), \theta) p(\theta|\mathcal{Y}_T) d\theta \right] p(\mathcal{Y}_T) d\mathcal{Y}_T \\
&= \int \left[\min_a \int \mathcal{L}(a(\mathcal{Y}_T), \theta) p(\theta|\mathcal{Y}_T) d\theta \right] p(\mathcal{Y}_T) d\mathcal{Y}_T
\end{aligned}$$

So, Bayes rule for each realization $\mathcal{Y}_T$ solves for minimum of the posterior risk!

Theorem 6. *All Bayesian decision rules are admissible. Also, under some conditions, all admissible decision rules are Bayesian.*

Reason 4 – Nuisance Parameters

Let $\omega = h(\theta)$. Let $C(\mathcal{Y}_T)$ be a set such that $P(\omega \in C(\mathcal{Y}_T)|\mathcal{Y}_T) = 1 - \alpha$. It is very easy to go from $p(\theta|\mathcal{Y}_T)$ to $p(\omega|\mathcal{Y}_T)$. For example, suppose $\theta = (\theta_1, \theta_2)$ and $\omega = \theta_1$, then

$$p(\theta_1|\mathcal{Y}_T) = \int p(\theta_1, \theta_2|\mathcal{Y}_T) d\theta_2$$

This example is especially relevant because there are many examples in econometrics where we want to eliminate nuisance parameters. Here, θ_2 would be the nuisance parameters. The Bayesian approach makes it very easy to deal with the nuisance parameters. Whereas in the frequentist world, nuisance parameters are an extremely difficult problem.

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

Reason 5 – Easier to Implement

In some cases it can be easier to compute Bayesian estimates than frequentist ones. For example, in the last lecture, we saw how it can be difficult to estiate DSGE models by MLE. The main difficulty is in multidimensional optimization. In some cases, it can be easier to estimate these models using Bayesian methods and MCMC.

Reason 6 – Not Fully Identified, Priors add Identification

This is probably a bad reason to be Bayesian. If your model is not identified, then your estimates will be strongly influenced by your prior.

Bayes estimates, tests and sets.

Point Estimation

There are several Byes estimates depending on your loss function.

Our action is to choose a parameter from the parameter set $a \in \mathcal{A} = \Theta$. There are a number of loss functions we could use:

1. Quadratic loss function:
$$\mathcal{L}(\delta, \theta) = (\delta - \theta)' Q (\delta - \theta)$$
For some positive definite Q. If $Q = I$, then the risk is MSE. The corresponding Byes rule will be posterior mean $\hat{\delta} = E(\theta | \mathcal{Y}_T)$.

2. The loss function could be
$$\mathcal{L}(\delta, \theta) = \begin{cases} 1 & \delta \neq \theta \\ 0 & \delta = \theta \end{cases}$$
. If the parameter space is finite, then the corresponding risk is the probability of choosing a wrong value of θ. The optimal decision rule is then
$$\delta(\mathcal{Y}_T) = \arg\max_{\theta'} P(\theta = \theta' | \mathcal{Y}_T), \text{ which is the mode}$$
In the continuous case, $\forall \epsilon > 0$,
$$\mathcal{L}(\delta, \theta; \epsilon) = 1 - \mathbf{1}_{\{\theta \in \mathcal{N}_\epsilon(\delta)\}}$$
Let δ_ϵ denote the optimal decision rule for this loss function. Then $\lim_{\epsilon \to 0} \delta_\epsilon$ = the mode of the posterior distribution.

3. Check function:
$$\mathcal{L}(\delta, \theta) = (1 - q)(\delta - \theta)\mathbf{1}_{\{\theta < \delta\}} + q(\theta - \delta)\mathbf{1}_{\theta > \delta}$$
Then $\hat{\delta}$ is the q-th quantile of the posterior distribution of θ. For $q = 1/2$ we have the median.

Testing

The null hypothesis is $H_0 : \theta \in \Theta_0$, and the alternative is $H_1 : \theta \in \Theta_1$. Our action space is $\mathcal{A} = \{0 = \text{reject}, 1 = \text{accept}\}$. In a frequentists setting, the null and alternative hypotheses are not treated equally. The null is accepted unless there is strong evidence against it. In a Bayesian setting, the way the null and alternative are treated depends on the loss function. Consider the following loss function:

$$\mathcal{L}(\delta, \theta) = \begin{cases} 0 & \text{if correct} \\ a_1 & \delta = 0, \theta \in \Theta_0 \text{ (type 1 error)} \\ a_2 & \delta = 1, \theta \in \Theta_1 \text{ (type 2 error)} \end{cases}$$

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

Then the optimal decision rule is

$$\delta(\mathcal{Y}_T) = \begin{cases} 1 & \text{if } P(\theta \in \Theta_0|\mathcal{Y}_T) \geq \frac{a_2}{a_1+a_2} \\ 0 & \text{otherwise} \end{cases}$$

That is, we accept if the expected posterior loss from a type 1 error is bigger than the expected loss of a type 2 error

$$\begin{aligned} a_1 P(\theta \in \Theta_0|\mathcal{Y}_T) \geq & a_2 P(\theta \in \Theta_1|\mathcal{Y}_T) \\ \frac{P(\theta \in \Theta_0|\mathcal{Y}_T)}{P(\theta \in \Theta_1|\mathcal{Y}_T)} \geq & \frac{a_2}{a_1} \end{aligned}$$

from which we see that the null and alternative are treated symmetrically (one could easily relabel the null and alternative).

Example 7. $\Theta = \{0, 1\}$, we observe $y \in \{0, 1, 2\}$. The distribution of y given θ is:

y	0	1	2
$P_{\theta=0}$	0.89	0.04	0.07
$P_{\theta=1}$	0.959	0.04	0.001

In a frequentist world, if we observe $y = 1$ and test $H_0 : \theta = 1$, we will reject because the $P(y \geq 1|\theta = 1) < 0.05$. In a Bayesian, world we condition our test on the observed value of y and $y = 1$ is not informative about whether $\theta = 1$ or not (the posterior equals the prior, $p(\theta = 1|y = 1) = p(\theta = 1)$).

OLS

As in the previous lecture, the model is:

$$y_t = \theta X + U$$

with $U \sim iidN$. The prior is $\theta \sim N(0, \tau^2 I_k)$. Then the posterior is $\theta|Y, X \sim N(\tilde{\theta}, \tilde{\Sigma})$ with

$$\begin{aligned} \tilde{\theta} = & (X'X + \frac{I_k}{\tau^2})^{-1} X'Y \\ \tilde{\Sigma} = & (X'X + \frac{I_k}{\tau^2})^{-1} \end{aligned}$$

Inequality test: We want to test $H_0 : \theta < 0$ vs $H_1 : \theta \geq 0$. Assume $a_1 = a_2$, we would accept H_0 if:

$$\begin{aligned} P(\theta < 0|\mathcal{Y}_T) = & P\left(\frac{\theta - \tilde{\theta}}{\sqrt{\tilde{\Sigma}}} < -\frac{\tilde{\theta}}{\sqrt{\tilde{\Sigma}}}|\mathcal{Y}_T\right) & > \frac{1}{2} \\ = & \Phi\left(-\frac{\tilde{\theta}}{\sqrt{\tilde{\Sigma}}}\right) & > \frac{1}{2} \\ \tilde{\theta} < & & 0 \end{aligned}$$

With $\alpha = 5\%$, the frequentist test is based on:

$$\begin{aligned} \frac{\tilde{\theta}}{\sqrt{(X'X)^{-1}}} < & 1.64 \\ \tilde{\theta} < & 1.64\sqrt{(X'X)^{-1}} \end{aligned}$$

The Bayesian test of this hypothesis is consistent in the sense that as the sample size grows, we get the correct answer with probability 1. In a frequentist world, the probability of getting the right answer for the true null is called 1-size and approaches $1 - \alpha$.

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

Point null: If we want to test $H_0 : \theta = 0$ vs $H_1 : \theta \neq 0$ and we use a continuous prior, then we will always reject the null. The solution is to specify a prior with a point mass of λ on 0. Take some continuous prior $p(\theta)$, let

$$p^*(\theta) = \lambda \Delta_{\theta=0} + (1-\delta)p(\theta)$$

where $\Delta_{\theta=0}$ is Dirac measure, then

$$\begin{aligned} p(\mathcal{Y}_T) =& \lambda f(\mathcal{Y}_T|0) + (1-\lambda)\int f(\mathcal{Y}_T|\theta)p(\theta)d\theta \\ p(\theta = 0|\mathcal{Y}_T) =& \frac{\lambda f(\mathcal{Y}_T|\theta=0)}{p(\mathcal{Y}_T)} \end{aligned}$$

The posterior odds ratio of the null to the alternative is

$$\frac{\lambda f(\mathcal{Y}_T|\theta=0)}{(1-\lambda)\int f(\mathcal{Y}_T|\theta)p(\theta)d\theta}$$

One difficulty here is in taking the integral in the denominator. Instead we can use the following trick: in the model without a mass point, we had $p(\mathcal{Y}_t) = \int f(\mathcal{Y}_T|\theta)p(\theta)d\theta = \frac{f(\mathcal{Y}_T|\theta)p(\theta)}{p(\theta|\mathcal{Y}_T)}$.

Continue our OLS example . How consider their a test $H_0 : \theta = 0$ vs $H_1 : \theta \neq \theta_0$ with point mass $\lambda = 1/2$. We know that $f(\mathcal{Y}_T|\theta)$ is normal $N(\theta X_T, I)$, prior $p(\theta)$ is $N(0, \tau^2 I)$ and hence the posterior is also normal with mean and variance calculated before. We can plug it in $\int f(\mathcal{Y}_T|\theta)p(\theta)d\theta = \frac{f(\mathcal{Y}_T|\theta)p(\theta)}{p(\theta|\mathcal{Y}_T)}$ and get $\int f(\mathcal{Y}_T|\theta)p(\theta)d\theta$. After some calculation we can obtain that the posterior odds is equal to

$$\frac{\lambda f(\mathcal{Y}_T|\theta=0)}{(1-\lambda)\int f(\mathcal{Y}_T|\theta)p(\theta)d\theta} = \tau^k |X'X + \frac{I}{\tau^2}|^{1/2} \exp\left(-\frac{1}{2}\left(Y'X\left(X'X + + \frac{I}{\tau^2}\right)^{-1} X'Y\right)\right)$$

If costs $a_1 = a_2$ then the null is accepted if posterior odds are bigger than 1.

We will compare this test to the frequentist by studying the asymptotics of the log posterior odds ratio as $T \to \infty$. Assume that $\frac{X'X}{T} \to Q$

$$\log(po) = k \ln \tau + \frac{k}{2}\ln T + \frac{1}{2}\ln|\frac{X'X}{T} + \frac{I}{T\tau^2}| - \frac{1}{2}\left(Y'X\left(X'X + + \frac{I}{\tau^2}\right)^{-1} X'Y\right)$$

$k \ln \tau$ is a constant, and $\frac{1}{2}\ln|\frac{X'X}{T} + \frac{I}{T\tau^2}|$ converges to a constant so we may ignore them. We are interesting in whether $\lim \log(po) > 0$. Suppose $\theta_0 = 0$. Then,

$$\begin{aligned} Y'X\left(X'X + + \frac{I}{\tau^2}\right)X'Y =& \frac{Y'X}{\sqrt{T}}\left(\frac{X'X}{T} + \frac{I}{T\tau^2}\right)^{-1}\frac{X'Y}{\sqrt{T}} \\ \Rightarrow& N(0,\Sigma)Q^{-1}N(0,\Sigma)' \end{aligned}$$

This last expression is asymptotically bounded, so $\log(p.o.) = k\ln T + O_p(1) \to \infty$ and we accept the null asymptotically.

Now suppose $\theta_0 \neq 0$. Then $\frac{Y'X}{T} \to \theta_0 \frac{X'X}{T} \to \theta_0 Q$, so

$$\begin{aligned} Y'X\left(X'X + \frac{I}{\tau^2}\right)X'Y =& T\frac{Y'X}{T}\left(\frac{X'X}{T} + \frac{I}{T\tau^2}\right)\frac{X'Y}{T} \\ \to& T\theta_0 Q(Q)^{-1}Q'\theta_0' \\ \to& T\theta_0 Q\theta_0' \end{aligned}$$

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

so this term (which is negative in $\ln po$) asymptotically dominates $\log(p.o) = -T\theta_0 Q\theta_0'/2 + k\ln T + O_p(1) \to -\infty$and we reject the null asymptotically. Again, this test is asymptotically consistent, but frequentist tests are not. Consider what the frequentist test would be in this situation. The likelihood ratio is:

$$\begin{aligned} LR =& 2\ln\frac{f(Y|X,\hat{\theta}_{ML})}{f(Y|X,\theta=0)} \\ =& Y'X(X'X)^1X'Y \Rightarrow \chi^2 \end{aligned}$$

In particular, both Bayesian and frequentist test look at $Y'X(X'X)^1X'Y$ and reject if it is greater than some number, but Bayesians use $k\ln T$ and frequentists use χ^2 critical value. As a result, Bayes test is consistent, but frequentist test has a power against local alternatives $\theta = c/\sqrt{T}$ (while Bayes not).

To understand why the Bayesian test is consistent and the frequentist one is not, we need two asymptotic facts,

1. CLT: $\frac{1}{\sqrt{T}}\sum y_t \Rightarrow N(0,1)$
2. Law of iterated logarithm: $\frac{1}{\sqrt{2T\ln\ln T}}\sum y_t$ is almost surely asyptotically in $\in [-1,1]$, meaning that this sum does not converge, but for any number in $[-1,1]$, we can find a subsequence converging to that number, and the limits of all converging subsequences are in $[-1,1]$. In particular, almost surely (for any realization) there are infinitely many T such that $\frac{1}{\sqrt{2T\ln\ln T}}\sum_{t=1}^{T} y_t > 1-\delta$ for any δ, so we can always find infinitely many samples that will reject a null if we use frequentist a test.

   Cynical interpretation: every null can be rejected as long as enough data is collected:)

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

MIT OpenCourseWare
http://ocw.mit.edu

14.384 Time Series Analysis

Fall 2013

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

14.384 Time Series Analysis, Fall 2007
Professor Anna Mikusheva
Paul Schrimpf, scribe
Novemeber 15, 2007
revised November 24, 2009

Lecture 21

Filtering. State space models. Kalman filter.

State-Space Models

In this lecture we consider state-space models, which often appear in macro, as well as other areas of economics.

Example 1. For example, suppose, GDP growth, y_t is given by

$$\begin{aligned} y_t =& \mu_t + \epsilon_t \\ \mu_t =& \mu_{t-1} + \eta_t \end{aligned}$$

where μ_t is the slow moving component of GDP growth and ϵ_t is noise with $(\epsilon_t, \eta_t) \sim iid\, N\left(0, \begin{pmatrix} \sigma_\epsilon^2 & 0 \\ 0 & \sigma_\eta^2 \end{pmatrix}\right)$

. Assume that a researcher observe y_t, but not the latent trend μ_t.

Example 2. *Markov Switching*

$$\begin{aligned} y_t =& \beta_0 + \beta_1 S_t + \epsilon_t \\ S_t \in& \{0, 1\} \\ P(S_t = 1 | S_{t-1} = 0) =& 1 - q \\ P(S_t = 1 | S_{t-1} = 1) =& p \end{aligned}$$

If y_t is GDP growth, we might think of S_t as representing whether or not we're in a boom. We observe y_t, but not the latent state S_t.

What is common for the two examples above is that there are some observed variable y_t whose behavior depends on unobserved state (μ_t or S_t), let's call it α_t. Some questions we might want to answer in these examples include:

1. Estimate parameters: e.g. in example 1 estimate σ_ϵ and σ_η
2. Extract unobserved state: e.g. in example 1 estimate μ_t
3. Forecast future values of y_t.

In a state space model, we have an (potentially unobserved) state variable, α_t, and measurements, y_t. Let $\mathcal{Y}_{t-1}$ be all measurable ($\{y_1, ..., y_{t-1}\}$) variables up to time $t-1$. The state space model is characterized by

(1) State equation, describing the evolution of state:

$$F(\alpha_t | \alpha_{t-1}, \mathcal{Y}_{t-1})$$

(2) Measurement equation, describing how the measurable variables relate to state variables

$$f(y_t | \alpha_t, \mathcal{Y}_{t-1})$$

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

The two equations allow us to write the joint likelihood of observed variables $(y_1, y_2, ..., y_T)$. There could be some unknown parameter θ which we will drop for simplicity later on.

$$f(y_1, ..., y_T; \theta) = f(y_1; \theta) \prod_{t=2}^{T} f(y_t | y_{t-1}, ..., y_1; \theta) = f(y_1; \theta) \prod_{t=2}^{T} f(y_t | \mathcal{Y}_{t-1}; \theta)$$

For the likelihood, we need to find $f(y_t|\mathcal{Y}_{t-1})$. This can be done in three general steps and is called filtering:

1.
$$f(y_t|\mathcal{Y}_{t-1}) = \int f(y_t|\alpha_t, \mathcal{Y}_{t-1}) f(\alpha_t|\mathcal{Y}_{t-1}) d\alpha_t$$

2. Prediction equation
$$f(\alpha_t|\mathcal{Y}_{t-1}) = \int F(\alpha_t|\alpha_{t-1}, \mathcal{Y}_{t-1}) f(\alpha_{t-1}|\mathcal{Y}_{t-1}) d\alpha_{t-1}$$

3. Updating equation
$$f(\alpha_t|\mathcal{Y}_t) = \frac{f(y_t|\alpha_t, \mathcal{Y}_{t-1}) f(\alpha_t|\mathcal{Y}_{t-1})}{f(y_t|\mathcal{Y}_{t-1})}$$

On a theoretical level, this process is clear and straightforward. We go from $f(\alpha_1|\mathcal{Y}_0)$ to $f(y_1|\mathcal{Y}_0)$ to $f(\alpha_1|\mathcal{Y}_1)$ to $f(\alpha_2|\mathcal{Y}_1)$ to finally get $f(y_2|y_1)$, the conditional likelihood. However, notice that the two first steps include integration, and in practice, it is usually difficult to compute these integrals. There are two cases where the integration is straightforward. With discrete distributions, the integrals are just sums. With normal distributions, sub-vector of normal vector is normal, and conditionals are normal as well, this simplify life a lot. The corresponding procedure for normals is called Kalman filter. For other situations, integration is very difficult, one approach is called particle filtering.

Kalman Filtering

Suppose we have a state model:

$$\alpha_t = T\alpha_{t-1} + R\eta_t \tag{1}$$

and a measurement:

$$y_t = Z\alpha_t + S\xi_t \tag{2}$$

with $\begin{pmatrix} \eta_t \\ \xi_t \end{pmatrix} \sim iid\, N\left(0, \begin{pmatrix} Q & 0 \\ 0 & H \end{pmatrix}\right)$. Then,

$$\begin{aligned} F(\alpha_t|\alpha_{t-1}) &\sim N(T\alpha_{t-1}, RQR') \\ f(y_t|\alpha_t, \mathcal{Y}_{t-1}) &\sim N(Z\alpha_t, SHS') \end{aligned}$$

If α_1 is normal, then since α_t's and y_t's are linear combinations of normal errors, the vector, $(\alpha_1, ..., \alpha_T, y_1, ..., y_T)$ is normally distributed. We will use the general fact that if

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim N\left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}\right)$$

then

$$x_1|x_2 \sim N(\tilde{\mu}, \tilde{\Sigma})$$

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

with $\tilde{\mu} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$ and $\tilde{\Sigma} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$

Using this result, we see that each of the distributions in the general steps above are normal. Since the normal distribution is characterized by mean and variance, we need only compute them. Let us introduce the following notation:

$$\begin{aligned}\alpha_t|\mathcal{Y}_{t-1} \sim& N(\alpha_{t|t-1}, P_{t|t-1})\\ \alpha_t|\mathcal{Y}_t \sim& N(\alpha_{t|t}, P_{t|t})\\ y_t|\mathcal{Y}_{t-1} \sim& N(y_{t|t-1}, F_t)\end{aligned}$$

From equation (1), we see that

$$\alpha_{t|t-1} = T\alpha_{t-1|t-1} \tag{3}$$

$$P_{t|t-1} = E\left((\alpha_t - \alpha_{t|t-1})(\alpha_t - \alpha_{t|t-1})'|\mathcal{Y}_{t-1}\right) = TP_{t-1|t-1}T' + RQR' \tag{4}$$

Here we used that $\alpha_t - \alpha_{t|t-1} = T(\alpha_{t-1} - \alpha_{t-1|t-1}) + R\eta_t$. These two equations can be used in step 2 above (prediction equation).

Now, looking at (2), we'll get the equations that we need for step 1.

$$y_{t|t-1} = Z\alpha_{t|t-1} \tag{5}$$

$$F_t = E\left((y_t - y_{t|t-1})(y_t - y_{t|t-1})'|\mathcal{Y}_{t-1}\right) = ZP_{t|t-1}Z' + SHS' \tag{6}$$

Note that so far we have only used the linearity of of (1) and (2).

For the updating step 3, we will need to use normality.

$$\begin{pmatrix} \alpha_t \\ y_t \end{pmatrix}|\mathcal{Y}_{t-1} \sim N\left(\begin{pmatrix} \alpha_{t|t-1} \\ y_{t|t-1} \end{pmatrix}, \begin{pmatrix} P_{t|t-1} & ? \\ ? & F_t \end{pmatrix}\right)$$

where $? = E((\alpha_t - \alpha_{t|t-1})(y_t - y_{t-1|t})'|\mathcal{Y}_{t-1}) = E((\alpha_t - \alpha_{t|t-1})(\alpha_t - \alpha_{t-1|t})'Z'|\mathcal{Y}_{t-1}) = P_{t|t-1}Z'$. We can use this and the general fact about normals to write the posteriority density of α_t given $\mathcal{Y}_t$.

$$\begin{aligned}\alpha_t|\mathcal{Y}_t = \alpha_t|(y_t, \mathcal{Y}_{t-1}) \sim& N(\alpha_{t|t}, P_{t|t})\\ \sim& N(\alpha_{t|t-1} + P_{t|t-1}Z'F_t^{-1}(y_t - y_{t|t-1}), P_{t|t-1} - P_{t|t-1}Z'F_t^{-1}ZP_{t|t-1})\end{aligned} \tag{7}$$

So, starting from some initial $\alpha_{1|0}$ and $P_{1|0}$ we use (5) and (6) to get $y_{1|0}$ and F_1 (the conditional density of y_1). Then using (7), we can get $\alpha_{1|1}$ and $P_{1|1}$. From there, we use (3) and (4) to get $\alpha_{2|1}$ and $P_{2|1}$. Repeating in this way, we can compute the entire likelihood (conditional on the initial conditions). We could just go ahead and use this procedure for computing the likelihood and then estimate the parameters by MLE.

Kalman Smoother

The Kalman filter uses data on the past and current observations, $\mathcal{Y}_t$, to predict α_t. This is what we want for computing the likelihood. However, you might want to estimate α_t. For this, you want to use all the data to predict α_t. This is called the Kalman smoother. The idea is as follows: let

$$E(\alpha_t|\mathcal{Y}_T) = \alpha_{t|T}$$

We know that $(\alpha_t, \alpha_{t+1})|\mathcal{Y}_t$ is normal, so

$$\begin{aligned}E(\alpha_t|\alpha_{t+1}, \mathcal{Y}_t) =& \alpha_{t|t} + E\left((\alpha_t - \alpha_{t|t})(\alpha_{t+1} - \alpha_{t+1|t})'|\mathcal{Y}_t\right)P_{t+1|t+1}^{-1}(\alpha_{t+1} - \alpha_{t+1|t})\\ =& \alpha_{t|t} + J_t(\alpha_{t+1} - \alpha_{t+1|t})\end{aligned}$$

given that $E\left((\alpha_t - \alpha_{t|t})(\alpha_{t+1} - \alpha_{t+1|t})'|\mathcal{Y}_t\right) = E\left((\alpha_t - \alpha_{t|t})(T(\alpha_t - \alpha_{t|t})) + R\eta_{t+1}|\mathcal{Y}_t\right) = P_{t|t}T$ we have $J_t = P_{t|t}T'P_{t+1|t}^{-1}$. So then,

$$E(\alpha_t|\alpha_{t+1}, \mathcal{Y}_t) = \alpha_{t|t} + J_t(\alpha_{t+1} - \alpha_{t+1|t})$$

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

Expectation $E(\alpha_t|\alpha_{t+1}, \mathcal{Y}_T)$ is the same as $E(\alpha_t|\alpha_{t+1}, \mathcal{Y}_t)$, because the knowledge of y_{t+j} for $j > 0$ would be of no added value if we already knew α_{t+1}. That is,

$$E(\alpha_t|\alpha_{t+1}, \mathcal{Y}_T) = \alpha_{t|t} + J_t(\alpha_{t+1} - \alpha_{t+1|t})$$

No we project the results on $\mathcal{Y}_T$ only and use the law of iterated expectations:

$$E(\alpha_t|\mathcal{Y}_T) = \alpha_{t|t} + J_t(\alpha_{t+1|T} - \alpha_{t+1|t})$$

Starting from $t = T$ and repeating in this way, we can compute $\alpha_{T|T}$, $\alpha_{T-1|T}$, ..., $\alpha_{1|T}$.

Things to remember: the Kalman filter and smoother are linear in data. The Kalman filter is a recursive procedure running forward. After that, we can run the Kalman smoother backward.

Summary

For a state-space model,

$$\begin{aligned} y_t &= Z_t\alpha_t + S_t\xi_t \\ \alpha_t &= T_t\alpha_{t-1} + R_t\eta_t \end{aligned}$$

with $\begin{pmatrix} \eta_t \\ \xi_t \end{pmatrix} \sim iid\, N\left(0, \begin{bmatrix} Q & 0 \\ 0 & H \end{bmatrix}\right)$. and the initial observation $y_1 \sim N(y_{1|0}, F_1)$. Kalman filter can be used for the following tasks:

- construct the conditional distribution of y_t and α_t given all past observations $|\mathcal{Y}_{t-1}$. We know that it is normal, Kalman filter calculates the conditional mean $y_{t|t-1}, \alpha_{t|t-1}$ and conditional variances.
- calculate a likelihood of $\{y_1, ..., y_T\}$
- construct the distribution of α given all observations of y, $\alpha_t|\mathcal{Y}_T$ (extract trend- Kalman smoother).
- calculate $y_{t|t-1}$, which is the best linear forecast of y_t given $|\mathcal{Y}_{t-1}$ even if errors are non-normal

What can be cast into state-space form?

We can write many models in state-space form:

Example 3. $AR(p)$:

$$y_t = \sum_{i=1}^{P} \phi_i y_{t-1} + \epsilon_t$$

Then our state equation is

$$\alpha_t = \begin{bmatrix} y_t \\ y_{t-1} \\ \vdots \\ y_{t-p+1} \end{bmatrix} = \begin{bmatrix} \phi_1 & ... & \phi_p \\ 1 & & 0 \\ & \ddots & \\ 0 & & 1 \end{bmatrix} \alpha_{t-1} + \begin{bmatrix} \epsilon_t \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

The measurement equation is

$$y_t = [1\, 0 \ldots 0]\alpha_t$$

For an AR model, it is straightforward to write down the likelihood directly, so there is no need to write down the state-space form or use the Kalman filter. However, for MA and $ARMA$ models, the likelihood is very difficult to evaluate without using the Kalman filter.

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

Example 4. MA Model:

$$y_t = \epsilon_t + \theta\epsilon_{t-1}$$

State equation:

$$\alpha_t = \begin{bmatrix} \epsilon_t \\ \epsilon_{t-1} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \alpha_{t-1} + \begin{bmatrix} \epsilon_t \\ 0 \end{bmatrix}$$

Measurement:

$$y_t = [1\,\theta]\alpha_t$$

Example 5. $ARMA$ Model:

$$y_t = \phi_1 y_{t-1} + \phi_2 y_{t-2} + \epsilon_t + \theta\epsilon_{t-1}$$

State equation:

$$\alpha_t = \begin{bmatrix} y_t \\ \phi_2 y_{t-1} + \theta\epsilon_t \end{bmatrix} = \begin{bmatrix} \phi_1 & 1 \\ \phi_2 & 0 \end{bmatrix} \begin{bmatrix} y_{t-1} \\ \phi_2 y_{t-2} + \theta\epsilon_{t-1} \end{bmatrix} + \begin{bmatrix} 1 \\ \theta \end{bmatrix} \epsilon_t$$

Measurement:

$$y_t = [1\,0]\alpha_t$$

What else can be done by Kalman filter? Error-in-variables

State-space models can also be used to analyze error-in-variable models. Assume that we want to model the behavior the ex ante real interest rate, ξ_t as (for example) an autoregressive process

$$\xi_t = \phi\xi_{t-1} + u_t$$

and want to estimate ϕ and make future forecasts. The problem though is that the theoretical ex ante real interest rate

$$\xi_t = i_t - \pi_t^e$$

is not observable since the expected inflation, π_t^e, is not observable. Instead we observe the realized interest rate, $i_t - \pi_t$ (and the realized inflation). The problem can be written in a state space model with measurement equation:

$$\xi_t^{realized} = i_t - \pi_t = \xi_t - (\pi_t - \pi_t^e) = \xi_t + v_t$$

and state equation:

$$\xi_t = \phi\xi_{t-1} + u_t$$

Since $v_t = \pi_t - \pi_t^e$ is the prediction error, it's natural to assume the iid structure of it.

Missing or unequally spaced Observations

Suppose we have a state-space model where we are missing observations. The model is:

$$\begin{aligned} y_t &= z_t\alpha_t + S_t\xi_t \\ \alpha_t &= T_t\alpha_{t-1} + R_t\eta_t \end{aligned}$$

But instead of observing all $\{y_t\}_{t=1}^T$, we only observe some subset $\{y_{i1}, .., y_{i\tau}\} = \{y_i | i \in I\}$. To produce a likelihood we can consider the following model:

$$\begin{aligned} y_t^* &= z_t^*\alpha_t + w_t^* \\ \alpha_t &= T_t\alpha_{t-1} + R_t\eta_t \end{aligned}$$

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

where $y_t^* = \begin{cases} y_t & t \in I \\ N_t & \text{otherwise} \end{cases}$, $z_t^* = \begin{cases} z_t & t \in I \\ 0 & \text{otherwise} \end{cases}$, and $w_t^* = \begin{cases} w_t & t \in I \\ N_t & \text{otherwise} \end{cases}$, with $N_t \sim iid\, N(0,1)$. Assume that we observe $y_t^* = 0$ when $t \notin I$, then the conditional likelihood is

$$f(y_t^*|\mathcal{Y}_{t-1}^*) = \begin{cases} \phi(0) & t \notin I \\ f(y_t|\{y_s : s \in I,\, s < t\}) \end{cases}$$

and the likelihood is $f(y_1^*, ..., y_T^*) = \phi(0)^{\#I^c} f(y_{i1}, ..., y_{i\tau}) = (2\pi)^{-\frac{\#I_c}{2}} f(y_{i1}, ..., y_{i\tau})$. It means that we can run Kalman filter for y_t^*, it produces(up to a constant) a valid likelihood for the initial model.

Example 6. Suppose:

$$y_t = \phi y_{t-1} + \epsilon_t$$

and we observe $t \in I = \{1, 3, 4, 5\}$. In state space form, as above, we have:

$$\alpha_t = \alpha_{t-1}\phi + \epsilon_t$$
$$y_t^* = z_t \alpha_t + w_t$$

where $z_t = \begin{cases} 1 & t \in I \\ 0 & \text{otherwise} \end{cases}$ and $w_t = \begin{cases} 0 & t \in I \\ N(0,1) & \text{otherwise} \end{cases}$. Let's think what Kalman filter would do: for $t \in I$ we observe $y_t = \alpha_t$, so our best linear predictor of α_t is y_t. For $t = 2$, y_2 is unrelated to α_2, so our best linear predictor of α_2 is just ϕy_1. Then the conditional means used in the Kalman filter are:

$$\alpha_{t|t} = \begin{cases} y_t & t \in I \\ \phi y_{t-1} & t = 2 \end{cases}$$

To form the conditional likelihood, we need the distribution of $y_t|\mathcal{Y}_{t-1}$, which has mean

$$y_{t|t-1}^* = \phi \alpha_{t-1|t-1} = \begin{cases} \phi y_{t-1} & t \in \{4, 5\} \\ \phi^2 y_1 & t = 3 \end{cases}$$

and variance

$$F_t = \begin{cases} \sigma^2 & t \in \{4, 5\} \\ (1 + \phi^2)\sigma^2 & t = 3 \end{cases}$$

We only need the conditional distribution at $t = 3, 4, 5$ because the likelihood is:

$$f(y_1, y_3, y_4, y_5) = f(y_1) f(y_3|y_1) f(y_4|y_3, y_1) f(y_5|y_4, y_3, y_1)$$

and the conditional (on y_1) likelihood is

$$\begin{aligned} f(y_1, y_3, y_4, y_5|y_1) =& f(y_3|y_1) f(y_4|y_3, y_1) f(y_5|y_4, y_3, y_1) \\ =& \frac{1}{\sigma\sqrt{1+\phi^2}} \phi\left(\frac{y_3 - \phi^2 y_1}{\sigma\sqrt{1+\phi^2}}\right) \frac{1}{\sigma}\phi\left(\frac{y_4 - \phi y_3}{\sigma}\right) \frac{1}{\sigma}\phi\left(\frac{y_5 - \phi y_4}{\sigma}\right) \end{aligned}$$

where $\phi(\cdot)$ is the normal pdf.

The nice thing is that Kalman filter does this reasoning automatically.

Cite as: Anna Mikusheva, course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare (http://ocw.mit.edu), Massachusetts Institute of Technology. Downloaded on [DD Month YYYY].

MIT OpenCourseWare
http://ocw.mit.edu

14.384 Time Series Analysis

Fall 2013

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.